THE EDITORIAL BIBLE

SpaceTalk

The constitution of the product — vision, brand, design, experience, intelligence, architecture and roadmap.

VERSION 1.0 · RATIFIED EDITION

FOURTEEN PARTS · TWENTY FIGURES · TWELVE DECISION RECORDS

Room to talk.

THE EDITORIAL BIBLE

SpaceTalk

The constitution of the product. Every design, engineering, brand and business decision traces back to this document.

Version 1.0 · Ratified edition · Fourteen parts

Prepared for executives, investors, designers, engineers, and everyone who joins the project after us.

Copyright and colophon

SpaceTalk Editorial Bible, Version 1.0. Ratified edition.

AUTHORITY

This document supersedes all prior product, brand and architecture guidance. Where any roadmap, ticket, mockup, specification or opinion conflicts with it, this document prevails until it is formally amended. Amendments require a written rationale recorded in Part 0 §0.10 and sign-off from the CEO, CPO and CDO.

COLOPHON

Set in Inter and Inter Display, designed by Rasmus Andersson, with JetBrains Mono for code, values and safety numbers. Inter is the interface typeface specified in Part 2 §2.13, so the manual is set in the same face as the product it governs.

Composed at US Letter, 6.30 inch measure, on a 4-point vertical rhythm. Colour throughout is drawn only from the palette defined in Part 2; every contrast ratio quoted in this document was computed against the WCAG relative-luminance formula rather than estimated.

Twenty figures were drawn as vectors specifically for this edition and rasterised at approximately 430 dots per inch for print.

A NOTE ON STATUS

Everything in this document is specification. At the time of this edition no SpaceTalk application code has been written; the numbers in Part 8 are budgets to be defended, not measurements taken. Where a claim rests on evidence we do not yet have, the text says so.

Document control

Who owns this document, what governs a change to it, and how it is reviewed.

Field	Value
Document title	SpaceTalk Editorial Bible
Version	1.0 — ratified edition
Status	Active. Governing document.
Classification	Internal — confidential
Document owner	Office of the CEO
Approvers	Chief Executive Officer · Chief Product Officer · Chief Design Officer
Contributing authorities	CTO · Principal Mobile Engineer · Principal Backend Architect · AI Systems Architect · Security Architect · Principal UX Researcher · Design Systems Lead · Brand Director · Growth Strategist
Scope	All SpaceTalk product, design, brand, engineering and business decisions
Supersedes	All prior product, brand and architecture guidance
Review cadence	Quarterly, and on any Part 0 §0.10 amendment
Amendment procedure	Written rationale appended to Part 0 §0.10, with CEO, CPO and CDO sign-off
Source of record	docs/spacetalk/ — fifteen Markdown documents under version control
Composition	14 parts · 179 headings · 70 tables · 20 figures · 12 decision records
Formats	DOCX and PDF, generated from one source; content is identical by construction

Both editions are generated from the same parsed source. The Word and PDF editions are not produced independently and then compared — they are rendered from a single intermediate representation of the Markdown source, so their content cannot drift apart. The PDF is produced from the DOCX itself.

Version history

Every change to a governing document is recorded. Nothing is edited silently.

Version	Scope of change	Authority
1.0	Founding ratified edition. Establishes Parts 0–13: the constitution and its ten non-negotiables, the brand system, the visual design system with measured contrast, the UX bible, the AI philosophy, thirteen MVP feature specifications, the technical architecture, the design system, performance budgets, the five-phase roadmap, the scope register, business and compliance, twelve architecture decision records, and the research programme with journeys and information architecture.	CEO · CPO · CDO
1.0	Publication edition. Typeset for print and distribution: cover, front matter, twenty commissioned figures, cross-references, index. **No change to the governed content.**	CDO

PENDING AMENDMENTS

One question is open and is recorded in Part 12, ADR-011. SpaceTalk is treated throughout this document as a new product line, distinct from the StromeX AI knowledge-work product that shares its repository. If SpaceTalk was instead intended as a rename of StromeX, ADR-011 is wrong and must be superseded — beginning with Part 0 §0.1, because the two products have different missions, different users and different architectures.

AMENDMENT RECORD

Part 0 §0.10 carries the authoritative amendment table. It is empty at version 1.0 beyond the founding ratification, which is the correct state for a document that has not yet been contradicted by reality.

How to use this document

Fourteen parts, four reading paths, one rule about precedence.

THE RULE ABOUT PRECEDENCE

Part 0 is the constitution. Parts 1–13 elaborate it. If any of them appears to contradict Part 0, Part 0 wins and the contradiction is a defect to be fixed. If this document contradicts a decision made in a meeting, this document wins until it is amended.

READING PATHS

Situation	Read
If you have twenty minutes	Executive summary, then Part 0 (the constitution), then Part 10 §10.1 (why the scope is what it is).
If you are joining as a designer	Parts 1, 2, 3 and 7 in order, then Part 13 for the journeys the interface has to serve.
If you are joining as an engineer	Part 6, then Part 12 (the decision records — they explain why Part 6 looks the way it does), then Part 8 for the budgets you will be held to.
If you are evaluating the business	Executive summary, Part 11, Part 9, then Part 10 for what was deliberately declined and why.
If you are about to propose a feature	Part 10 §10.12 first. It asks six questions, and most proposals do not survive the fourth.
If you are about to break a rule	Part 0 §0.6. Ten clauses may not be traded away for growth, revenue, a deadline, or a competitor’s launch.

CONVENTIONS

Convention	Meaning
Section numbers	Numbering matches the source documents exactly. A reference to §6.8 means Part 6, section 6.8, everywhere in this document.
Cross-references	Blue text is a live link. References of the form “Part 6 §6.8” and “ADR-003” jump to their target in both the Word and PDF editions.
Figures	Numbered by part — Figure 6.2 is the second figure in Part 6. All twenty are listed in the contents.

Convention	Meaning
Callout panels	A tinted panel marks a passage that changes what you should do, not merely what you should know.
Measured values	Every contrast ratio, every budget and every threshold is a specific number. Where a number is a target rather than a measurement, the text says so.

Executive summary

SpaceTalk is a communication space that is fast, quiet and private, in which intelligence is ambient, invited and accountable.

Messaging is the highest-traffic software on earth, and almost all of it has drifted away from its purpose. Feeds, stores, badge counts and discovery surfaces have accumulated around the act of sending a message to someone you care about. The interfaces have grown louder while the conversations have not grown better. Separately, a real capability arrived — models that translate, summarise, transcribe and detect fraud in real time — and it has been bolted on as a chatbot in a tab.

SpaceTalk is the correction to both drifts. It is allowed to be known for exactly three things: the fastest messaging experience in the world, the most useful intelligence ever integrated into communication, and the cleanest interface in the category. Every proposal is tested against those three, and one that strengthens none of them is rejected regardless of merit.

THE FIVE DECISIONS EVERYTHING ELSE FOLLOWS FROM

Decision	Where	Consequence accepted
No advertising, ever	ADR-009	Which is why there is no feed, no ranking model and no profiling pipeline. Those absences are load-bearing, not incidental. Revenue comes from subscriptions, a business platform and a 10 % creator take rate — deliberately below the market's 30 %, because we do not have an advertising business to fund.
AI runs on the device by default	ADR-005	It never touches end-to-end encrypted content without a visible, revocable, per-conversation grant. Target: over 85 % of AI invocations on-device. This is the privacy promise made measurable, and it also makes the largest cost line in a modern AI product close to zero.
Delivered messages are deleted from the server	ADR-004	The server is a relay, not an archive. A privacy decision that turns out to be what makes the economics work: the hot dataset scales with undelivered traffic, not with total history.
No address-book upload	ADR-010	The largest deliberate growth cost in the product, taken with open eyes. Hashed phone numbers leak; we decline to ship a discovery system we cannot make private. Growth comes instead from share links, channels and language-led market entry.
Performance budgets are release gates	Part 8 §8.11	Not dashboards, not tickets. Cold start, frame rate, memory and battery are measured in CI on physical hardware, per pull request, per device tier. A regression fails the build.

THE PRODUCT

The first public version ships thirteen features and nothing else: secure messaging, voice notes, voice calls, video calls, group conversations, channels, stories, an AI assistant, file sharing, search, a profile system, notifications and multi-device support. Each is specified in Part 5 to purpose, user problem, success metrics, interface behaviour, edge cases, failure cases and future roadmap.

The founding brief for this project asked for every feature of seven competing products, plus payments, healthcare, education, commerce and a creator studio — roughly ninety proposals. Part 10 triages all of them: thirteen in the MVP, fifty-one scheduled across four later phases, twenty-six rejected permanently with the reasoning recorded so it does not have to be re-derived under pressure. Nothing was silently dropped, including what we declined.

THE STACK

Flutter on every client surface. A modular monolith in Go — four deployables, not twelve microservices. PostgreSQL, Redis and object storage. The Signal Protocol by way of libsignal, never a cryptographic implementation of our own. A local-first SQLite store with a durable outbox, so the device is the source of truth and the network is a synchronisation mechanism. A LiveKit SFU with insertable-stream encryption for group calls, so the media server forwards frames it cannot decrypt.

WHAT WOULD MAKE THIS FAIL

Part 11 §11.10 carries the full risk register. The two entries that matter most are not technical. The first is network effects: nobody switches messenger alone, and ADR-010 makes that harder on purpose. The second is dilution — thirty small, individually reasonable compromises over four years, at the end of which the product is another loud messenger with an AI tab. No competitor will beat SpaceTalk in a way that shows up on a dashboard. That is what the constitution in Part 0 exists to prevent, and it only works if people actually invoke it.

> **The standards do not scale with the company.** Cold start under 1,500 ms on entry-level hardware, end-to-end encryption by default, zero AI processing without an explicit grant, zero notifications from non-humans, four primary navigation destinations, and no advertising — these are identical at Phase 1 and Phase 5. The infrastructure required to hold them changes enormously. The standards themselves do not change at all. That is what makes them standards.

Contents

Every entry is a live link. Section numbers match the source documents exactly, so a reference in one part resolves to the same number in another.

FRONT MATTER

THE FOURTEEN PARTS

BACK MATTER

FIGURES

PART 00

00

The Constitution

Part 0

Version 1.0. This document governs every design, engineering, brand, and business decision made about SpaceTalk. Where any other document, roadmap, ticket, mockup, or opinion conflicts with this one, this one wins until it is formally amended. Amendments require a written rationale appended to Part 0.10 and sign-off from the CEO, CPO, and CDO.

> "The hardest work in this product is deletion. Every feature we do not ship is a feature nobody has to learn."

IN THIS PART

0.1 Purpose

SpaceTalk exists to give people back the feeling of talking to someone.

Messaging apps are now the highest-traffic software on earth, and almost all of them have drifted away from that purpose. They have accumulated feeds, stores, games, discovery tabs, badge counts, and engagement machinery until the act of sending a message to a person you care about is surrounded by things designed to keep you from leaving. The interfaces have grown louder while the conversations have not grown better.

At the same time, a genuine capability arrived — language models that can translate, summarise, recall, transcribe, and detect fraud in real time — and it has been bolted onto those products as a chatbot in a tab, or as a mascot that interrupts. Nobody has yet built a communication product where intelligence is *part of the medium* rather than a passenger inside it.

SpaceTalk is the correction to both drifts. It is a communication space that is fast, quiet, and private, in which intelligence is ambient, invited, and accountable.

0.2 Mission

To build the fastest, calmest, most trustworthy way for people to talk — and to make useful intelligence a native property of conversation rather than a product bolted onto it.

0.3 Long-Term Vision

By 2036, SpaceTalk is the default communication layer for hundreds of millions of people who chose it deliberately — not because their family was already there, but because using anything else feels like going back to a slower, noisier machine. The language barrier is, in practice, gone inside SpaceTalk. Fraud and impersonation are meaningfully harder here than anywhere else. The app has not grown a feed, and it has not grown an advertising business.

The proof of success is negative as much as positive: in 2036 SpaceTalk still opens in under a second, still has fewer than five primary destinations, and a person who learned it in 2027 still knows where everything is.

0.4 Values

Value	What it means in practice
Calm	The product never manufactures urgency. No engagement notifications, no streaks, no “someone you may know,” no red badge for anything that isn’t addressed to you.
Speed as respect	Latency is a moral property, not a technical one. Every millisecond we spend is a millisecond of someone’s life.
Privacy by construction	We design so that we *cannot* read what we promise not to read. Policy is a weaker guarantee than architecture, and we prefer the stronger one.

Value	What it means in practice
Invited intelligence	AI acts when asked, or when it has explicit standing permission. It never inserts itself into a conversation on its own initiative.
Legibility	A user should always be able to answer: what just happened, who can see this, and why am I seeing it.
Restraint	The hardest work in this product is deletion. Every feature we do not ship is a feature nobody has to learn.
Craft	Nothing ships that we would not be happy to have screenshotted and compared, pixel for pixel, against the best software in the world.

0.5 Design Philosophy

Content is the interface. Chrome recedes; messages, faces, and voices are the only things allowed to be visually loud.

Typography before decoration. Hierarchy is established with type, weight, and space. Colour is used to encode meaning — state, identity, danger — not to entertain.

Motion explains, never performs. Every animation answers "where did this come from and where did it go." An animation that does not explain something is deleted.

One-handed by default. The primary market holds the phone in one hand, often while doing something else. Anything a user does more than five times a day must be reachable by a thumb.

Dark mode is not a theme, it is a first-class design. Both modes are designed, not derived. Neither is an inversion of the other.

The same product on every screen. A user who learns SpaceTalk on a 5-inch Android phone should not have to relearn it on a desktop. Layouts adapt; concepts do not.

0.6 Non-Negotiable Principles

> These are the clauses that may not be traded away for growth, revenue, a deadline, or a competitor's feature launch. Violating one of these is grounds for reverting a release.

1. **No advertising business, ever.** Not banner ads, not sponsored messages, not "promoted" contacts, not a discovery surface sold to brands. The moment attention is the product, calm is impossible. Revenue comes from subscriptions and from businesses paying to use the platform (see Part 11).
2. **No algorithmic feed of strangers.** SpaceTalk shows you what you subscribed to and who talked to you, in the order it happened. There is no engagement-ranked timeline of people you did not choose.

3. **Personal messages are end-to-end encrypted by default and cannot be silently downgraded.** There is no "off" switch we can flip server-side, and no key-escrow backdoor for any party, including us.
4. **AI never reads an end-to-end encrypted conversation without an explicit, revocable, per-conversation grant from the user, shown in the interface.** Silent processing of private content is a firing offence, not a bug.
5. **Notifications are for humans addressing you.** The system may never notify a user to drive re-engagement.
6. **No dark patterns.** No confirm-shaming, no hidden unsubscribe, no fake scarcity, no interstitial upsell between a user and a message.
7. **Data minimisation.** We do not collect metadata we do not operationally need, and we delete what we no longer need on a published schedule.
8. **No feature ships that regresses cold-start, frame rate, or crash-free-session targets** (Part 8). Performance is a release gate, not a follow-up ticket.
9. **Accessibility is a launch requirement.** WCAG 2.2 AA and platform screen-reader parity for every shipped surface — not a Phase 2 item.
10. **The user owns their data and can export and delete all of it**, in a machine-readable format, without contacting support.

0.7 Identity: The Only Three Things

SpaceTalk is allowed to be known for exactly three things. Every roadmap review asks which of the three a proposal strengthens; a proposal that strengthens none of them is rejected regardless of merit.

1. **The fastest messaging experience in the world.** Measured, published, and defended (see Part 8).
2. **The most useful intelligence ever integrated into communication** — translation, recall, summarisation, and fraud protection that work inside the conversation.
3. **The cleanest, most enjoyable interface in the category.** Learnable in ninety seconds, still elegant after five years of feature pressure.

Everything else — stories, channels, files, profiles — is *supporting cast*. Supporting cast is held to a lower ambition ceiling and a higher deletion risk. This is deliberate.

0.8 The MVP Boundary

The first public version ships exactly this, and nothing else:

Secure messaging · Voice notes · Voice calls · Video calls · Group conversations · Channels · Stories · AI assistant · File sharing · Search · Profile system · Notifications · Multi-device support.

Feature specifications are in Part 5. The full triage of every rejected and deferred idea — including the large "build everything from every app" ambition that this project began with — is in Part 10. The scope register is a permanent, maintained document; ideas are not silently dropped, they are recorded with a reason and a phase.

0.9 Decision Rules

> When a decision is genuinely close, apply these in order. They are ordered; a higher rule overrides a lower one.

FIGURE — THE DECISION RULES, IN PRIORITY ORDER

When a decision is genuinely close, apply these in order. A higher rule overrides a lower one.

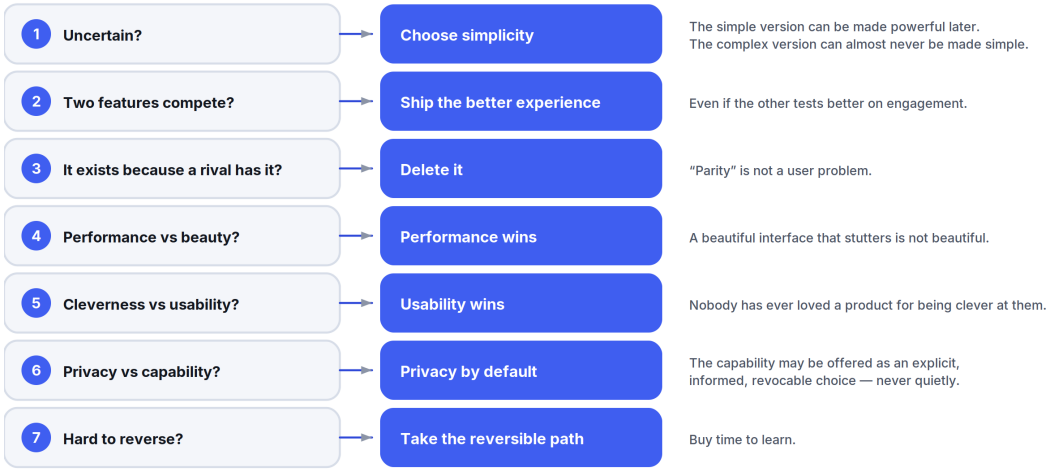


Figure 0.1 Part 0 §0.9 — an ordered ladder, so a close call is resolved by rule rather than by whoever argues longest.

1. **When uncertain, choose simplicity.** The simpler version can be made powerful later. The complex version can almost never be made simple later.
2. **When two features compete, ship the one with the better experience**, even if the other tests better on engagement.
3. **When a feature exists only because a competitor has it, delete it.** "Parity" is not a user problem.
4. **When performance and beauty conflict, performance wins.** A beautiful interface that stutters is not beautiful.
5. **When cleverness and usability conflict, usability wins.** Nobody has ever loved a product for being clever at them.
6. **When privacy and capability conflict, privacy wins by default — and the capability may be offered as an explicit, informed, revocable choice.** We do not decide for the user, and we do not decide *quietly*.
7. **When a decision cannot be reversed cheaply, take the reversible path** and buy time to learn.

0.10 Amendments

#	Date	Clause	Change	Rationale	Approved by
—	—	—	Ratified v1.0	Founding document	CEO / CPO / CDO

Document map. Part 1 Brand · Part 2 Visual Design System · Part 3 UX · Part 4 AI Philosophy · Part 5 Features · Part 6 Technical · Part 7 Design System · Part 8 Performance · Part 9 Roadmap · Part 10 Scope Governance · Part 11 Business & Compliance · Part 12 ADRs.

PART 01

01

The Brand Bible

Part 1 — Identity, Voice, and Craft

Governed by Part 0. Brand decisions that conflict with Part 0.6 are void.

> "The word space in this product means room, not outer space."

IN THIS PART

1.1 The Name

SpaceTalk.

The word *space* in this product means **room**, not **outer space**.

Room to think before replying. Room between messages. Room on the screen. A space that belongs to two people, or to forty. The name is a promise about generosity and quiet, and every brand expression must honour that reading.

> **This is the single most-violated brand rule, so it is stated first and absolutely:**

> **No rockets. No planets. No stars, orbits, satellites, astronauts, galaxies, nebulae, constellations, or purple-on-black cosmic gradients. Ever.**

The cosmic reading is the lazy reading. It is also the one every competitor's design language already looks like. If a design could be reused unchanged by a crypto exchange or a sci-fi game, it is wrong.

Tagline: *Room to talk.*

Secondary lines, used sparingly and never stacked: *Fast. Quiet. Yours.* · *Conversation, with room to breathe.*

1.2 Brand Personality

SpaceTalk is a **quiet, exceptionally competent host**.

Dimension	We are	We are not
Temperament	Composed, unhurried	Excitable, chirpy
Intelligence	Evidently capable, never showing off	Clever at the user's expense
Warmth	Genuinely warm, understated	Cloying, over-familiar, emoji-laden
Authority	Precise, admits limits	Certain about things it can't know
Humour	Dry, rare, always at our own expense	Quirky, meme-chasing, mascot-driven
Aesthetic	Timeless, considered	Trendy, maximal, skeuomorphic

If SpaceTalk were a person: the friend who answers in one accurate sentence, remembers what you told them last month, never forwards you a chain message, and does not perform.

1.3 Brand Voice

Principles.

1. **Say the thing.** Lead with the outcome, not the preamble.
2. **Second person, active voice, present tense.** “Your message will send when you’re back online.” Not “Message delivery has been deferred.”
3. **Short sentences.** If a sentence needs a comma to survive, it probably needs a full stop instead.
4. **No exclamation marks.** One exception exists in the entire product: nothing. There is no exception.
5. **Never apologise theatrically.** “Couldn’t send” — not “Oops! Something went wrong :(“.
6. **Never blame the user.** Not “Invalid input.” Say what will work: “Phone numbers need a country code.”
7. **No jargon that leaks the implementation.** Users do not have “sessions,” “tokens,” “sync conflicts,” or “payloads.”
8. **Never use fear or FOMO.** No “Don’t miss out.” No “3 friends are waiting.”
9. **Numbers over adjectives.** “Encrypted end-to-end” beats “military-grade security” — and “military-grade” is banned outright as a meaningless claim.
10. **Localisation-first phrasing.** Avoid idioms, puns, and constructions that require English word order to work.

Vocabulary.

Use	Never use
Chat, conversation	Thread (except in Channels), DM
Space (a group’s home)	Server, room, guild
Channel	Broadcast list, feed
Story	Status, Moment, Fleet
Assistant	Bot, AI buddy, copilot, mascot name
Linked device	Companion, satellite
Ended-to-end encrypted	Military-grade, unhackable, NSA-proof
Send	Blast, shoot, ping (as a verb for messaging)

Capitalisation. Sentence case everywhere — buttons, titles, menus, notifications. Title Case is reserved for the product name and proper nouns. This is not a style preference; sentence case measurably reads faster and localises better.

Punctuation. Oxford comma: yes. Em dashes: sparingly, spaced en dashes are not used. Ellipses only for genuine truncation, never for hesitation.

Voice in practice

Situation	Wrong	Right
No network	"Oops! You appear to be offline."	"You're offline. Messages will send when you reconnect."
Failed call	"Call failed!"	"Couldn't connect. Try again?"
Empty inbox	"No chats yet! Start one now!"	"No conversations yet. Search for someone, or share your link."
AI uncertain	"I think it's probably X."	"Best guess: X. I'm not confident — worth checking."
Scam warning	"⚠ DANGER! This may be a SCAM!!"	"This message asks for a payment code. SpaceTalk never asks for one."
Upgrade prompt	"Unlock the full experience!"	"Files over 2 GB need SpaceTalk Plus."
Permission ask	"Allow notifications for the best experience"	"Turn on notifications so you know when someone replies."

1.4 Naming Rules

- **Features get plain nouns.** Stories. Channels. Spaces. Calls. If a name needs explaining, it is the wrong name.
- **No invented capitalised nouns for ordinary things.** We do not have "Beams," "Pods," "Vibes," or "Signals."
- **The assistant has no personal name and no persona.** It is "the assistant" or "SpaceTalk assistant." Giving it a name invites anthropomorphism, which invites trust it has not earned. This is a Part 0.6 adjacent rule, not a style choice.
- **Subscription tiers:** SpaceTalk (free), **SpaceTalk Plus**, **SpaceTalk Business**. No "Pro," "Premium," "Ultra," "Max."
- **Internal codenames never leak to the UI.** Ever.

1.5 Logo Philosophy

The mark is a **counter-form** — the shape of the space *between* two people talking, not a picture of talking.

Construction. A single soft-cornered aperture: two overlapping rounded forms whose intersection creates a lens-shaped void at the centre. The void is the subject. It reads as an opening, a doorway, or a shared area — depending on scale — and never as a speech bubble with a tail.

Rules.

- Built on a 24-unit grid; all radii are multiples of 2 units so the mark scales cleanly to a 16 px favicon and a 1024 px app icon from one source.
- Monochrome-first. The mark must be fully legible in single-colour black, single-colour white, and 1-bit. Colour versions are derived from the monochrome master, never the reverse.
- Minimum clear space on all sides = the height of the central void.
- Minimum size: 16 px (mark only), 96 px (mark + wordmark).
- **Never:** rotate, add a gradient across the mark, add a drop shadow, outline it, animate it as a loading spinner, place it on a busy photograph, or use it as a message-bubble avatar.
- The app icon is the mark on a flat surface. It carries no gloss, no bevel, no inner glow, and no gradient beyond a maximum 6 % luminance shift.

Wordmark. Set in the brand display face at a tightened tracking of −2 %; the “S” and “T” are the only characters that may be optically adjusted. Lowercase “spacetalk” is never used as an official wordmark.

1.6 Photography Direction

Photography exists in marketing and in onboarding illustration-adjacent moments. It never appears as decoration inside the app.

- **Real people in real rooms.** Available light, no studio strobes, no white cyclorama.
- **Candid over posed.** People mid-conversation, mid-laugh, mid-thought — not smiling at the lens holding a phone up like a product.
- **The phone is usually not the subject** and is often not visible. We photograph the relationship, not the device.
- **Global by default.** Casting, interiors, clothing, and languages must reflect the actual user base, not a Californian one. Every campaign is reviewed for regional plausibility by someone from that region.
- **Colour treatment:** slightly desaturated, warm shadow tint, no heavy grading, no teal-and-orange. Grain acceptable, filters not.
- **Never:** stock-photo handshakes, over-the-shoulder screen shots with fake UI, people in bed lit by phone glow, cosmic double-exposures.

1.7 Illustration Style

Illustration carries empty states, onboarding, error states, and education.

- **Geometric, flat, two-to-three colours plus a neutral.** Built from the same radii family as the logo.
- **Line weight is uniform** within a piece and never below 1.5 px at rendered size.
- **No faces.** People are represented as simple forms without features — this scales globally, ages well, and avoids representing a demographic by accident.
- **No perspective, no gradients-as-shading, no 3-D renders, no isometric business scenes.**
- **Each illustration must communicate one idea.** If it needs a caption to be understood, it fails; if it needs *no* caption at all, it is probably decorative and should be deleted.
- Illustrations ship as SVG, are theme-aware (two colour sets), and must be legible at 120 px height.

1.8 Icon Style

- **Grid:** 24 × 24, 1.5 px stroke, rounded caps and joins, 2 px corner radius on rectangular forms.
- **Outline by default; filled for the active state** in navigation. Never mix outline and filled within one row.
- **Optical, not mathematical, alignment.** A triangle is centred by weight, not by bounding box.
- **One metaphor per concept, product-wide.** A magnifier is always search and never zoom.
- **No brand-specific glyphs for universal actions** — we do not reinvent the share, back, or delete icon.
- Icons never carry colour of their own; they inherit text colour, except for semantic status icons (Part 2 §2.4).
- Every icon has a text alternative for screen readers. An icon-only button without a label is an accessibility defect.

1.9 Animation Philosophy

Motion is the grammar that tells the user where things came from and where they went. Detailed timing tokens live in Part 7 §7.7; the philosophy is here.

1. **Every animation must explain a spatial or causal relationship.** If you can remove it and lose no meaning, remove it.
2. **Things come from where they are going back to.** A sheet rises from the button that opened it and returns there.

3. **Fast in, slower out.** Responses to a user's touch are near-instant (≤120 ms); things leaving may take longer (up to 260 ms).
4. **Nothing bounces.** No spring overshoot on primary UI. Overshoot reads as playful the first time and as sloppy the hundredth.
5. **Never animate to fill time.** Skeletons and progress indicators are honest reports of state, not entertainment.
6. **Interruptibility is mandatory.** Any animation can be reversed or cut through by a new gesture at any frame. A user must never wait for an animation to finish.
7. **Respect `prefers-reduced-motion`.** With it on, position/scale transitions become cross-fades of ≤100 ms; parallax and any looping motion stop entirely. This is a hard requirement, not an option.
8. **60 fps is the floor, 120 fps is the target on capable displays.** An animation that cannot hold frame rate is simplified until it can (Part 0.9 rule 4).

1.10 Brand Anti-Patterns

A list of things that will look tempting and are permanently banned:

- A mascot of any kind.
- Confetti, celebration bursts, haptic "achievements."
- Gamified streaks, levels, or badges.
- Neon glow, glassmorphism as decoration, "AI shimmer" gradients on non-AI surfaces.
- Countdown timers used for urgency.
- Auto-playing sound.
- Copy that refers to the assistant as "he," "she," or "your new best friend."
- Announcing a feature as "revolutionary," "magical," or "game-changing."
- Comparing ourselves to competitors by name in product copy.

PART 02

02

The Visual Design System

Part 2 — Colour, Type, Space, Surface

Governed by Part 0 and Part 1. Every colour below states its HEX value, its job, and why it exists. Every contrast ratio quoted was computed against the WCAG 2.x relative-luminance formula and is accurate to two decimal places.

> "Human content and machine content never wear the same colour."

IN THIS PART

Naming. Palette families are named for what they do, not for what they look like: **Orbit** (the brand action colour), **Aurora** (the intelligence colour), **Void** (the neutral scale). These names are internal tokens only — see Part 1 §1.1 for the ban on cosmic *imagery*; a token name is not imagery.

2.1 Colour Doctrine

Four rules that determine every colour decision:

1. **Colour encodes meaning, never mood.** If a colour is not communicating identity, state, or hierarchy, it should be a neutral.
2. **The interface is 90 % neutral.** Brand colour appears on roughly one element per screen. A screen with three coloured things has one too many.
3. **Human content and machine content never wear the same colour.** Orbit is the user's colour. Aurora belongs exclusively to the assistant. A user must be able to tell, at a glance and without reading, whether a person or a model produced something. This is a trust mechanism, not a decoration scheme.
4. **Contrast is verified, not eyeballed.** Every token pair used for text ships with a measured ratio. Anything below 4.5:1 is barred from body text, no exceptions for "it looks fine."

2.2 Primary Palette: Orbit

A blue with a slight violet cast (hue ≈ 231°). Pure blue at this saturation reads corporate and is the single most-used hue in software; the violet lean keeps it distinct from iOS system blue (`#007AFF`), Telegram (`#2AABEE`), and Meta blue (`#0866FF`) at a glance, without drifting into the purple that AI products have colonised.

Token	HEX	Why it exists
`orbit-50`	`#EEF2FF`	Tint background for selected rows and information banners in light mode.
`orbit-100`	`#DDE4FF`	**Outgoing message bubble, light mode.** Light enough to carry `void-900` text at **14.33:1** — the highest-traffic surface in the product had to be the most readable.
`orbit-200`	`#BCC9FF`	Hover/pressed state of `orbit-100`; disabled primary button fill.
`orbit-300`	`#92A7FF`	**Primary action text and icons in dark mode** — **7.93:1** on `void-900`.
`orbit-400`	`#6681FB`	Secondary emphasis in dark mode; **5.26:1** on `void-900`. Links inside dark bubbles.
`orbit-500`	`#3F5EF0`	**The brand colour.** Primary button fill and focus ring in light mode. White text on it: **5.16:1**.

Token	HEX	Why it exists
`orbit-600`	`#2E48D8`	Pressed state of `orbit-500`; primary *text* colour on white where 5.16 is too tight (**6.92:1**).
`orbit-700`	`#2438AE`	Link text on light tinted surfaces — **8.34:1** on `orbit-50`.
`orbit-800`	`#1E2F8A`	**Outgoing message bubble, dark mode.** `void-50` text on it: **10.61:1**.
`orbit-900`	`#1A2769`	Deep accent for pressed dark-mode bubbles and large brand fields.
`orbit-950`	`#101739`	Rare — the darkest brand field, used behind full-bleed onboarding art only.

> **Why one accent and not two.** Every additional brand hue doubles the number of decisions a designer makes per screen and halves the meaning each hue carries. One action colour makes "what is the primary action here?" answerable without thought.

2.3 Secondary Palette: Aurora (the intelligence colour)

A green-cyan (hue ≈ 170°). Deliberately far from Orbit on the wheel so the two never read as variants of each other, and deliberately *not* the purple/magenta that the industry has made synonymous with generative AI — we are signalling *useful and calm*, not *magical*.

Aurora may only be used on assistant output, assistant affordances, and translation/transcription indicators. Using Aurora anywhere else is a design defect.

Token	HEX	Why it exists
`aurora-300`	`#5FE3CE`	Assistant text/icons in dark mode — **11.53:1** on `void-900`.
`aurora-400`	`#2ACFB6`	Assistant icon fills, dark mode (**9.23:1**).
`aurora-500`	`#12B39B`	**Fill only** — assistant avatar, indicator dots. **2.64:1** on white, so it is barred from text and from any icon that carries meaning alone.
`aurora-700`	`#0A7466`	**Assistant text and icons in light mode** — **5.67:1** on white, **5.24:1** on `void-50`.
`aurora-800`	`#08594E`	Assistant text on tinted light surfaces (**8.24:1** on white).
`aurora-050`	`#E6FAF6`	Assistant response background in light mode; keeps AI content visually separable without a border.

Token	HEX	Why it exists
aurora-950	#062F2A	Assistant response background in dark mode.

2.4 Semantic Colours

Each has a light-mode value (on white) and a dark-mode value (on void-900). Every value listed for text meets 4.5:1 minimum; fill-only values are labelled.

Success

Token	HEX	Ratio	Use
success-fill	#12A150	3.37:1 on white — **non-text only**	Delivered/read ticks, connection-restored dot, toggle-on fill.
success-text	#08602F	**7.71:1** on white	"Sent," "Verified," success banner text. White on it: **7.71:1**; on void-50: **7.12:1**.
success-dark	#35D07F	**9.04:1** on void-900	Both text and fill in dark mode.
success-surface	#E8F7EE / #0B2C1B	—	Banner backgrounds, light / dark.

Why a separate fill and text value: the green that reads correctly as a status dot is too light to read as text. Rather than compromise both, we specify both.

Warning

Token	HEX	Ratio	Use
warning-text	#B45309	**5.02:1** on white; white on it **5.02:1**	"Message not delivered," storage nearly full, unverified device.
warning-dark	#FFC46B	**11.53:1** on void-900	Dark-mode equivalent. void-950 on it: **12.37:1**; on void-850: **10.50:1**.
warning-surface	#FEF4E6 / #301F05	—	Banner backgrounds.

Amber rather than yellow: true yellow cannot reach 4.5:1 on white at any saturation worth having, and warning states must be readable, not merely visible.

Danger

Token	HEX	Ratio	Use
`danger`	`#D92D20`	**4.83:1** on white; white on it **4.83:1**	Destructive buttons, failed-send state, block/report.
`danger-text`	`#C0271B`	**5.92:1** on white	Danger text at small sizes, where 4.83 is uncomfortably tight.
`danger-dark`	`#FF6B60`	**6.49:1** on `void-900`	Dark-mode equivalent.
`danger-surface`	`#FDECEA` / `#38120F`	—	Banner backgrounds.

Danger is the *only* colour permitted on a destructive confirmation. It never appears as decoration, and never as an unread badge.

Information

Information reuses **Orbit** rather than introducing a fifth hue. Informational banners use `orbit-50` / `orbit-950` surfaces with `orbit-700` / `orbit-300` text. A dedicated "info blue" would be a colour whose only job is to be a slightly different blue — Part 0.9 rule 1 deletes it.

Security states (a semantic family, not decoration)

State	Light	Dark	Meaning
Encrypted (normal)	`void-500` `#676F80`	`void-400` `#8B94A6`	Baseline. Shown once at conversation start, never as a persistent badge — E2EE is the default, and defaults are not announced repeatedly.
Verified contact	`success-text`	`success-dark`	Safety number confirmed by both parties.
Safety number changed	`warning-text`	`warning-dark`	The contact's keys changed. Requires user acknowledgement.
Unverified linked device	`warning-text`	`warning-dark`	A new device joined the account.
Suspected fraud	`danger`	`danger-dark`	Anti-scam signal fired (Part 4 §4.7).

2.5 Neutrals: Void

Cool-tinted greys (a slight blue cast, hue ≈ 220°) so neutrals sit harmoniously with Orbit rather than fighting it. Warm greys against a violet-blue accent produce a muddy interference that is very hard to unsee once noticed.

FIGURE — THE COLOUR SYSTEM, WITH MEASURED CONTRAST

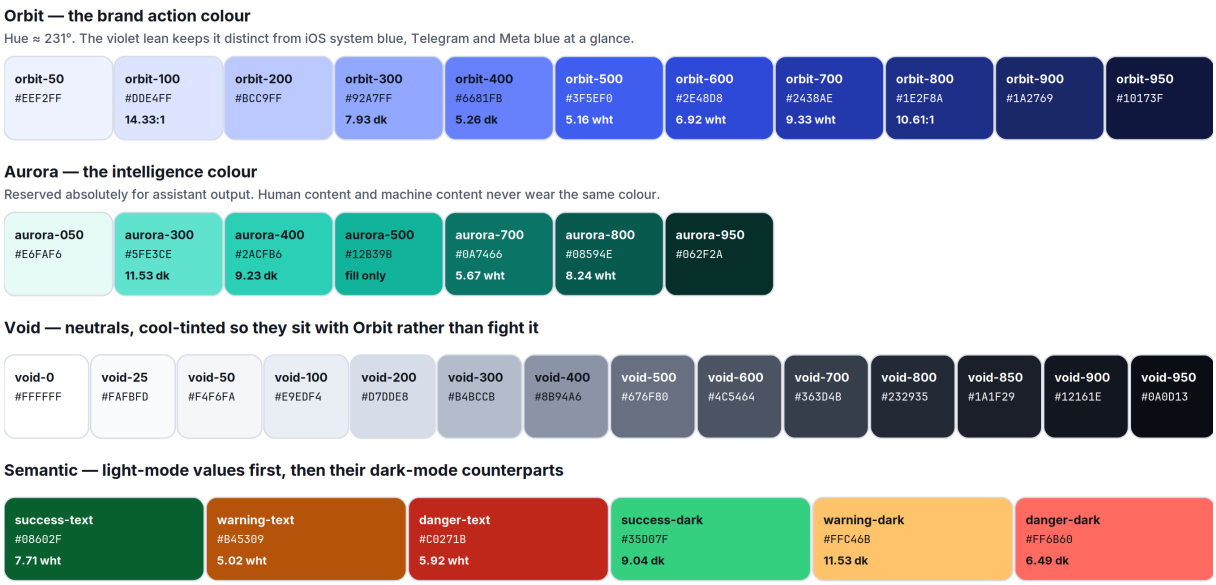


Figure 2.1 The full palette with measured WCAG ratios, and the accessibility limit we found by measuring rather than asserting.

Token	HEX	Light-mode job	Dark-mode job
`void-0`	`#FFFFFF`	Primary surface (cards, sheets, bubbles)	Primary text on dark (`18.12:1` inverse)
`void-25`	`#FAFBFD`	App background beneath cards	—
`void-50`	`#F4F6FA`	Conversation list background; incoming bubble alt	Primary text on dark — **17.97:1** on `void-950`
`void-100`	`#E9EDF4`	**Incoming message bubble, light mode** — `void-900` text at **15.43:1**	Secondary text on dark — **15.43:1** on `void-900`
`void-200`	`#D7DDE8`	Dividers, input borders, skeleton base	—

Token	HEX	Light-mode job	Dark-mode job
void-300	#B4BCCB	Disabled text/icon (light); placeholder	Body text on dark — **9.49:1** on void-900
void-400	#8B94A6	**Non-text only** on white (3.05:1) — decorative icons, inactive tab glyphs paired with a label	Secondary text on dark — **5.94:1** on void-900
void-500	#676F80	Secondary text — **5.05:1** on white, **4.66:1** on void-50	Tertiary/disabled text on dark
void-600	#4C5464	Body text where full black is too heavy — **7.61:1**	—
void-700	#363D4B	Strong body text — **10.90:1**	Dark-mode divider / border
void-800	#232935	Headings on light	**Incoming bubble, dark mode** — void-50 at **13.48:1**
void-850	#1A1F29	—	Elevated surface (sheets, cards, composer)
void-900	#12161E	Primary text — **18.12:1** on white	**Primary dark surface**
void-950	#0A0D13	—	App background beneath dark cards; OLED-friendly

Why void-900 and not #000000 for text, and not for the dark background either: pure black on pure white is a contrast overshoot that causes halation for many readers, and pure-black OLED backgrounds produce visible smearing during scroll on most panels. #12161E costs almost nothing in measured contrast (18.12:1 vs 21:1) and reads materially calmer.

2.6 Light Mode and Dark Mode

Both modes are designed independently against the same tokens. Dark mode is not a computed inversion.

Role	Light	Dark
App background	void-25 #FAFBFD	void-950 #0A0D13
Primary surface	void-0 #FFFFFF	void-900 #12161E
Elevated surface (sheet, menu, dialog)	void-0 + shadow	void-850 #1A1F29 + shadow
Primary text	void-900	void-50
Secondary text	void-500	void-400
Tertiary / timestamp	void-400 (with ≥16 px or paired label)	void-500

Role	Light	Dark
Divider	`void-200`	`void-700`
Incoming bubble	`void-100`	`void-800`
Outgoing bubble	`orbit-100`	`orbit-800`
Assistant surface	`aurora-050`	`aurora-950`
Primary action	`orbit-500` fill, white label	`orbit-300` label on transparent, or `orbit-500` fill

Elevation in dark mode is expressed by lightness, not shadow. Shadows are nearly invisible on dark surfaces; each elevation step raises the surface lightness instead (`void-950` → `void-900` → `void-850`). Shadows still ship in dark mode but only as a subtle ambient occlusion to separate overlapping surfaces.

Mode switching is instant and stateless. No cross-fade, no reload, no flash of the wrong theme at cold start (theme is read synchronously before the first frame — see Part 8 §8.2).

2.7 Gradients

Gradients are permitted in exactly three places. Everywhere else they are banned, because a gradient is a decorative element pretending to be a functional one.

1. **The assistant's activity indicator** — `aurora-500` → `aurora-300`, animated at ≤0.5 Hz, only while the assistant is actively working. It is the one place in the product where "something is thinking" needs a non-textual signal.
2. **Media scrims** — a black gradient from `rgba(10,13,19,0.72)` to `transparent` over photos and video, so white controls remain legible over unknown content. Functional, not decorative.
3. **Onboarding and marketing full-bleed fields** — `orbit-950` → `orbit-800`, linear, ≤35° from vertical. Never behind interface controls.

Banned: multi-stop rainbow gradients, gradient text, gradient borders, gradients on buttons, "AI shimmer" on anything that is not the assistant indicator.

2.8 Shadows and Elevation

Six levels. Shadows use a cool tint (`rgba(18, 22, 30, α)`) rather than neutral black, so they sit correctly on the Void scale.

Level	Use	Light mode	Dark mode
`e0`	Flat content, message bubbles	none	none

Level	Use	Light mode	Dark mode
e1	Cards, list rows on tinted backgrounds	0 1px 2px rgba(18,22,30,0.06)	surface void-900
e2	Composer bar, sticky headers	0 2px 8px rgba(18,22,30,0.08)	surface void-850, 0 2px 8px rgba(0,0,0,0.40)
e3	Bottom sheets, popover menus	0 8px 24px rgba(18,22,30,0.12)	surface void-850, 0 8px 24px rgba(0,0,0,0.48)
e4	Dialogs, floating action controls	0 16px 40px rgba(18,22,30,0.16)	surface void-850, 0 16px 40px rgba(0,0,0,0.56)
e5	Incoming-call full-screen overlay	0 24px 64px rgba(18,22,30,0.24)	as e4 + scrim

Message bubbles cast no shadow. They are content, not objects. A chat transcript with 200 shadowed bubbles is 200 unnecessary composited layers and a visibly noisier screen.

2.9 Glass (translucency)

Translucent blur is expensive on the GPU and unreadable over arbitrary content. It is permitted in exactly two places, both of which sit over scrolling content the user needs to keep tracking:

1. **The conversation header** while a transcript scrolls beneath it.
2. **The composer bar** while a transcript scrolls beneath it.

Specification: 24 px backdrop blur, 180 % saturation, over `rgba(255,255,255,0.72)` (light) / `rgba(18,22,30,0.72)` (dark). A 1 px `void-200` / `void-700` hairline is drawn on the content-facing edge so the boundary is unambiguous.

Mandatory fallbacks. Blur is disabled and replaced with the opaque surface colour when: the device is in a low-power state, the app has dropped below the frame-rate floor in the last 5 seconds, the platform reports reduced-transparency accessibility settings, or the device is below the Tier-B hardware baseline (Part 8 §8.7). Blur is a garnish; it is never load-bearing for legibility.

Banned everywhere else: glass cards, glass sheets, glass tab bars over static backgrounds, frosted modals.

2.10 Corner Radius

A single geometric family, so the whole product feels cut from one material. Radii use *continuous* (squircle) curvature where the platform supports it, matching the logo construction.

Token	Value	Applied to
`radius-xs`	4 px	Tags, inline code, small checkboxes
`radius-sm`	8 px	Input fields, small buttons, menu items
`radius-md`	12 px	Cards, media thumbnails, attachment tiles
`radius-lg`	18 px	**Message bubbles**
`radius-xl`	24 px	Bottom sheets, dialogs (top corners)
`radius-full`	999 px	Avatars, pills, FAB, primary CTA in onboarding

> **The bubble rule.** Bubbles are `radius-lg` on three corners and `radius-xs` on the corner nearest the sender's edge — but *only on the last message in a run*. Consecutive messages from the same sender within 60 seconds keep all four large corners and tighten to 2 px vertical spacing, so a burst of messages reads as one utterance. This is the single most important detail in the transcript's visual rhythm.

Nesting rule. An inner radius equals the outer radius minus the padding between them, never the same value. A 12 px card with 8 px padding contains 4 px children.

2.11 Spacing

A 4 px base unit. All spacing is a token; arbitrary values are a lint failure.

Token	Value	Typical use
`space-1`	4 px	Icon-to-label, tightest internal padding
`space-2`	8 px	Between related inline elements
`space-3`	12 px	Inside bubbles (vertical), list row internal padding
`space-4`	16 px	**Screen edge margin**; between bubbles from different senders
`space-5`	20 px	Between distinct list rows with avatars
`space-6`	24 px	Between content groups
`space-8`	32 px	Section separation
`space-10`	40 px	Above a screen's primary heading
`space-12`	48 px	Empty-state vertical rhythm
`space-16`	64 px	Onboarding vertical rhythm

Rule of proximity: the space *inside* a group is always smaller than the space *around* it. If two elements are 12 px apart, the group they form must have more than 12 px around it. Most "cluttered" screens are proximity failures, not density failures.

2.12 Grid

- **Mobile:** a single fluid column with a 16 px gutter. No multi-column layouts on phone portrait — the transcript is a column, and columns are what phones are for.
- **Tablet / foldable open / desktop:** a two-pane layout — conversation list (fixed 360 px, min 320 px, max 400 px) and conversation (fluid, content capped at 720 px and centred within the pane). A third pane (profile, thread, files) slides over the second below 1280 px and sits alongside it above.
- **Content max width:** 720 px. Beyond that, line length exceeds comfortable reading measure and the transcript stops feeling like a conversation.
- **Baseline:** 4 px vertical rhythm; text blocks snap to it.
- **Safe areas:** honoured on every platform. Nothing interactive within 8 px of a display cutout or home indicator.

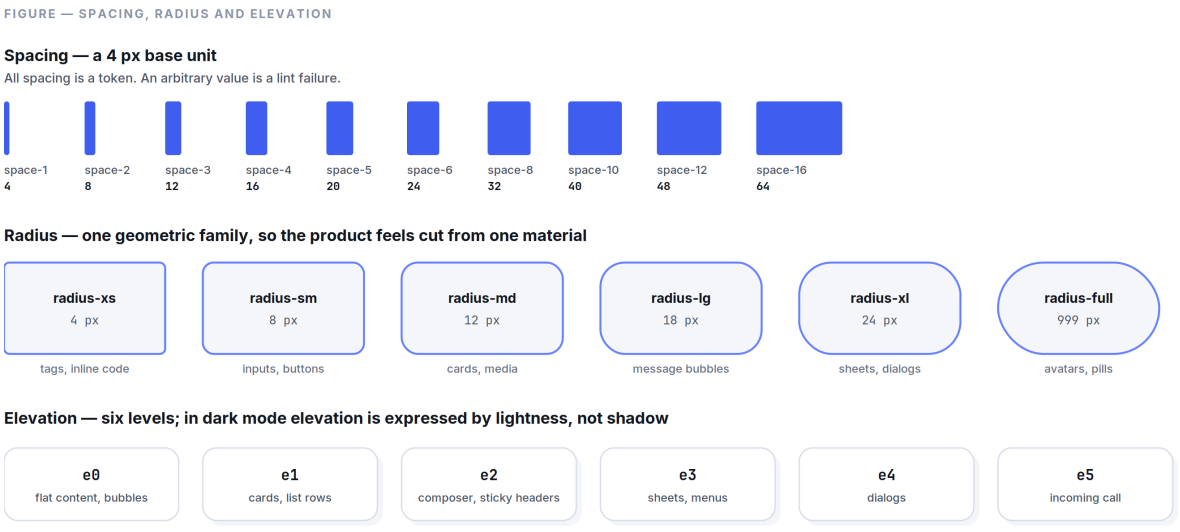


Figure 2.2 The three geometric systems, drawn to scale.

2.13 Typography

Typefaces.

Role	Face	Why
UI + body	**Inter** (variable)	Exceptional legibility at small sizes, huge glyph coverage, true optical sizing, open-licensed — no per-seat cost as we scale, and no legal exposure.

Role	Face	Why
Arabic / Persian / Urdu	**IBM Plex Sans Arabic**	A genuine Arabic design, not a Latin face with Arabic bolted on; weights map cleanly to Inter's.
CJK	Platform default (SF / Noto Sans CJK)	Nothing we could bundle would beat the platform faces, and the file-size cost of shipping CJK is unjustifiable.
Numerals in timestamps, counters, call duration	Inter **tabular** figures	Non-tabular numerals cause visible jitter on any counter that updates.
Code / hashes / safety numbers	**JetBrains Mono**	Safety-number verification requires unambiguous 0/O and 1/l/I.

Scale. A modest ratio (≈1.18) because a communication app has few hierarchy levels and needs none of the drama a 1.5 scale gives.

Token	Size / line-height	Weight	Use
`display`	32 / 38	600	Onboarding headline only
`title-1`	24 / 30	600	Screen titles
`title-2`	20 / 26	600	Section headings, dialog titles
`title-3`	17 / 22	600	Conversation name in list rows, contact name
`body`	16 / 22	400	**Message text** — the most-read type in the product
`body-strong`	16 / 22	600	Emphasis within body
`callout`	15 / 20	400	Message preview in list, sheet body copy
`subhead`	14 / 19	400	Secondary metadata, group member counts
`footnote`	13 / 17	400	Timestamps, "delivered," helper text
`caption`	11 / 14	500	Badges, overline labels, media duration

Rules.

- **Body messages are never smaller than 16 px.** Not for density, not for "more content on screen," not on tablets.
- **Maximum three type sizes per screen.** A screen needing four has too many jobs.
- **Never set body text below 400 weight or above 600** — Inter's lighter and heavier weights fail at small sizes on low-DPI displays.
- **Dynamic Type / font scale is honoured up to 200 %.** Every layout is tested at 200 %; nothing may clip, truncate a critical label, or overlap. Layouts reflow vertically rather than shrinking type.

- **Line length target 45–75 characters.** Enforced by the 720 px content cap.
- **Emoji-only messages render at 32 px** for up to three emoji, then drop to inline size — a rare, deliberate exception to the "three sizes" rule, because it is a real communication signal.

Bidirectional text. The full interface mirrors for RTL locales, including message-bubble corner geometry, back-navigation direction, and progress-bar fill. Mixed-direction runs inside one message use the Unicode Bidirectional Algorithm with explicit isolates around embedded LTR fragments (URLs, @mentions, numbers) — the most common bidi bug is an unisolated URL flipping the punctuation at the end of an Arabic sentence, and it is a release blocker.

2.14 Component Specifications (visual)

Interaction behaviour is in Part 3; the component API is in Part 7. Visual specs live here.

Buttons

Variant	Fill	Label	Height	Radius	Use
Primary	`orbit-500` / dark `orbit-500`	`void-0`, `body-strong`	48 px	`radius-sm`	The one action that matters on the screen. Max one per screen.
Secondary	transparent, 1 px `void-200` / `void-700`	`void-900` / `void-50`	48 px	`radius-sm`	Alternative actions.
Tertiary / text	none	`orbit-600` / `orbit-300`	44 px	`radius-sm`	Low-emphasis, inline.
Destructive	`danger`	`void-0`	48 px	`radius-sm`	Delete, block, leave. Requires confirmation.
Icon	none / `void-100` on press	inherits text colour	44 × 44 px	`radius-full`	Toolbar actions; always has an accessibility label.

States: **hover** −4 % lightness (pointer devices only) · **pressed** −8 % lightness plus a 0.97 scale over 80 ms · **disabled** 38 % opacity, no pointer events, never rendered as a colour a user could mistake for enabled · **focus** 2 px `orbit-500` ring at 2 px offset, always visible for keyboard users, never suppressed.

Minimum touch target is 44 × 44 px regardless of visual size — a 24 px icon has a 44 px hit area. This is not negotiable and is verified by an automated test.

Cards

`void-0` / `void-900` surface, `radius-md`, `e1`, `space-4` internal padding. No border in light mode (shadow does the work); 1 px `void-700` border in dark mode where shadow cannot.

Lists

Row height 72 px with avatar (48 px avatar, `space-4` gutter, two text lines), 56 px without. Dividers are inset to the text baseline, not full-bleed, and are `void-200` / `void-700` at 1 px. Pressed state fills the row with `void-50` / `void-850` — never a coloured highlight.

Inputs

48 px height, `radius-sm`, 1 px `void-200` border, `void-0` fill, `space-3` horizontal padding. Focus: border becomes 2 px `orbit-500` (the extra pixel is drawn inward so the field does not shift). Error: 2 px `danger` border plus `footnote danger-text` message beneath — never a tooltip, never colour alone. Placeholder is `void-400` and never substitutes for a label.

The composer is the exception: it grows from 48 px to a maximum of 5 lines (140 px) then scrolls internally, keeping the send control pinned to the bottom-right and the transcript's last message visible.

Navigation

- **Bottom bar, 4 destinations maximum:** Chats · Calls · Stories · Settings. 56 px + safe area. Active item uses a filled icon and `orbit-600` / `orbit-300`; inactive uses outline and `void-500` / `void-400`. Labels are always shown (icon-only bars fail comprehension tests and screen readers alike).
- **Channels** live inside Chats as a filter, not as a fifth destination — see Part 5 §5.6.
- **Top bar** is 56 px, carries at most a title, a back affordance, and two actions.

2.15 Accessibility Requirements

Non-negotiable per Part 0.6 clause 9.

Requirement	Standard
Text contrast	≥4.5:1 body, ≥3:1 for ≥24 px or ≥19 px bold. Every token pair in this document is measured.
Non-text contrast	≥3:1 for interface component boundaries, focus indicators, and meaningful icons.
Touch targets	≥44 × 44 px, ≥8 px separation between adjacent targets.
Colour independence	No information conveyed by colour alone. Delivery state has distinct *glyphs*, not just colours. Unread state uses a dot **and** a weight change.
Screen readers	Every interactive element has a label, role, and state. The transcript is navigable message-by-message. Announcements for incoming messages are polite, not assertive.

Requirement	Standard
Dynamic type	Layouts hold to 200 % scale without clipping or overlap.
Reduced motion	Honoured product-wide (Part 1 §1.9 rule 7).
Reduced transparency	Blur replaced with opaque surfaces (§2.9).
Keyboard	Full operation on desktop/web: tab order matches visual order, focus is always visible, Escape closes, no keyboard traps.
Captions	Auto-transcription available for every voice note and, from Phase 2, live captions in calls.
Colour-blind safety	See the note below — lightness separation is maximised where possible but is **not** sufficient on its own, so distinct glyphs are mandatory.

> **A measured note on colour-blind safety, recorded honestly.** We wanted to claim that the semantic trio is distinguishable in greyscale. It is not fully, and the constraint is physical: in light mode every semantic *text* colour must clear 4.5:1 on white, which caps it at roughly CIE L* *48. Green, amber, and red all land in a narrow band there. Measured values:* `success-text #08602F` *L** 35.2 · `danger-text #C0271B` L* *42.4* · `warning-text #B45309` *L** 46.9. Success is comfortably separable; warning and danger differ by ~4.5 L* — *visible side by side, unreliable in isolation. Dark mode is better because the ceiling lifts:* `danger-dark #FF6B60` *L** 63.9 · `success-dark #35D07F` L* *74.4* · `warning-dark #FFC46B` *L** 82.8.

The design consequence is a hard rule, not a caveat: **every semantic state ships a distinct glyph and, wherever space allows, a word.** A failed message is a filled circle with an exclamation *and* the word “Not sent.” Delivery states use distinct tick geometry, not tick colour. Any surface where the only difference between “warning” and “error” is the hue is rejected in design review.

Testing. Automated checks (contrast, target size, missing labels) run in CI on every component. Manual screen-reader passes (VoiceOver + TalkBack) are a release gate for any changed surface. See Part 6 §6.12.

PART 03

03

The UX Bible

Part 3 — How the Product Behaves

Governed by Part 0. Visual specifications live in Part 2; this document governs behaviour.

> "Latency is a moral property, not a technical one."

IN THIS PART

3.1 Navigation Philosophy

Four destinations. One level of depth to reach any conversation. Never more than three taps to any feature in the product.

FIGURE — INFORMATION ARCHITECTURE AND NAVIGATION

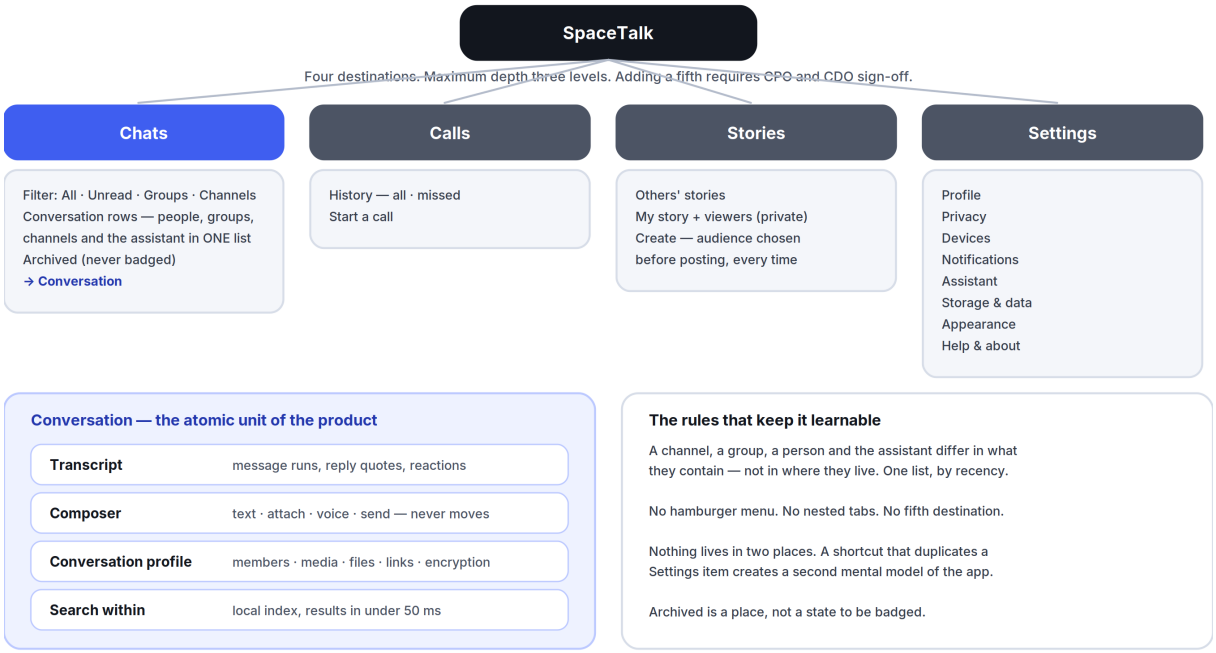


Figure 3.1 Four destinations, one conversation list, and the rules that stop the structure from growing.

```
Chats  ·  Calls  ·  Stories  ·  Settings
```

Why these four, and why nothing else. Chats is where 90 % of time is spent. Calls is a distinct mental mode (real-time vs. asynchronous) with its own history. Stories is ephemeral and must not compete with the transcript for attention. Settings holds identity, privacy, and devices. Channels is *not* a fifth tab — a channel is a conversation you subscribe to, so it lives in Chats behind a filter chip. Adding a fifth destination is a Part 0.9 decision requiring CPO and CDO sign-off, and the default answer is no.

Rules.

1. **Back always means back.** The back gesture and control undo the last navigation, never "up to a parent you didn't come from."
2. **Tapping the active tab scrolls to top; tapping again jumps to the oldest unread.** Two-stage, learnable, no menu required.
3. **No hamburger menu.** A drawer hides structure behind an unlabelled affordance and doubles the depth of everything inside it.
4. **No nested tabs.** Tabs inside tabs are the clearest sign the information architecture has failed.

5. **Modals are for one decision.** A modal that contains navigation is a screen wearing the wrong clothes.
6. **Deep links resolve to real state with a real back stack** — opening a message from a notification and pressing back returns to Chats, not to the home screen.
7. **The conversation is the atomic unit.** Everything — a person, a group, a channel, the assistant — is a row in one list, sorted by recency. There is no separate inbox for different kinds of talking.

3.2 Interaction Philosophy

Optimistic, reversible, and honest.

- **Optimistic by default.** Sending a message renders it immediately at `body` weight with a pending indicator. The network confirms afterwards. A user should never watch a spinner to find out whether they said something.
- **Everything reversible where physics allows.** Deleting a chat, leaving a group, and unsending a message all offer undo for 5 seconds via a non-blocking snackbar. Undo is preferred over a confirmation dialog for anything recoverable — confirmations tax every use to prevent the rare mistake, undo taxes only the mistake.
- **Confirm only the irreversible.** Deleting an account, unlinking a device you are currently using, and clearing a chat for everyone get a typed or explicitly-labelled confirmation. Nothing else does.
- **Long-press is the universal "more."** Every message, row, and media item reveals its full action set on long-press with a haptic tick. Nothing is *only* available on long-press.
- **Gestures are shortcuts, never the only path.** Swipe-to-reply is fast; a reply action also exists in the long-press menu, because gestures are undiscoverable and unavailable to switch-control users.
- **Direct manipulation over menus.** Drag a photo into the composer; drag the video-call window anywhere; pull a message down to reply.
- **The composer never moves.** Nothing that appears — attachment tray, emoji picker, assistant panel — displaces the text field or the send control. Muscle memory is a feature.
- **No confirmation of success.** Nothing says "Message sent!" A message that appears in the transcript with a delivered tick has already said it.

The gesture set (fixed — additions require CDO approval)

Gesture	Result
Swipe right on a message	Reply

Gesture	Result
Swipe left on a message	(Reserved — no action at MVP, so it cannot conflict with a system back gesture)
Long-press message	Action menu: react, reply, forward, copy, edit, translate, delete, info
Double-tap message	Apply the last-used reaction
Swipe right on a conversation row	Read/unread toggle
Swipe left on a conversation row	Mute, then Archive
Pull down in a transcript	Search this conversation
Long-press the mic	Record; slide up to lock hands-free; slide left to cancel
Pinch in media viewer	Zoom; drag down to dismiss

3.3 Motion Guidelines

Philosophy is Part 1 §1.9; tokens are Part 7 §7.7. Behavioural rules:

Transition	Duration	Curve	Note
Touch feedback (press)	80 ms	`standard`	Instant enough to feel physical
Sheet in	240 ms	`emphasised-decelerate`	Rises from the trigger
Sheet out	200 ms	`emphasised-accelerate`	Returns to the trigger
Screen push	280 ms	`emphasised`	Shared-element where an avatar or image persists
Message send	180 ms	`standard-decelerate`	Bubble rises from the composer to its resting place
List reorder	220 ms	`standard`	Moved item is tracked, never teleported
Skeleton → content	120 ms cross-fade	`linear`	Never a “pop”

> **The 100 ms rule.** Any transition that begins in direct response to a touch must show its first changed frame within 100 ms, even if the underlying work has not finished. If data is not ready, the destination appears in its loading state. Waiting on the origin screen is the single worst

> thing an interface can do.

3.4 Loading Behaviour

A hierarchy of four states, applied in strict order of preference.

1. **No loading state at all** — the content was already there. This is the goal, and it is achievable for the conversation list, the last screen of every open transcript, all cached media, and the user's own profile, because they are read from the local database (Part 6 §6.8).
2. **Stale content plus a quiet refresh indicator** — show what we have, update in place. Never blank out correct-but-old content to show a spinner.
3. **Skeleton** — only for content never seen before, and only if it will plausibly take >300 ms. Skeletons must match the real layout's geometry so nothing shifts on arrival.
4. **Spinner** — the last resort, and only for indeterminate work of unknown length, never for list loads.

Rules.

- **No full-screen loading screens after the first launch.** Cold start goes straight to the conversation list, populated from disk.
- **Nothing may shift after it has been rendered.** Reserve space for images using their known dimensions (stored in the message envelope) so the transcript never jumps.
- **Progress must be honest.** A determinate bar reflects real bytes. If we don't know, we don't fake a bar.
- **Anything over 10 seconds becomes backgroundable** with a persistent, dismissible status row — a large upload never holds the user hostage on one screen.

3.5 Error Handling

Principles. Errors are events in a system, not accusations. Every error message answers three questions in this order: *what happened*, *what it means for you*, *what to do next*. If we can do the next step ourselves, we do it and don't show an error at all.

Severity ladder.

Level	Presentation	Example
Silent	Nothing shown; retried automatically	One failed message delivery attempt on a flaky connection
Ambient	Inline state change on the affected object	A single message shows "Not sent · Tap to retry"

Level	Presentation	Example
Snackbar	Non-blocking, 4 s, with an action	“Couldn’t upload photo. Retry”
Banner	Persistent, dismissible, top of screen	“You’re offline”
Dialog	Blocking — only when the user must decide	“This contact’s safety number changed. Verify or continue?”
Full screen	Only when the app cannot function	Account suspended; version end-of-life

Rules.

- **Retry automatically before telling anyone.** Exponential backoff with jitter, capped; only surface after the retry budget is spent.
- **Never show an error code alone.** If diagnostics are needed, put the code behind “Details,” and make it copyable.
- **Never lose user input.** A failed send keeps the text in the transcript as a retryable draft. A crashed composer restores its draft on relaunch. Drafts are persisted per conversation on every keystroke pause.
- **Distinguish “we failed” from “you can’t.”** A permission error is not a system failure and must not read like one.
- **One error at a time.** Errors do not stack; the newest replaces the oldest of equal severity.

3.6 Empty States

Every empty state has exactly three parts: an illustration (per Part 1 §1.7), a one-line explanation of *why* it is empty, and a single action that fills it. No more.

Surface	Line	Action
No conversations	“No conversations yet.”	“Find someone”
No calls	“No calls yet.”	“Start a call”
Search, no results	“Nothing found for ‘<query>’.”	“Search all messages” (widens scope)
Offline with no cache	“You’re offline and this hasn’t loaded yet.”	“Retry”
Channel with no posts	“Nothing posted yet.”	(none — this is the owner’s cue, not a task)
Archived, empty	“Nothing archived.”	(none)

Empty states never sell. They do not advertise Plus, do not suggest inviting five friends, and do not congratulate the user on inbox zero.

3.7 Offline Behaviour

SpaceTalk is a local-first application that syncs, not a client that fetches. Every screen is rendered from the on-device database; the network updates that database in the background. This is an architectural commitment (Part 6 §6.8), and the UX consequences are:

- **The entire app is fully usable offline** except for the acts that inherently need a peer: placing a call, and loading media that was never downloaded.
- **Composing is always available.** Messages, voice notes, reactions, edits, and deletions queue in an outbox and send in order when connectivity returns. Queued items are shown in the transcript with a clock glyph, in place, not in a separate outbox screen.
- **Offline is stated once, quietly.** A single top banner, not a per-message error, not a modal, not a toast every 30 seconds.
- **Reconnection is invisible.** The banner disappears, the queue drains, ticks update in place. No "Back online!" celebration.
- **Conflicts resolve by rule, never by asking.** Message ordering is by the server's assigned sequence within a conversation; edits use last-writer-wins by device timestamp with the loser preserved in edit history; deletions always win over edits. The user is never shown a merge dialog.
- **Airplane-mode is a tested state**, exercised in CI on every release (Part 6 §6.12).

3.8 Search Philosophy

One search field. It searches everything the user can see, and nothing they cannot.

- **Instant local results first.** Keystroke-by-keystroke over the on-device index, ranked: exact-match contacts → conversations → messages → media → files. Results appear within 50 ms of the keystroke (Part 8).
- **Server search only for what cannot be local** — public channel discovery, and message history not yet downloaded on this device. It is clearly labelled as a separate, later-arriving section, never blended into local results in a way that makes the list jump.
- **Semantic search is opt-in and additive** (Part 4 §4.6). Literal search always works first and never gets slower because semantic search exists.
- **Filters are chips, not a form.** From · In · Type · Date, applied progressively, always visible, always removable.
- **Search never leaves the encryption boundary.** Message search over E2EE content runs entirely on-device against a locally built index. We cannot search server-side for content we cannot read, and we will not build a system that lets us.
- **Recent searches are local and clearable**, and are never used to personalise anything.

3.9 Notification Philosophy

> **A notification is a promise that something needs you. Every false promise costs trust that cannot be repurchased.**

What may notify:

- A message addressed to you (direct, group, or an @mention in a muted group).
- An incoming call.
- A security event on your account (new linked device, safety-number change).
- A completion you explicitly asked to be told about (large file finished uploading).

What may never notify: re-engagement prompts, "you have unread messages," feature announcements, tips, someone joining SpaceTalk, someone posting a story, streaks, birthdays, anniversaries, "your friend is active now," or anything a growth team wants.

Behaviour.

- **Content is decrypted on the device, never sent by the server.** Push payloads carry only an encrypted envelope and a conversation identifier; the device fetches and decrypts to build the notification text (Part 6 §6.6). If the user hides previews, we show "New message" — and the server could not have shown more even if it wanted to.
- **Grouped by conversation, always.** Ten messages from one person is one notification with a count, never ten.
- **Actionable inline:** reply, mark read, mute — without opening the app.
- **Sound is one short, low, non-startling tone**, and a distinct one for calls. No custom sounds at MVP; per-conversation sounds are Phase 2.
- **Mute is granular and honest:** 8 hours, 1 week, always. A muted conversation produces no sound, no badge, no banner — mute means mute.
- **Badges count only messages addressed to you.** Muted conversations never contribute. A badge that overstates is a lie told in a number.
- **Permission is requested in context, after value.** Never on first launch. We ask when the user has just sent their first message, framed as "so you know when they reply."
- **Quiet hours are a first-class setting** and default to off — we do not guess someone's sleep.

3.10 Onboarding Principles

Target: from app open to first message sent in under 90 seconds, with 4 screens maximum.

1. **Value before permission.** Nothing is requested until it is needed to do something the user already wants.
2. **No account tour.** No carousel of features. Nobody has ever read one.
3. **Progressive disclosure.** The assistant, channels, stories, and linked devices are discovered in use, not taught up front. Each has a single first-use coach mark that appears once and never returns.
4. **Identity is the only required step.** Choose a username, confirm a contact method, set a display name. Photo, bio, and everything else are skippable and clearly marked so.
5. **No address-book upload at MVP.** We do not ask for the contact list, because we have decided not to build a contact-discovery system we cannot make private (Part 12 ADR-010). Users find each other by username, by QR code, or by a share link. This is slower to grow and it is the correct decision.
6. **Restoring beats re-onboarding.** A returning user or a new linked device goes through a distinct, shorter path.
7. **The empty first screen must be inviting, not accusatory** (§3.6).

3.11 Accessibility (behavioural)

Visual requirements are in Part 2 §2.15. Behavioural requirements:

- **Screen-reader transcript navigation** is message-by-message; each message announces sender, content, time, and delivery state in that order, and reactions are a sub-element rather than an interruption.
- **Incoming messages announce politely** (queued behind whatever the user is reading), never assertively.
- **Voice notes always offer a transcript** — this serves deaf and hard-of-hearing users and, in practice, most users in loud or quiet places.
- **Calls surface live captions** from Phase 2, on-device where the platform allows.
- **All gestures have a non-gesture equivalent** (§3.2).
- **No time-limited interactions.** Nothing disappears on a timer that a user must act within, except a call ringing — and that has a generous duration and a missed-call record.
- **Switch and keyboard control** reach every action.
- **Haptics are informational, never decorative**, and fully disableable.

3.12 One-Handed Use and Thumb Reach

The product is designed for a person holding a large phone in one hand, thumb anchored at the bottom corner.

FIGURE — THUMB REACH AND ONE-HANDED USE

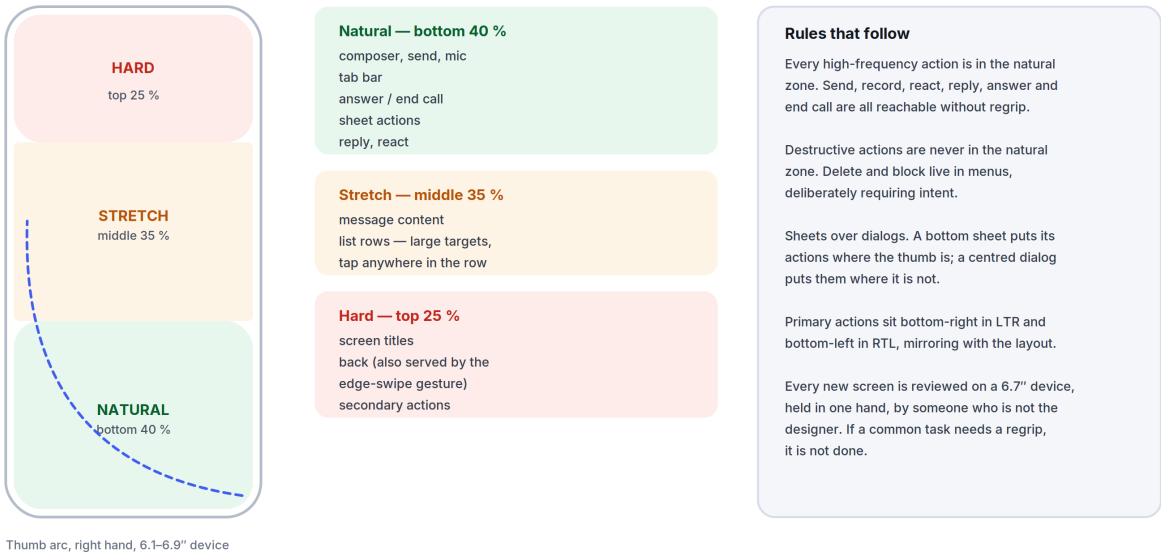


Figure 3.2 The reach model the whole interface is laid out against.

The reach zones, on a 6.1″–6.9″ device held one-handed:

Zone	Screen region	What lives there
Natural	Bottom 40 %	Composer, send, mic, tab bar, primary actions, sheet actions
Stretch	Middle 35 %	Message content, list rows (large targets, tap-anywhere)
Hard	Top 25 %	Titles, back (also served by the edge-swipe gesture), secondary actions

Rules.

1. **Every high-frequency action is in the natural zone.** Send, record, react, reply, answer, and end call are all thumb-reachable without regrip.
2. **Destructive actions are never in the natural zone.** Delete and block live in menus, deliberately requiring intent.
3. **Sheets over dialogs.** A bottom sheet puts its actions where the thumb is; a centred dialog puts them where it isn't. Dialogs are reserved for blocking decisions (§3.5).
4. **Primary actions sit bottom-right in LTR and bottom-left in RTL**, mirroring with the layout.
5. **The top-of-screen back control is always redundant** with an edge-swipe gesture.

6. **Tested one-handed.** Every new screen is reviewed on a 6.7″ device, held in one hand, by someone who is not the designer. If it needs a regrip for a common task, it is not done.
7. **Foldables and tablets shift to the two-pane grid** (Part 2 §2.12) and move primary actions to the pane edges nearest the thumbs, not the screen centre.

PART 04

04

The AI Philosophy

Part 4 — Intelligence Inside a Private Medium

Governed by Part 0, especially clauses 0.6.3 and 0.6.4. This is the hardest document in the Bible, because it governs the one place where two of our promises genuinely pull against each other.

> "Do it on the device. If it cannot be done on the device, ask. If neither is acceptable, the feature does not ship."

IN THIS PART

4.1 The Central Tension, Stated Plainly

> We promise that personal messages are end-to-end encrypted and that we cannot read them. We also promise the most useful intelligence ever put inside a communication product. Useful intelligence requires plaintext.

FIGURE — THE AI PRIVACY DECISION, IN PRIORITY ORDER

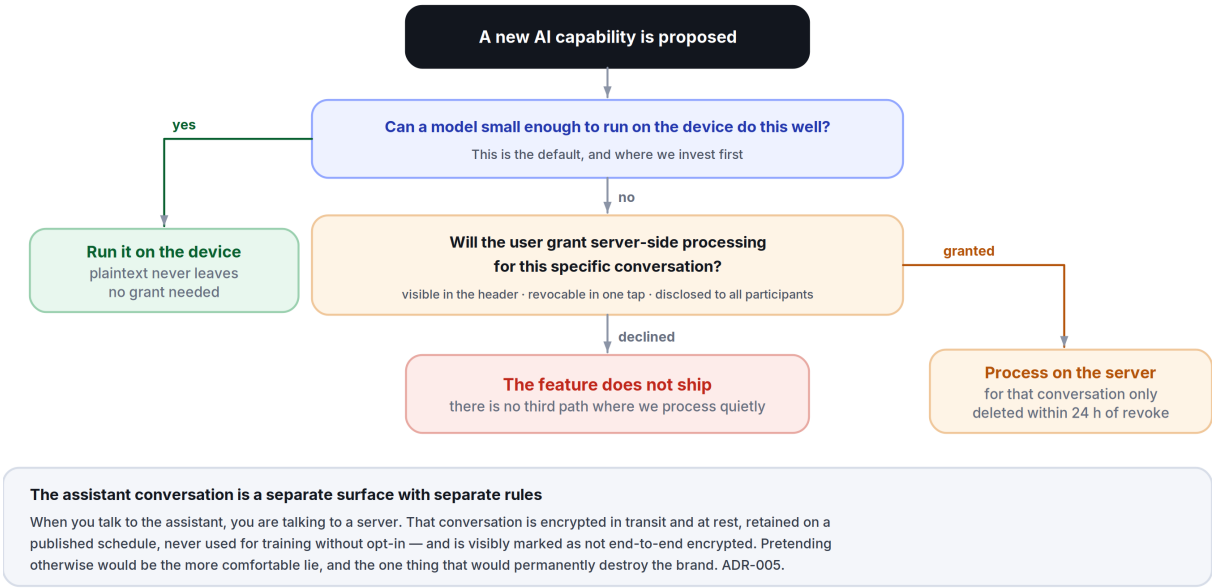


Figure 4.1 The three-tier rule that resolves the product's one genuine contradiction — on-device, then granted, then not at all.

Most companies resolve this by quietly weakening the first promise, and describing the result in language vague enough to survive a press cycle. We will not do that. Our resolution has three parts, in strict priority order:

1. **Do it on the device.** If a model small enough to run locally can do the job at acceptable quality, it runs locally and the plaintext never leaves. This is the default, and it is where we invest first.
2. **If it cannot be done on the device, ask — specifically, per conversation, revocably, and visibly.** A user may grant server-side assistance for a particular conversation. The grant is shown persistently in the conversation header, is revocable in one tap, and is disclosed to the *other* participants in that conversation, because their words are involved too.
3. **If neither is acceptable, the feature does not ship.** There is no third path where we process private content quietly because it would make a metric go up.

The assistant conversation itself is a different surface with different rules, and we say so in the interface. When you talk *to* the assistant, you are talking to a server. That conversation is encrypted in transit and at rest, is retained under a published schedule, is never used to train a model without opt-in, and is visibly marked as *not* end-to-end

encrypted. Pretending otherwise would be the more comfortable lie, and it is the one thing that would permanently destroy the brand.

4.2 Principles

1. **Invited, never volunteered.** The assistant does not speak unless addressed, or unless it has standing permission for a specific job the user configured.
2. **In the medium, not beside it.** Intelligence appears as a property of a message — a translation under it, a summary at the top of an unread run — rather than as a chatbot the user must visit.
3. **Always attributable.** Any text a model produced is visually distinct (Aurora, per Part 2 §2.1 rule 3) and labelled. A user must never be uncertain whether a person or a machine wrote something.
4. **Suggestion, never substitution.** The assistant never sends a message on the user's behalf. Draft, always; send, never — not with a delay, not with a confirmation, not as a setting.
5. **Calibrated, not confident.** When the model is unsure, the interface says so in plain words. We do not launder uncertainty through fluent prose.
6. **Fast or absent.** An AI feature that adds perceptible latency to a core interaction is removed from that interaction. Nothing about intelligence may make messaging slower (Part 0.9 rule 4).
7. **Fully disableable.** A user can turn every AI feature off, permanently, in one setting, and the product remains complete. If turning AI off breaks the app, we built the app wrong.
8. **No training on private content by default.** Ever. Opt-in only, per surface, with a plain-language description of what that means.
9. **The assistant has no persona** (Part 1 §1.4). No name, no personality, no emotional performance, no "I'm so happy to help." Warmth comes from being useful.
10. **Refusals are explained.** When the assistant will not do something, it says why in one sentence and, where possible, offers what it can do.

4.3 Reply Suggestions

What it is. Up to three short, contextually plausible replies offered above the composer.

Where it runs. On-device only, always. This is the highest-frequency AI surface in the product and the one closest to private content — it is not worth any server round trip, either in latency or in trust.

Rules.

- Appears only when the last message is from someone else and is plausibly answerable in a short reply.

- Never for messages classified as sensitive (grief, medical, conflict, financial). Suggesting "Sounds good!" under a death in the family is the kind of error that ends a product's reputation, and the classifier is tuned to be over-cautious here by design.
- Suppressed entirely for the first 3 messages of any new conversation — the opening of a relationship is not ours to autocomplete.
- Tapping a suggestion **inserts it into the composer**; it does not send. The user always presses send (§4.2 rule 4).
- Suggestions inherit the conversation's language, including code-switching.
- One tap dismisses them for that conversation; two dismissals in a week disables them globally with a quiet confirmation, because the user has told us twice.

Success metric. Acceptance rate is *not* the target — a high acceptance rate could mean we are making conversation more generic. The target is: no measurable increase in time-to-reply, and a dismissal rate below 15 %.

4.4 Translation

The feature we expect to matter most, and the one most likely to define the product.

Three modes:

Mode	Where it runs	Behaviour
Tap to translate	On-device (downloadable language pack)	A translation appears beneath the original in Aurora. The original is never replaced.
Always translate this conversation	On-device where a pack exists; server with explicit grant otherwise	Every incoming message arrives with a translation attached. Indicated persistently in the header.
Translate what I send	On-device or granted server	Shows the translation *and* a back-translation before sending, so the sender can sanity-check what the other person will read.

Rules.

- **The original is always present and always primary.** Translation is an annotation, never a replacement. Tap the translation to collapse it.
- **The recipient is told a message was translated**, and by which side. Hidden translation creates a false impression of shared fluency.
- **Confidence is shown for low-confidence output** — "Rough translation" is an honest and useful label.
- **Names, @mentions, code, and numbers are never translated.**
- **Language packs download over Wi-Fi by default**, are 20–60 MB each, and are removable. We do not silently consume a metered connection.

- **Live call translation is Phase 3, not MVP.** Doing it badly is worse than not doing it, and doing it well requires latency work we will not have finished.

4.5 Summaries

Unread summaries. When a user opens a conversation with more than 30 unread messages, a collapsed card offers "Summarise 47 messages." It is never generated automatically — generating it automatically would mean processing content the user may not have wanted processed, and would burn compute on conversations nobody opens.

Group call summaries (Phase 2). Opt-in per call, with an unmistakable indicator visible to every participant for the entire call, and a consent prompt for each participant on join. Recording or summarising people who did not agree is not a feature we will ship in any jurisdiction, regardless of what local law permits.

Rules.

- Summaries are always labelled, always collapsible, and always sit above the transcript without displacing it.
- A summary links each claim back to the specific message it came from — one tap jumps to the source. An unverifiable summary is worse than no summary.
- Summaries never include content from messages that were deleted or that disappeared.
- The user can regenerate, and can report a bad summary in one tap.

4.6 Voice Transcription and Smart Search

Transcription. Every voice note offers a transcript, generated **on-device** by default. It is available to the sender before sending (so they can check what will be understood) and to the recipient on demand. Recipients in loud environments, deaf and hard-of-hearing users, and people who simply read faster than they listen all benefit; this is an accessibility feature that happens to use a model.

Smart search. Literal search always runs first and is never slowed by anything here (Part 3 §3.8). Semantic search is a second, clearly-labelled result group built from an **on-device** embedding index over locally-held messages. Server-side semantic search over private content is not built, because it cannot be built without breaking §4.1.

The practical consequence, stated honestly: semantic search covers only what is on this device. A user searching a five-year-old conversation from a newly linked device will not find it semantically until that history syncs. We accept that limitation rather than resolve it by uploading plaintext.

4.7 Spam, Scam, and Fraud Protection

This is the most valuable AI in the product, and it must work without reading private conversations.

How it works within the encryption boundary:

- **Signals we can legitimately use, server-side:** account age, message *fan-out* rate (how many distinct new recipients an account contacts per hour), the ratio of messages sent to unknown parties versus known contacts, block and report rates, registration patterns, and device attestation. All of these are metadata we necessarily process to route messages — none require reading content.
- **Content-based detection runs on the device.** A small local classifier examines *incoming* messages from senders not in the user's contacts and flags known fraud patterns: advance-fee requests, one-time-passcode phishing, romance-scam scripts, fake-delivery lures, crypto-recovery scams, and impersonation of SpaceTalk itself. The model runs locally; the message never leaves the device for this purpose.
- **Reporting is user-initiated and consent-based.** If a user reports a message, that specific message is shared with us — with a clear statement that it will be. Nothing is exfiltrated silently.

Interface behaviour.

- A flagged message shows an inline warning strip in `danger`, above the message, stating the *specific* pattern: "This message asks for a verification code. SpaceTalk staff never ask for one."
- The user can always read the message. We warn; we do not censor personal communication.
- Unknown senders are held in a **message request** state — one preview, no delivery receipt, no notification sound — until accepted. This single mechanism defeats most spam without any model at all.
- **False positives are worse than false negatives here.** A warning on a legitimate message from a grandmother is a trust catastrophe. The threshold is set conservatively and every flag type is tracked for precision, with a rollback trigger if precision drops below 95 % for any pattern class.

Deepfake and impersonation detection is explicitly Phase 4 research, not an MVP claim. We will not advertise a capability we cannot measure.

4.8 Conversation Memory

Default: off. Scope: the assistant conversation only.

The assistant may remember facts you tell it *in the assistant conversation* — your timezone, your preferred language, that you are planning a trip in March. It does not build a

profile from your private conversations with other people. That would require the plaintext access that §4.1 forbids.

Rules.

- **Memory is a list, not a black box.** Settings → Assistant → Memory shows every stored item as a plain sentence, with when it was learned and from where.
- **Every item is individually deletable**, and there is a single "forget everything" control.
- **Nothing is remembered silently.** When the assistant stores something, it shows a small, non-modal "Remembered: you prefer metric units" that can be undone in one tap.
- **Memory never crosses accounts**, and it never crosses into what other people can see.
- **Memory is exportable** in the standard data export.
- **Deletion is real deletion** — removed from the store and from any index within 24 hours, verified by an automated audit job, not by policy alone.

4.9 Privacy Boundaries (the definitive table)

Surface	Runs where	Sees plaintext	Default	Consent
Reply suggestions	Device	Local only	On	Disable in settings
Tap-to-translate	Device	Local only	On demand	Per tap
Always-translate a conversation	Device, or server with grant	Server only with grant	Off	Per conversation, both parties informed
Voice-note transcription	Device	Local only	On demand	Per note
Smart (semantic) search	Device	Local only	On	Disable in settings
Unread summary	Device if feasible; server with grant	Server only with grant	Off	Per invocation
Call summary	Server	Yes, for that call	Off	Every participant, per call
Scam detection (content)	Device	Local only	On	Disable in settings
Spam detection (metadata)	Server	**No content**	On	Not disableable — it is abuse prevention on data we necessarily hold

Surface	Runs where	Sees plaintext	Default	Consent
Assistant conversation	Server	Yes — this is a server conversation, and it is labelled as one	N/A	Using it is the consent; the label is permanent
Model training	—	—	**Off**	Explicit opt-in, per surface, revocable

FIGURE — THE AI PIPELINE

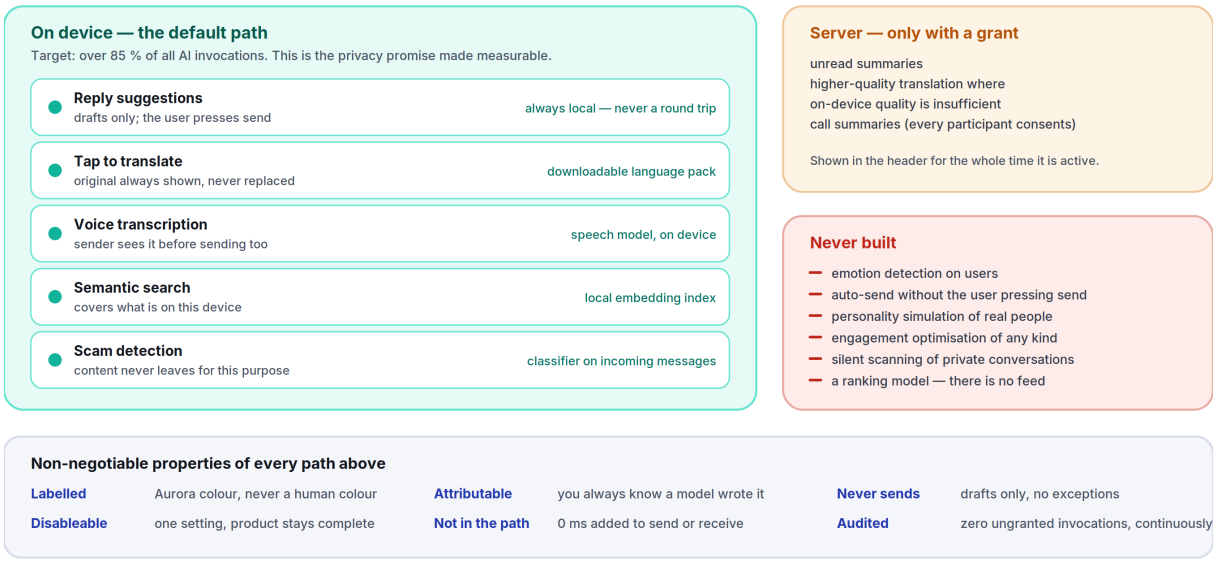


Figure 4.2 What runs where — and the six properties that hold on every path.

> **Rule of thumb for any future feature:** if you cannot fill in a row of this table honestly, the feature is not designed yet.

4.10 AI Transparency

1. **Every AI-generated element is visually and programmatically labelled.** Aurora colour, an "assistant" glyph, and a screen-reader label of "AI generated."
2. **"Why am I seeing this?"** is available on every AI surface and explains, in one plain sentence, what triggered it and what data was used.
3. **A permanent, one-tap Privacy Centre** shows: which conversations have server-side AI grants, what the assistant remembers, what is enabled, and what data has left the device in the last 30 days.
4. **We publish which models we use, where they run, and who operates them**, in human-readable form, and we update it when it changes. Users deserve to know whose infrastructure their words touch.

5. **We publish an accuracy note per feature** — for translation and transcription, per language, including the languages where we are weak. Advertising uniform quality across 100 languages when quality is not uniform is a lie that users detect immediately in the languages that matter to them.
6. **Failures are visible.** When the assistant cannot do something, it says so. There is no silent degradation to a worse model.
7. **A transparency report** covering law-enforcement requests, what we were able to provide (and what we were architecturally unable to provide), and moderation actions, published twice a year from Phase 2.

4.11 What We Will Not Build

Recorded here permanently so it does not need re-litigating:

- **Emotion detection on users**, in text, voice, or video. Inaccurate, invasive, and discriminatory in practice.
- **Automatic replies sent without the user pressing send.**
- **Personality-simulating avatars of real people.**
- **Engagement optimisation of any kind** — no model whose objective function is time-in-app.
- **Silent content scanning of private conversations**, for any purpose, including ones we would consider good. Once the capability exists, the pressure to widen its use never stops, and the architecture that resists that pressure is the one where the capability was never built.
- **A model that decides what a user sees from strangers** — because there is no feed (Part 0.6 clause 2), there is no ranking model, and this is not a gap.

PART 05

05

The Feature Bible

Part 5 — The MVP, Specified

Governed by Part 0 §0.8. This document specifies the thirteen features of the first public version and nothing else. Every idea outside this list is triaged in Part 10 — deferred with a phase, or rejected with a reason. Nothing is silently dropped.

> "No metric in this document is an engagement metric."

IN THIS PART

Format. Every feature states: **Purpose · User problem · Success metrics · UI behaviour · Edge cases · Failure cases · Future roadmap.**

> **A note on metrics.** No metric in this document is an engagement metric. We do not target time-in-app, sessions per day, or messages per user — those go up when a product gets worse as reliably as when it gets better. We measure whether the thing worked, how fast, and whether people came back.

5.1 Secure Messaging

Purpose. The core of the product: send text to a person or a group, encrypted end-to-end, delivered faster than anything else on the market, and readable forever afterwards.

FIGURE — ANATOMY OF THE MESSAGE BUBBLE

The most important component in the product. Everything else is held to a lower ambition ceiling.

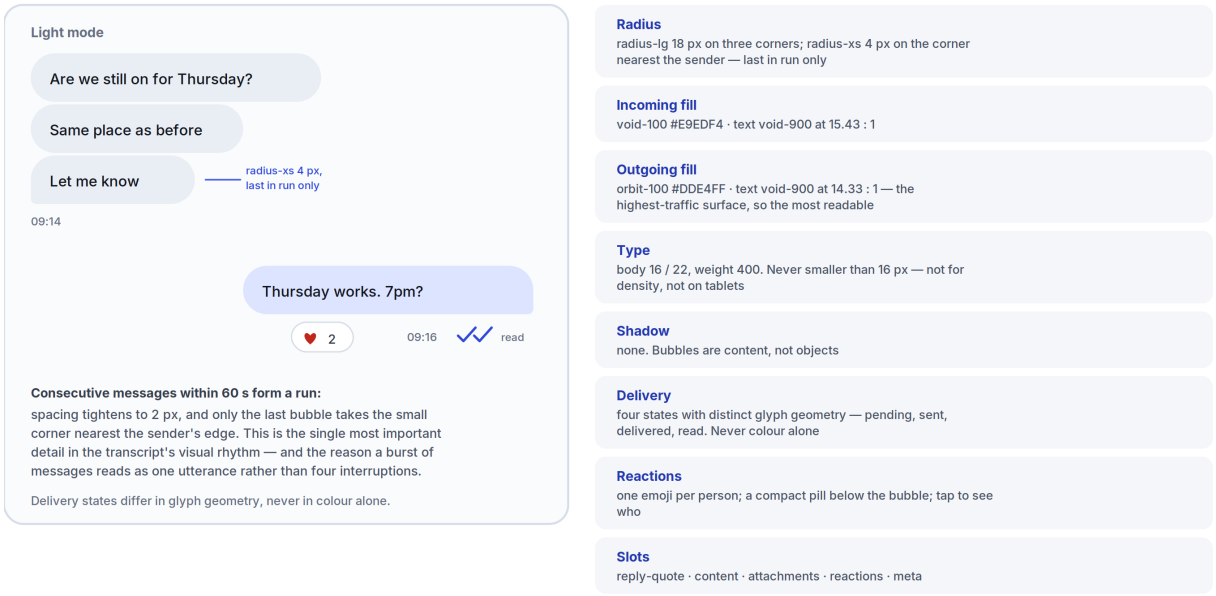


Figure 5.1 Bubble geometry, fills, contrast and run-grouping — specified once so it is never re-decided.

User problem. Existing messengers are either fast but not private, private but slow and awkward, or both but visually exhausting. Users have also learned to expect that history is fragile — lost on device change, capped by storage, or trapped on one phone.

Success metrics.

- p50 send-to-delivered (sender network → recipient device, both on 4G+) **< 250 ms**; p95 < 700 ms.
- Message loss rate **0** — measured as: every message accepted by the client eventually reaches every recipient device or is explicitly surfaced as failed. This is a correctness target, not a percentage.
- Transcript scroll holds the frame-rate floor (Part 8) on Tier-B hardware with a 50,000-message history.

- Day-7 retention of new users who sent ≥1 message: the primary product health number.

UI behaviour.

- Composer per Part 2 §2.14 and Part 3 §3.2. Send is optimistic: the bubble appears instantly with a pending glyph.
- **Delivery states** are four, each with a distinct *glyph* (not just colour): pending (hollow clock) → sent (single tick) → delivered (double tick) → read (double tick, filled). Read receipts are reciprocal — turning yours off turns off your ability to see others'.
- Consecutive messages from one sender within 60 s group into a run with tightened spacing and shared corner geometry (Part 2 §2.10).
- **Reply** quotes the target inline above the new message; tapping the quote jumps to the original and briefly highlights it.
- **Reactions** are a single emoji per person per message, shown as a compact pill below the bubble. Tapping the pill lists who reacted.
- **Edit** is allowed for 15 minutes, marks the message "edited," and keeps a viewable edit history. **Delete for me** is always available; **delete for everyone** is allowed for 48 hours and leaves a tombstone ("This message was deleted") rather than silently altering history.
- **Disappearing messages** are a per-conversation setting (off / 24 h / 7 d / 90 d), applied from the moment it is set, announced in the transcript, and changeable by any participant with a visible system message.
- Link previews are generated **on the sender's device** and sent as part of the message. The recipient's device never contacts the linked site — this closes an IP-leak vector that most messengers still have open.

Edge cases.

- *Clock skew:* ordering uses the server-assigned per-conversation sequence number, never device clocks. Display timestamps use device time but ordering never does.
- *A recipient device that has been offline for 6 months:* the server retains undelivered envelopes for 30 days; beyond that, the sender's other devices backfill on reconnect where possible, and history that no live device holds is genuinely gone. This is stated in the interface, not hidden.
- *Message arrives before the conversation exists* (first contact): the conversation is created locally from the envelope; the sender lands in message requests (Part 4 §4.7).
- *Very long message* (>4,000 characters): collapsed with "Show more" rather than rejected.
- *Simultaneous edit from two linked devices:* last-writer-wins by device timestamp; the losing edit is preserved in edit history and never silently discarded.
- *User deletes for everyone while a recipient is offline:* the deletion is queued as its own envelope; it always beats a pending edit on arrival.

Failure cases.

- *No network:* message sits in the outbox with a clock glyph, sends on reconnect, order preserved (Part 3 §3.7).
- *Send rejected by the server (rate limit, blocked, account suspended):* the specific reason is shown inline; a rate limit says when to retry.
- *Decryption failure* (missing session, corrupted envelope): the message renders as "Couldn't decrypt this message" with a one-tap "Ask sender to resend," which sends a machine-readable retransmit request. We never render a decryption failure as an empty bubble — silent loss is the worst failure mode in a messenger.
- *Local database corruption:* the client detects it on open, rebuilds the index from the message store, and if the store itself is unrecoverable, re-syncs from the server what the server still holds — telling the user exactly what could and could not be recovered.

Future roadmap. Phase 2: scheduled send, message pinning, per-conversation themes, formatted text (bold/italic/lists/code), threads inside groups. Phase 3: MLS group protocol migration (Part 12 ADR-003), post-quantum key agreement, secret chats with per-device isolation.

5.2 Voice Notes

Purpose. Say it faster than you can type it, and let the recipient consume it however suits them.

User problem. Voice notes are the fastest way to communicate warmth and nuance, and the worst way to receive information — you cannot skim them, cannot search them, cannot listen in a meeting, and cannot tell whether a 4-minute note contains anything urgent.

Success metrics.

- Record-start latency (touch to first captured sample) **< 120 ms**.
- Transcript availability **> 95 %** of notes in the top 12 supported languages, generated on-device.
- Playback abandonment (started, stopped before 30 %) **< 20 %** — a proxy for "the transcript let people skip what they didn't need."

UI behaviour.

- Long-press the mic to record; slide up to lock hands-free; slide left to cancel with an unmistakable cancel affordance. Release to review-and-send, not to send blind — a one-tap send follows, so an accidental release never fires off an unintended note.
- A live waveform renders during recording, and the same waveform is the playback scrubber.
- **Playback speed** 1×/1.5×/2×, remembered per user.

- **Transcript** available under every note via a single tap; the sender can also see their own transcript *before* sending.
- Playback continues across conversations and into the background, with a compact persistent control.
- Notes played through the earpiece when the phone is raised to the ear, speaker otherwise.

Edge cases.

- *Interrupted by an incoming call:* recording stops and is saved as a draft, never lost.
- *Extremely long note* (>10 min): capped at 15 minutes with a visible countdown from 14:00.
- *Silent recording* (microphone muted at OS level, or a dead mic): detected on stop; we warn before sending rather than delivering silence.
- *Bluetooth device connects mid-recording:* recording continues on the original input; switching inputs mid-note is not attempted, as it produces audible artefacts.
- *Language not supported for transcription:* the note sends normally, with a quiet "Transcript not available for this language" rather than a broken transcript.

Failure cases.

- *Microphone permission denied:* an inline explanation with a one-tap route to settings; the mic control never simply does nothing.
- *Upload fails:* the audio is retained locally and retried; the user is never told it sent when it didn't.
- *Transcription model fails or times out:* the note is unaffected; the transcript control shows "Couldn't transcribe" with a retry.

Future roadmap. Phase 2: voice-note replies with quoted audio segments, noise suppression on-device, transcript search integrated into global search. Phase 3: instant voice-to-voice translation of notes (with the original always attached, per Part 4 §4.4).

5.3 Voice Calls

Purpose. One-to-one and small-group real-time voice that connects faster and sounds better than the phone network.

User problem. VoIP calls in existing apps take 3–8 seconds to connect, degrade unpredictably, and give no honest indication of what is wrong when they do.

Success metrics.

- **Time-to-ring < 1.2 s** (initiate → callee's device rings) at p75.
- **Mouth-to-ear latency < 200 ms** at p75 on a 4G connection.

- **Call setup success rate > 99 %**; drop rate < 1 % per 10-minute call.
- MOS ≥ 4.0 at p50 on the reference network profile.

UI behaviour.

- Full-screen incoming call with two large targets (answer/decline), reachable one-handed, plus a "reply with message" option.
- In-call: mute, speaker, video-upgrade, add-participant, end. Five controls, in the natural thumb zone, nothing hidden.
- **Honest network state:** a connection-quality indicator that names the actual problem ("Your connection is unstable") rather than a generic bar count.
- Ongoing calls collapse to a system-level pill so the user can use the rest of the app.
- Calls are E2EE; the encryption state is shown once at connect, not as a persistent badge.

Edge cases.

- *Callee on another call:* caller hears busy state immediately, callee gets a call-waiting prompt.
- *Both call each other simultaneously:* deterministic resolution by user ID ordering; one call is auto-answered into the other, no double-ring.
- *Network switch mid-call* (Wi-Fi → cellular): the session migrates without dropping; a brief "Reconnecting" state, and if it exceeds 20 s the call ends cleanly with a recorded duration.
- *Callee has no compatible device linked:* stated plainly before dialling.
- *Do Not Disturb / focus modes:* honoured; the platform's repeat-caller escape hatch is supported.

Failure cases.

- *NAT traversal failure:* automatic relay fallback (TURN); the user sees nothing except slightly higher latency.
- *No audio path* (headset routing failure): detected within 3 s and surfaced with a route picker, rather than leaving two people saying "hello?"
- *Server capacity exhaustion:* calls fail *before* ringing, with a clear message — never a ring that cannot connect.

Future roadmap. Phase 2: group voice up to 32, call recording with all-party consent, live captions. Phase 3: spatial audio for groups, live translated calls (Part 4 §4.4), voice rooms for communities.

5.4 Video Calls

Purpose. See the person. Work on a bad connection. Never be surprised by how you look or what is behind you.

User problem. Video calls fail hardest exactly where they are needed most — low bandwidth, cheap devices, poor light. Most apps respond by freezing video and dropping audio, which is precisely backwards.

Success metrics.

- **Audio survives to 40 kbps** — when bandwidth collapses, video degrades progressively and audio is protected absolutely. Audio never sacrifices for video. This is the defining engineering rule of the feature.
- 720p30 sustained at p75 on a 5 Mbps connection; graceful ladder down to 180p and then to audio-only.
- Connect time < 2 s at p75; battery drain < 12 % per hour of 1:1 video on the Tier-B reference device.

UI behaviour.

- **A pre-call preview by default** — see your camera, your background, and your mic level before you are visible to anyone.
- Self-view is a small draggable tile that can be minimised entirely; it snaps to corners and never covers the active speaker.
- Controls auto-hide after 4 s and return on any tap.
- Group video (up to 8 at MVP) uses an active-speaker layout with a filmstrip; no gallery grid on phone, because 8 faces on a 6″ screen is 8 unrecognisable faces.
- One-tap switch between voice and video, in either direction, mid-call, with the other party notified.

Edge cases.

- *Rotation mid-call:* re-layout without renegotiating the media session.
- *Backgrounding the app:* video pauses with a clear “Camera off” state shown to others; audio continues.
- *Very slow device:* the encoder ladder is capped by measured device capability at call start, not attempted and failed.
- *Participant with no camera:* joins audio-only with an avatar tile; no error.

Failure cases.

- *Camera permission denied or camera busy:* the call proceeds as audio with a clear inline explanation.

- *Sustained packet loss > 15 %:* automatic step-down through the quality ladder, with one honest message ("Poor connection — video paused").
- *Encoder crash:* the call recovers to audio-only rather than ending.

Future roadmap. Phase 2: screen sharing, background blur (on-device), live captions, group video to 32. Phase 3: real-time translated subtitles; low-bandwidth mode targeting 2G-class networks.

5.5 Group Conversations

Purpose. A shared space for a family, a team, or a community of up to 1,000 people, that stays legible as it grows.

User problem. Groups get loud, then get muted, then get abandoned. The failure is structural: every message is treated as equally urgent, and there is no way to participate partially.

Success metrics.

- Groups still active (≥1 message/week) at day 90 after creation: **> 40 %**.
- Mute rate below 30 % for groups under 20 members — high mute rates mean the notification model is failing (Part 3 §3.9).
- Fan-out delivery: a message to 1,000 members reaches p95 of online devices within **1 s**.

UI behaviour.

- Any user can create a group; a name is optional (a default is derived from members) and a photo is optional.
- **Roles are minimal at MVP:** owner and member. Owners can rename, set the photo, add/remove members, and configure who may add others. Full role systems are deliberately deferred (Part 10).
- **@mentions** cut through mute — this is what makes mute safe to use, and therefore what makes groups survivable.
- **Invite links** with optional expiry and optional admin approval. Links are revocable and rotate on demand.
- **Join/leave system messages** are collapsed into a single line per hour rather than one per event; noise from membership churn is the most common reason groups become unreadable.
- Group profile lists members with a search field, and shows shared media, files, and links.

Edge cases.

- *Last owner leaves:* ownership transfers to the longest-tenured active member, announced in the transcript.
- *Member removed while offline:* they receive the removal on reconnect and retain their local history — we cannot and do not delete data from a device we do not control, and we say so rather than implying otherwise.
- *A user is added to a group by someone they blocked:* the add is rejected. Blocks are transitive to group invitations.
- *1,000-member group where 300 come online at once:* fan-out is queued and rate-shaped server-side; no client behaviour changes.
- *Someone joins a group and scrolls to the beginning:* new members see history from their join point by default; the owner may enable full-history sharing at creation, and it cannot be changed retroactively (it is an encryption-key decision, not a policy toggle).

Failure cases.

- *Partial fan-out failure:* per-recipient retry with an eventual delivery guarantee; the sender's ticks reflect real per-device delivery, not an optimistic aggregate.
- *Sender-key rotation failure* after a member leaves: the group is held in a re-keying state for at most 10 s with a visible indicator; messages queue, nothing is sent under a key the departed member holds.
- *Group state divergence between devices:* the server's membership state is authoritative and reconciles on reconnect.

Future roadmap. Phase 2: threads, polls, admin roles and permissions, group description and rules, member-level muting. Phase 3: communities (groups of groups), events, moderation tooling, groups above 1,000 via MLS.

5.6 Channels

Purpose. One-to-many broadcast — a creator, a school, a business, or a government agency reaching subscribers — without becoming a feed.

User problem. Announcements today happen in groups (where 500 people can reply and destroy the signal) or on social platforms (where an algorithm decides whether the message is seen at all). Neither is acceptable for information that matters.

Success metrics.

- **Delivery rate to subscribers: 100 %.** There is no ranking, so there is no reason for a subscriber to miss a post. This is the entire value proposition, and it is a correctness metric, not a percentage to optimise.
- Subscriber retention at day 30 > 70 %.
- Median time from post to p95 device delivery **< 2 s** for channels under 100,000 subscribers.

UI behaviour.

- Channels appear **in the Chats list** behind a filter chip — not as a separate destination (Part 3 §3.1). A channel you follow is a conversation you subscribe to.
- **Chronological, always.** No ranking, no "suggested," no engagement sorting. Ever (Part 0.6 clause 2).
- Subscribers can react and, if the owner allows, comment in a discussion area that is visually separate from the posts.
- Channels have a public link, a handle, and a verified state for identity-verified organisations.
- **Channels are not end-to-end encrypted, and the interface says so** on the channel's header and at subscribe time. A public broadcast to 100,000 strangers is not a private conversation, and pretending it is would be dishonest. Content is encrypted in transit and at rest.
- Basic analytics for owners: subscriber count, post reach, reactions. No demographic profiling of subscribers — we do not collect it, so we cannot report it.

Edge cases.

- *A channel grows past 1M subscribers:* fan-out shifts to a pull-based model with push wake-ups; users notice nothing.
- *Owner deletes a post:* removed for everyone; a tombstone appears only if the post had been delivered.
- *Subscriber has notifications off:* posts still arrive, silently, in the list.
- *A channel is reported:* moderation applies to channels, which are public content, under the published policy (Part 11 §11.7). Private conversations are not moderated because they cannot be — a distinction we state clearly rather than blurring.

Failure cases.

- *Post fails partway through fan-out:* the post is durable server-side; delivery resumes and completes. Owners see true delivery counts.
- *Impersonation of a known organisation:* handled by verification plus proactive name-similarity detection at creation time, not by reactive takedowns alone.

Future roadmap. Phase 2: scheduled posts, post drafts, multi-admin, richer analytics, paid subscriptions. Phase 3: creator monetisation (Part 11), live audio broadcast to subscribers.

5.7 Stories

Purpose. Lightweight, ephemeral sharing with the people you actually talk to.

User problem. Stories in existing apps have become performance venues with viewer counts, rankings, and pressure. The original idea — a low-stakes glimpse of your day for people who know you — is worth keeping; the anxiety machinery around it is not.

Success metrics.

- Share-back rate: proportion of viewers who *reply* rather than merely view. This is the health metric; a story that is watched but never answered is content, not communication.
- Posting frequency stable over time (not growing) — if we see it climbing, we are creating pressure, which is a failure.
- Zero notifications generated by stories, verified continuously in production.

UI behaviour.

- Stories live in their own destination, not as a ring above the conversation list — putting stories above the inbox is the design decision that turned messaging apps into social apps, and we are declining it deliberately.
- 24-hour expiry. Photo, video (≤30 s), or text on a solid colour. No filters at MVP beyond crop and rotate.
- **Audience is chosen per story:** all contacts, selected people, or everyone except selected people. The audience is shown before posting, every time.
- **No public viewer counts.** The poster sees who viewed; nobody else sees anything. There is no ranking of who watched most.
- Replies to a story open a normal 1:1 conversation with the story quoted — this is the point of the feature.
- Stories are end-to-end encrypted to the chosen audience.

Edge cases.

- *Viewer's clock is wrong:* expiry is enforced server-side and client-side against server time.
- *Poster deletes early:* removed from all viewers immediately; already-viewed copies are removed from the viewer's cache.
- *Screenshot:* not detected and not reported. Screenshot detection creates false security; we tell users plainly that anything they post can be captured.
- *Large video on a slow connection:* uploads in the background with a progress row; the story posts when it completes.

Failure cases.

- *Upload fails:* draft is retained locally, retried, never silently discarded.
- *Media processing fails:* the original is preserved and posted at reduced quality rather than lost.

Future roadmap. Phase 2: story replies with reactions, close-friends list, text/drawing tools, mute-a-poster. Phase 3 and beyond: nothing. Stories are deliberately capped — see Part 10 for why Reels-style content is rejected outright.

5.8 AI Assistant

Purpose. Ambient, invited intelligence: translation, transcription, summarisation, recall, and fraud protection — delivered inside conversations, plus one dedicated assistant conversation for direct questions.

User problem. People need help across a language barrier, in a long unread group, with a voice note they cannot play, and against fraudsters — and today they must leave the conversation and go to a different app to get any of it.

Success metrics.

- **On-device execution rate > 85 %** of all AI invocations. This is the primary metric, because it is the privacy promise made measurable.
- Translation user-reported accuracy ≥ 90 % in the top 12 language pairs.
- Scam-warning precision **> 95 %** per pattern class, with automatic rollback below that (Part 4 §4.7).
- **Zero** AI invocations on E2EE content without a recorded, user-visible grant — audited continuously; any non-zero result is a Sev-1 incident.
- p95 added latency to any core interaction from an AI feature: **0 ms** (AI is never in the critical path of sending or receiving).

UI behaviour. Fully specified in Part 4. In summary: Aurora colour exclusively; labelled; never sends on the user's behalf; disableable entirely; a permanent Privacy Centre showing every grant.

Edge cases.

- *Model unavailable / not yet downloaded:* the feature is hidden rather than shown-and-broken, with a one-tap download offer.
- *Device too weak for on-device models* (below Tier-C): AI features are offered only in their server form, with explicit consent, and the user is told why.
- *User revokes a grant mid-operation:* the operation is cancelled and any server-side copy is deleted within 24 hours, verified by audit.
- *Mixed-language conversation:* per-message language detection, not per-conversation.

Failure cases.

- *Model produces nothing or times out:* "Couldn't do that" with a retry. Never a fabricated fallback.

- *Server-side provider outage:* on-device features are unaffected — a deliberate architectural benefit of the on-device-first rule.
- *Wrong or harmful output:* one-tap report on every AI element, routed to a standing review queue with weekly triage.

Future roadmap. Phase 2: call summaries with all-party consent, richer assistant tools, live captions. Phase 3: live call translation, cross-device assistant memory. Phase 4: assistant actions on user data (find, organise, schedule) under an explicit permission model.

5.9 File Sharing

Purpose. Send any file to anyone, encrypted, without a size limit that forces people back to email.

User problem. File sharing in messengers is arbitrarily capped, silently recompresses images into mush, and loses files when the sender's device changes.

Success metrics.

- Upload success rate > 99.5 % including resumed uploads.
- Median time-to-first-byte on download < 300 ms via edge cache.
- Image quality: **original-quality sending is the default option shown**, not buried. Complaints about recompression: near zero.

UI behaviour.

- Limits: **2 GB per file free, 10 GB on Plus.** Stated up front, not discovered at 99 %.
- **Resumable uploads and downloads** across app restarts and network changes.
- Send photos at original quality or compressed — the choice is offered on send, with the resulting size shown for both, and the choice is remembered per conversation.
- Files, media, and links are browsable per conversation from the conversation profile.
- Every file is client-side encrypted with a per-file key; the storage layer holds ciphertext only.
- Downloaded files land in the platform's normal file location and are visible to the user's own file manager — they are the user's files.

Edge cases.

- *Storage full on the receiving device:* checked before download starts, with a clear message and the required amount.
- *File type blocked by the platform* (e.g. iOS restrictions): stated before sending, not after.

- *Executable file types:* delivered but marked with a plain warning; we do not block file types unilaterally, because it breaks legitimate use, and we do not scan file contents, because we cannot decrypt them.
- *Sender deletes the file locally after sending:* the encrypted copy on the server persists for the retention window; recipients are unaffected.

Failure cases.

- *Upload interrupted:* resumed from the last committed chunk; never restarted from zero.
- *Storage backend unavailable:* upload is queued locally and retried; the message shows a pending state rather than a failure.
- *Retention expiry* (files are retained 90 days free / 1 year on Plus after last access): the user is warned at 7 days, and expiry is shown in the file's info. No file ever silently vanishes.

Future roadmap. Phase 2: folders in shared media, in-app document preview, cross-conversation file search. Phase 3: cloud storage tier with unlimited retention, collaborative documents.

5.10 Search

Purpose. Find any message, person, file, or conversation instantly, without our servers being able to read any of it.

User problem. Search in encrypted messengers is usually terrible, because doing it properly requires an on-device index that most products never invested in.

Success metrics.

- **Local results within 50 ms of a keystroke** at p95, over a 100,000-message history.
- Search success rate (a search followed by a result tap) > 70 %.
- Index size < 5 % of the message store; index build for 100,000 messages < 30 s on Tier-B hardware, running in the background without blocking the UI.

UI behaviour. Fully specified in Part 3 §3.8: one field, local-first, chips for filters, semantic results as a labelled second group.

Edge cases.

- *Search in a language with no word boundaries* (Chinese, Japanese, Thai): n-gram indexing rather than word tokenisation.
- *RTL and mixed-direction queries:* normalised before indexing, with correct bidi rendering of highlighted results.
- *Diacritics and Arabic orthographic variants:* normalised on both index and query, so a search without diacritics finds text with them.

- *Newly linked device with no history:* search states honestly that it covers messages on this device, and offers to sync more.
- *Index rebuild in progress:* search still works against the message store directly, slower, with a quiet indicator.

Failure cases.

- *Index corruption:* detected on open, rebuilt in the background; search degrades to a slower direct scan rather than failing.
- *Query too broad:* results are streamed progressively rather than blocking on a full scan.

Future roadmap. Phase 2: search inside file contents on-device, search within voice-note transcripts. Phase 3: cross-device federated local search (query fanned to your own linked devices, results returned encrypted — never through a server that can read them).

5.11 Profile System

Purpose. Be findable by the people you want, and invisible to everyone else.

User problem. Most messengers make the phone number the identity, which means anyone with a number can reach you, and leaving means losing everything.

Success metrics.

- Profile completion (display name + photo) > 80 %.
- **Percentage of accounts created without a phone number**: tracked as a strategic metric, because phone-optional identity is a differentiator.
- Unwanted-contact reports per 1,000 users: the number this feature exists to keep near zero.

UI behaviour.

- **Identity is a username** (`@handle`), globally unique, changeable twice a year, with the old handle reserved for 90 days.
- A phone number or email is used for account recovery and is **optional to expose**. Discoverability by phone/email is off by default.
- Profile carries: display name, photo, optional short bio, optional links. No follower counts, no “last seen” by default.
- **Privacy controls are per-field**, each with three options — everyone / contacts / nobody: profile photo, bio, last seen, read receipts, story audience, who can add me to groups, who can call me.
- **QR code and share link** are the primary ways to connect (Part 3 §3.10 rule 5).
- **Safety numbers** per contact for out-of-band verification, rendered in a monospaced face with an unambiguous glyph set (Part 2 §2.13).

- **Block** is complete and silent: blocked users see no state change, and receive no signal that they were blocked.

Edge cases.

- *Username squatting:* handles inactive for 12 months are reclaimable through a published process; trademark disputes follow a documented policy.
- *Changing a username mid-conversation:* existing conversations follow the account, not the handle; a system message notes the change so impersonation is harder.
- *Two accounts, same display name:* the handle disambiguates and is always shown in ambiguous contexts.
- *Account recovery with no phone or email:* the user is warned clearly at signup that recovery will be impossible, and must acknowledge it. We will not build a recovery backdoor.

Failure cases.

- *Recovery method lost:* the account and its history are unrecoverable. This is a direct consequence of E2EE, we state it at signup, and we do not soften it.
- *Impersonation report:* handled by a verification process, with proactive similarity checks at registration.

Future roadmap. Phase 2: multiple accounts on one device, profile verification for organisations, per-contact nicknames. Phase 3: portable identity/key export, business profiles.

5.12 Notifications

Purpose. Tell the user when a human needs them, and never otherwise.

FIGURE — A NOTIFICATION THAT THE SERVER CANNOT READ

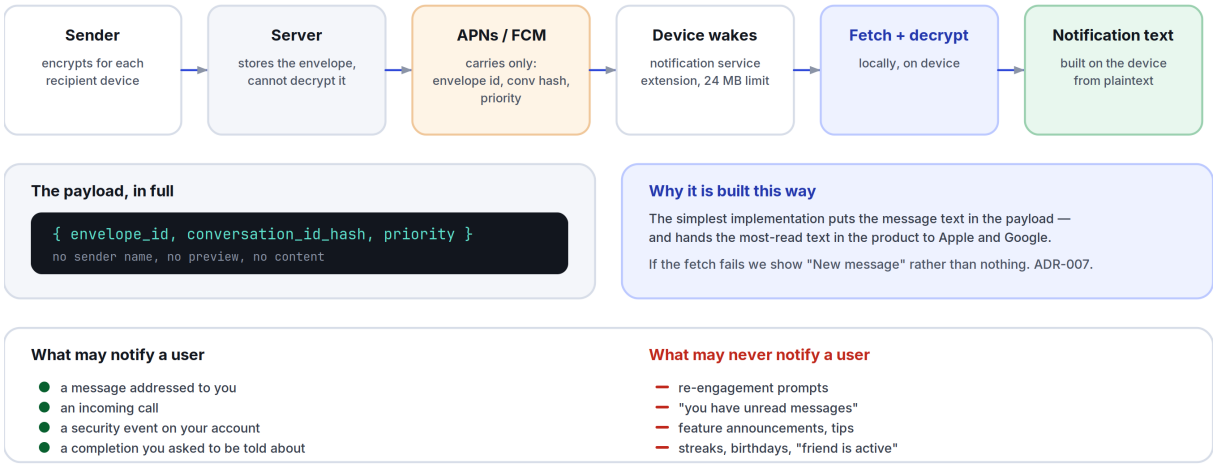


Figure 5.2 Push carries a wake-up signal and nothing else; the notification text is assembled on the device.

User problem. Notifications have become an advertising channel. Users respond by turning them off entirely, which breaks the actual purpose of a messenger.

Success metrics.

- **Notification opt-out rate < 10 %** — the single clearest measure of whether we kept the promise.
- Delivery latency p95 **< 2 s** from send to device notification.
- Notification-to-open rate > 40 % (high, because every notification should matter).
- **Zero** non-human-originated notifications in production, verified by an automated audit of every notification type on every release.

UI behaviour. Fully specified in Part 3 §3.9. Key architecture: push payloads carry **no content** — only an encrypted envelope and a conversation identifier; the device decrypts locally to build the text (Part 6 §6.6).

Edge cases.

- *Push service unavailable* (no Google services, or a restricted network): a persistent foreground socket with battery-aware backoff, and the trade-off is explained to the user rather than silently degrading.
- *Device asleep in a deep power state:* high-priority push wake-up, with the platform's own rate limits respected.
- *Notification arrives for a message already read on another device:* dismissed automatically across all linked devices within 1 s of the read event.
- *Aggressive battery managers* (a real, documented problem on several Android OEM builds): detected, with a one-time explanation and a direct route to the relevant setting.

Failure cases.

- *Decryption fails on the notification path:* fall back to "New message" rather than showing nothing — a silent notification failure is indistinguishable from being ignored.
- *Push token invalid:* re-registered on next foreground; undelivered messages arrive on reconnect.

Future roadmap. Phase 2: per-conversation custom sounds, quiet hours schedules, notification summaries for muted groups. Phase 3: cross-device notification intelligence (notify only the device you are actually using).

5.13 Multi-Device Support

Purpose. Use SpaceTalk on your phone, your laptop, and your tablet — all end-to-end encrypted, with no device designated as the master that must stay online.

User problem. Multi-device support in encrypted messengers has historically been either absent, or implemented as a tethered mirror that dies when the phone's battery does.

Success metrics.

- Up to **4 linked devices** per account at MVP.
- Sync latency between linked devices **< 500 ms** at p95 when both are online.
- **Phone-independent operation:** a linked device works with the phone off. Verified as a release gate.
- Link-flow completion rate > 90 %.

UI behaviour.

- **Each device has its own identity key**; messages are encrypted per-device (Part 12 ADR-003). There is no key sharing between devices, and no primary device.
- Linking is a QR-code scan from an already-linked device, with the new device's safety number shown on both screens for confirmation.
- **Every linking event generates a notification on all existing devices** and a permanent entry in a device log that shows: name, platform, when linked, last active, and current location by coarse region. Silent device addition is the attack that breaks E2EE for real people, and it must be loud.
- Any device can unlink any other device, immediately.
- History sync to a new device is explicit and user-controlled: none, last 30 days, or everything — with the transfer size shown before it starts.
- At MVP the linked-device client is the **Flutter desktop-class web client**; iOS and Android are full clients (Part 12 ADR-012).

Edge cases.

- *Device linked while offline:* the link is pending until it can be confirmed by a live device; it never activates on the strength of a stale QR code (codes expire in 60 s).
- *All devices lost:* the account is recoverable via the recovery method, but the message history is not (§5.11 failure case). Stated at signup.
- *A device is compromised:* unlinking it immediately rotates all group sender keys and invalidates its sessions; messages sent to it before unlinking were already delivered and cannot be recalled, and we say so.
- *Clock skew between devices:* server sequence numbers, never device clocks (§5.1).
- *Same account, two devices editing the same draft:* drafts are per-device and are not synced. Attempting to sync drafts creates more confusion than it resolves.

Failure cases.

- *Sync divergence* (one device missing messages): a per-conversation sequence-gap detector requests retransmission automatically; unrecoverable gaps are shown in the transcript as an explicit "Some messages could not be synced here" marker rather than an invisible hole.

- *Session establishment failure with a new device:* messages queue and retry; the sender sees a partial-delivery state, never a false "delivered."

Future roadmap. Phase 2: native macOS/Windows/Linux clients, unlimited history sync, per-device notification preferences. Phase 3: wearables (notification and reply only), car integration via platform standards, TV for calls only.

5.14 Cross-Cutting Requirements

Every feature above must also satisfy, without exception:

Requirement	Where specified
Meets the performance budget for its surface	Part 8
Full RTL, bidi, and 200 % dynamic-type support	Part 2 §2.13, §2.15
Screen-reader complete, keyboard complete	Part 3 §3.11
Works offline or degrades honestly	Part 3 §3.7
Every string localised, no concatenated sentences	Part 11 §11.6
Errors follow the severity ladder	Part 3 §3.5
No notification unless a human is addressing the user	Part 3 §3.9
Data exportable and deletable	Part 0.6 clause 10
Instrumented for its own success metrics, and no others	Part 11 §11.5

PART 06

06

The Technical Bible

Part 6 — Architecture, Engineering, and Operations

Governed by Part 0. Decisions with lasting consequence are recorded as ADRs in Part 12 and referenced here. This document describes the system we intend to build for the MVP and the first two phases beyond it — not a hypothetical system for a billion users. Scale plans that are not yet needed are marked as such.

> "Design so that a compromised server still cannot read what we promised not to read."

IN THIS PART

6.1 Architectural Principles

1. **Boring where it doesn't differentiate.** We are inventing in exactly two places: client performance and the on-device intelligence layer. Everywhere else — datastores, queues, deployment — we use the most well-understood option available.
2. **Local-first.** The device's database is the source of truth for the interface. The network is a synchronisation mechanism, not a data source (ADR-008).
3. **The server should be unable to do the things we promise not to do.** Design so that a compromised server, a subpoenaed server, or a malicious insider still cannot read message content.
4. **One codebase per concern.** One client codebase for all platforms (ADR-001). One backend language (ADR-002).
5. **Scale when measured, not when imagined.** Every scaling step in §6.10 has a numeric trigger. Building for 100 M users at 100 K users is how startups die with excellent architecture diagrams.
6. **Every dependency is a liability.** A dependency must save more work than the sum of its upgrade, audit, and outage costs over three years.

FIGURE — SYSTEM ARCHITECTURE

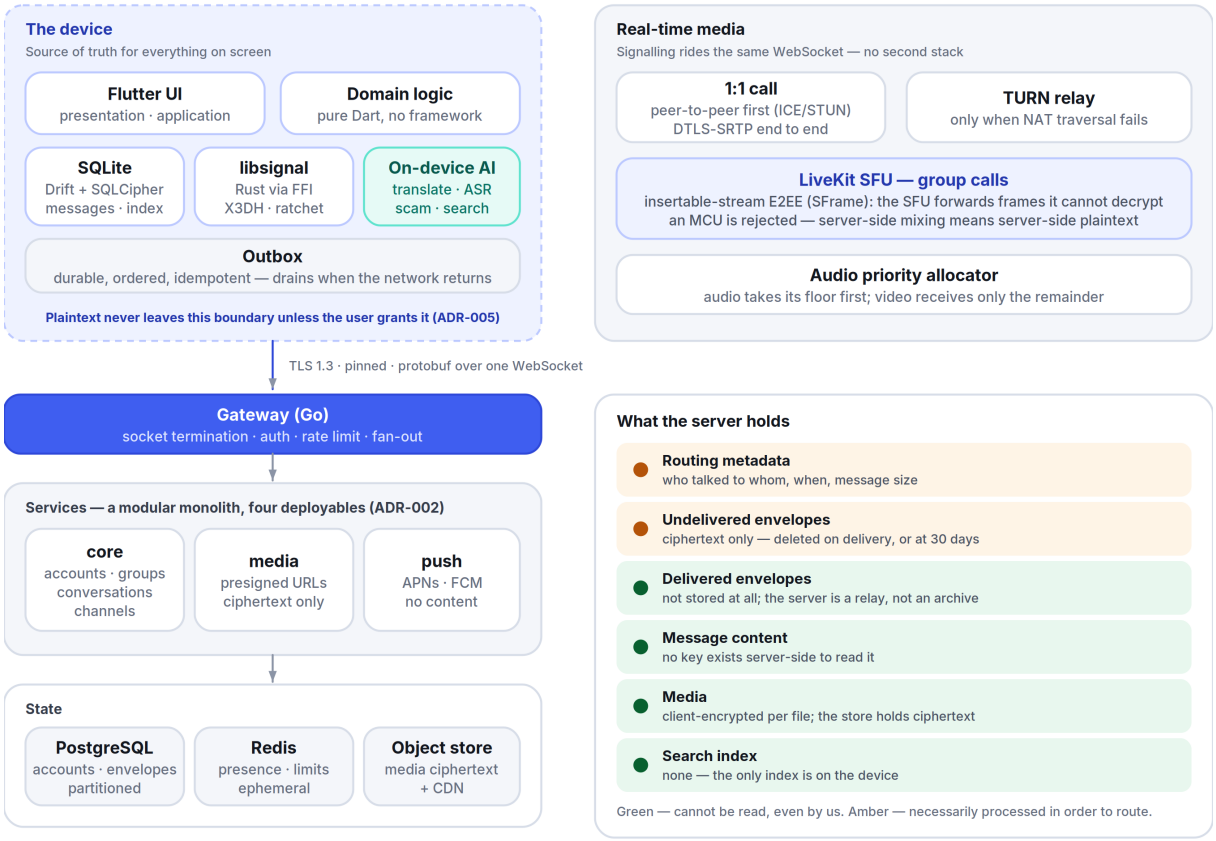


Figure 6.1 System architecture — the device is the source of truth; the server is a relay that cannot read what it carries.

6.2 Client: Flutter

Flutter 3.x, Dart 3.x, one codebase for iOS, Android, web, and desktop (ADR-001).

Layering.

```
presentation/      Widgets. Stateless where possible. No business logic. No I/O.
application/       Riverpod providers, view-models, orchestration.
domain/            Pure Dart entities and use-cases. Zero framework imports.
                   Fully unit-testable with no mocks of Flutter.
data/              Repositories, local store (Drift/SQLite), remote (gRPC/WS), crypto FFI.
platform/          Thin channels for notifications, camera, keystore, background tasks.
```

Dependencies point inward only. `domain` imports nothing from the other layers, which is what makes the business logic testable at speed.

State management: Riverpod. Chosen for compile-time safety, testability without a widget tree, and fine-grained rebuilds — the last matters because the transcript must rebuild individual bubbles, not the list.

Local database: SQLite via **Drift** (typed queries, migrations, and a compile-time schema). SQLCipher for at-rest encryption of the message store, with the key held in the platform keystore (iOS Keychain with `kSecAttrAccessibleAfterFirstUnlockThisDeviceOnly`; Android Keystore with StrongBox where available).

Cryptography: `libsignal` (Rust) via `dart:ffi`, not a Dart reimplementation (ADR-003). This is the single highest-risk integration in the client and is treated as such: pinned versions, reproducible builds, and a conformance test suite run against the official test vectors on every commit.

Rendering rules that follow from the performance targets (Part 8):

- The transcript is a `CustomScrollView` with slivers and stable keys; message heights are measured once and cached, so scrolling never re-measures.
- Images decode off the platform thread at their *display* resolution, never full resolution — the most common source of jank and out-of-memory kills on cheap Android devices.
- `RepaintBoundary` around every bubble; no `Opacity` or `ClipRRect` in scrolling content (both force expensive save layers) — rounded corners come from `ShapeDecoration`.
- No `setState` above a list item. Ever.
- Shader warm-up on first launch to avoid the first-run jank Flutter is known for; verified with `--purge-persistent-cache` in the performance test suite.

Platform split. iOS and Android are full clients at MVP. The linked-device client is the Flutter web build (ADR-012); native desktop follows in Phase 2 from the same codebase.

6.3 Backend

Go 1.2x, a modular monolith, deployed as a handful of services (ADR-002). Not microservices. Not Kubernetes at MVP.

FIGURE — BACKEND SERVICES AND THEIR SCALING TRIGGERS

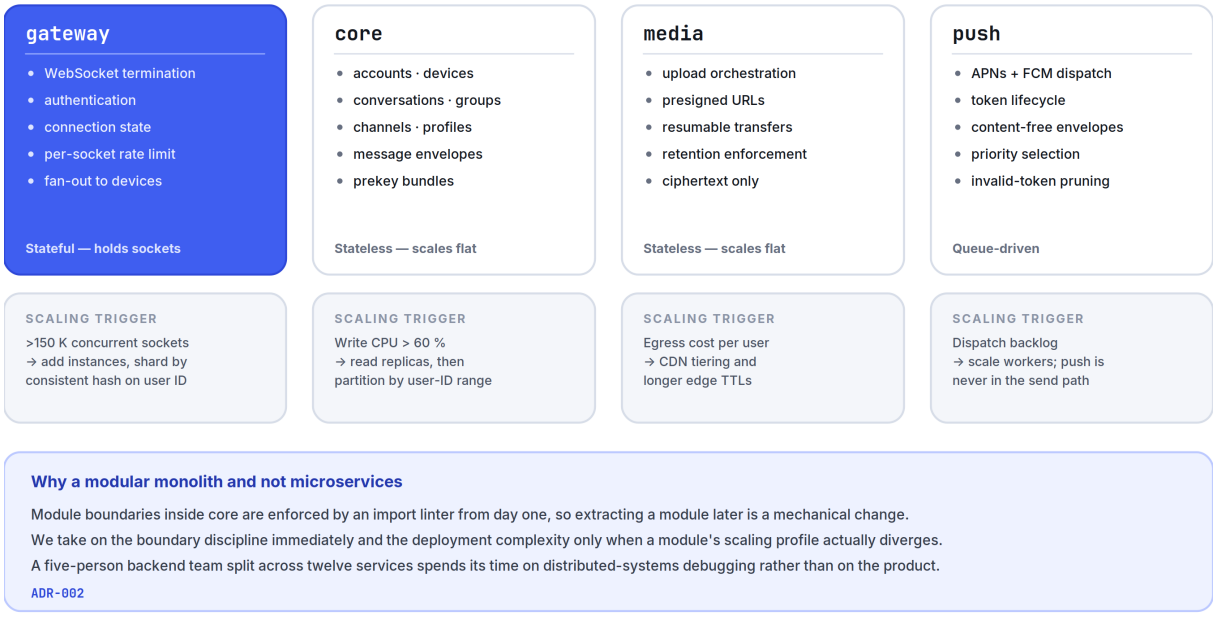


Figure 6.2 The four Phase 1 deployables, what each owns, and the measured trigger that would split it out.

Services at MVP (four deployables):

Service	Responsibility	Scaling property
gateway	WebSocket termination, authentication, connection state, per-connection rate limiting, fan-out to connected devices	Stateful (holds sockets); scales horizontally with consistent hashing over user ID
core	Everything transactional: accounts, conversations, message envelopes, groups, channels, profiles, devices	Stateless; scales horizontally
media	Upload/download orchestration, presigned URLs, thumbnail generation of *ciphertext-adjacent* metadata only	Stateless
push	APNs/FCM dispatch, token lifecycle, wake-up envelopes	Stateless; queue-driven

Why a modular monolith. At MVP, a five-person backend team splitting into twelve microservices spends its time on distributed-systems debugging rather than product. The module boundaries inside `core` are enforced by package structure and an import linter, so extracting a module into its own service later is a mechanical change. We take on the

boundary discipline immediately and the *deployment* complexity only when a module's scaling profile actually diverges. Trigger conditions are in §6.10.

Realtime transport. A single WebSocket per device carrying length-prefixed Protocol Buffers. Chosen over raw HTTP/2 streams for browser parity and over MQTT for control. The protocol is a small, versioned, bidirectional command/event stream — not JSON, because envelope overhead at 100 K messages/second is real money and real latency.

6.4 API Architecture

Three surfaces, deliberately different:

1. **Realtime (WebSocket + protobuf)** — everything in the hot path: send, receive, typing, presence, receipts, calls signalling. Bidirectional, ordered per conversation, with per-connection backpressure.
2. **Request/response (gRPC, with a gRPC-Web gateway)** — account operations, group management, media orchestration, settings. Strongly typed, versioned, code-generated for Dart and Go from one `.proto` source of truth.
3. **Public HTTP/JSON API (Phase 3)** — for business integrations and bots. Deliberately separate from the internal API so we are never forced to keep an internal shape stable for external reasons.

Contracts.

- Protobuf schemas live in one repository and are the single source of truth. Breaking changes require a new field number, never a reused one; a CI check (`buf breaking`) fails the build on any incompatible change.
- **Every client supports N and N−1 protocol versions.** Servers support N through N−3, giving roughly 12 months of client-upgrade runway before a forced update.
- Idempotency keys on every mutating call, so a retry after a timeout can never duplicate a message.
- Errors are typed enums, never strings parsed by clients.

Envelope model. The server stores and routes *envelopes*: `{conversation_id, sender_device_id, recipient_device_id, sequence, ciphertext, ciphertext_len, server_timestamp}`. The server can see who talked to whom and when — this is unavoidable metadata for routing — and it can see nothing else. Minimising and expiring that metadata is covered in §6.13.

6.5 Database

PostgreSQL 16 as the primary store at MVP (ADR-004), with Redis for ephemeral state and S3-compatible object storage for media.

Data	Store	Notes
Accounts, devices, prekeys, profiles	Postgres	Small, highly relational, strongly consistent
Conversations, membership, groups, channels	Postgres	Same
Message envelopes (undelivered)	Postgres, partitioned by month	Deleted on delivery to all devices, or at 30 days
Message envelopes (delivered)	**Not stored**	Devices hold history; the server is a relay with a retention window, not an archive. This is a privacy decision *and* a cost decision.
Media blobs	S3-compatible object storage + CDN	Client-side encrypted; the storage layer holds ciphertext only
Presence, typing, connection routing	Redis	Ephemeral, TTL'd, never persisted
Rate limits	Redis	Sliding window
Push tokens	Postgres	Rotated, pruned on invalidation
Search index	**On-device SQLite FTS5 only**	No server-side message index exists, because there is no server-side plaintext

Key schema decisions.

- Message envelopes are partitioned by month and dropped by partition, not deleted by row — deleting 100 M rows individually is how a database dies.
- `(conversation_id, sequence)` is the ordering authority, assigned server-side, monotonic per conversation, and never derived from a clock.
- Device fan-out is materialised at send time: one envelope row per recipient device, so delivery state is per-device and truthful (Part 5 §5.5).
- Prekey bundles are consumed atomically with `SELECT ... FOR UPDATE SKIP LOCKED`, with a last-resort signed prekey when one-time keys are exhausted.
- All destructive operations are soft-delete plus a scheduled purge, so a bug cannot cause instant irreversible loss.

Scaling path (triggered, not pre-built). When envelope write throughput sustains >30 K/s or the undelivered table exceeds ~500 GB, envelopes migrate to ScyllaDB behind the existing repository interface. That interface exists from day one precisely so the migration is a swap; Scylla itself is not deployed until the trigger fires (ADR-004).

6.6 Authentication, Identity, and Push

Registration. Username plus a recovery method (phone or email, optional but strongly recommended and clearly explained). Verification by code. Registration is rate-limited by IP, device attestation, and proof-of-work escalation under attack.

FIGURE — IDENTITY, SESSION, AND DEVICE LINKING

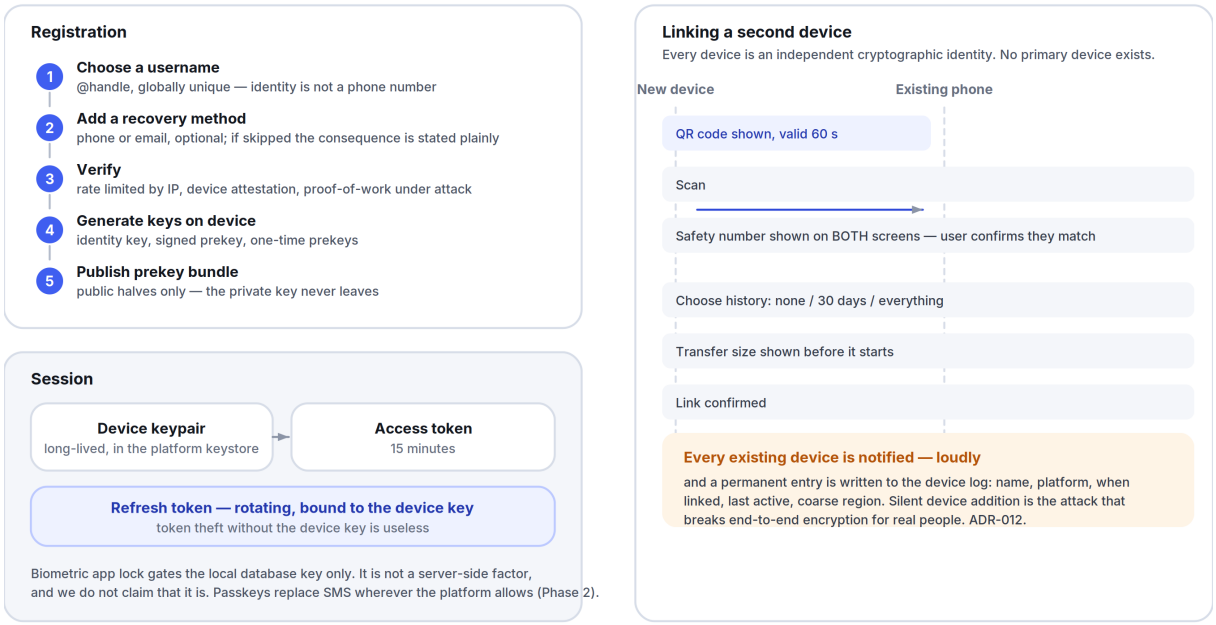


Figure 6.5 Registration, session handling, and the deliberately loud device-linking flow.

Session authentication. Per-device long-lived credentials backed by a device keypair; access tokens are short-lived (15 min) and refreshed with a rotating refresh token bound to the device key. Token theft without the device key is useless.

Passkeys / WebAuthn for account recovery from Phase 2, replacing SMS wherever the platform supports it — SMS is the weakest link in every messenger's security model and we intend to reduce our dependence on it rather than build on it.

Biometric app lock is local-only: it gates access to the local database key in the platform keystore. It is not a server-side authentication factor, and we do not claim it is.

Push notifications.

- APNs (with an iOS Notification Service Extension) and FCM.
- **Payloads contain no content**: `{envelope_id, conversation_id_hash, priority}`. The extension fetches the envelope and decrypts locally to build the notification (Part 5 §5.12).
- A fallback persistent socket exists for devices without Google services, with battery-aware backoff.

- Push tokens are rotated and pruned; an invalid token is a signal to prune, never to retry indefinitely.

6.7 Encryption

The protocol (ADR-003): **X3DH** for asynchronous key agreement, **Double Ratchet** for message encryption, via `libsignal`. We are not designing a protocol. Rolling our own cryptography would be the single most likely way for this project to cause real harm to real people.

FIGURE — WHERE THE ENCRYPTION PROMISE STARTS AND STOPS

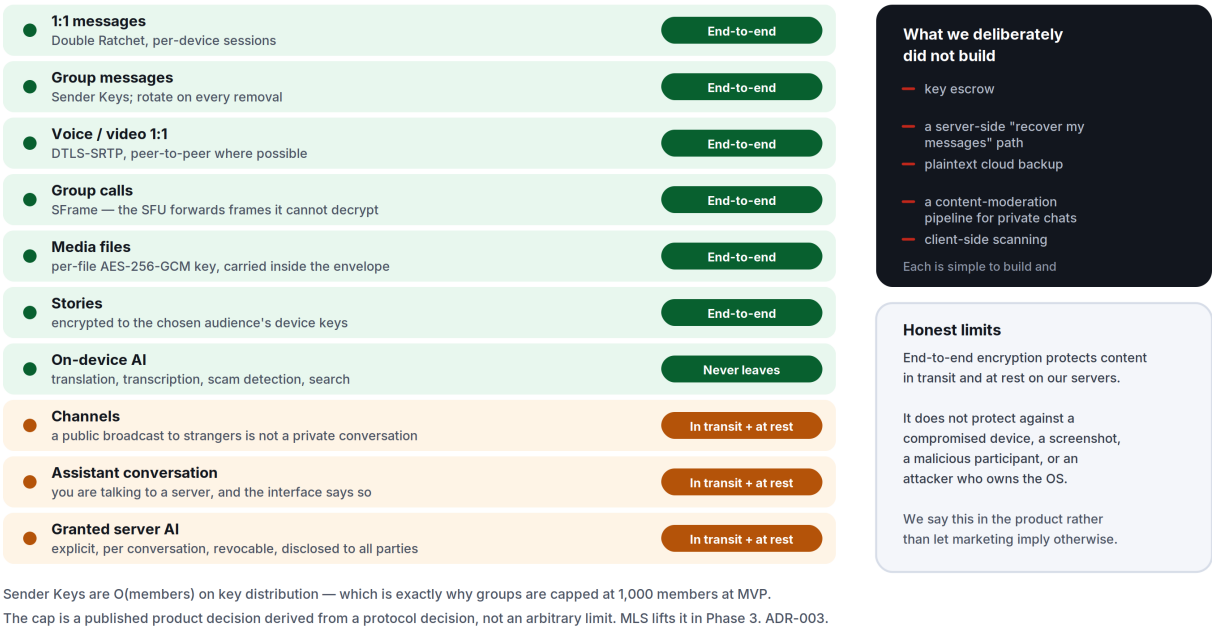


Figure 6.3 Every surface, and whether it is end-to-end encrypted — including the two that deliberately are not.

Concern	Approach
1:1 messages	Double Ratchet, per-device sessions, forward secrecy and post-compromise security
Group messages	**Sender Keys** — each sender has a per-group chain key distributed pairwise; sender keys rotate on every membership removal
Group scaling limit	Sender Keys are O(members) on key distribution. Practical ceiling ~1,000 members, which is exactly why Part 5 §5.5 caps groups there at MVP. **MLS (RFC 9420) is the Phase 3 migration** that lifts it — planned, scheduled, and not pretended to exist earlier.
Multi-device	Each device is a separate cryptographic identity with its own sessions. No key sharing between devices, no primary device (ADR-012).
Media	Per-file AES-256-GCM key, generated client-side, transmitted inside the encrypted message envelope. Storage holds ciphertext and cannot decrypt it.

Concern	Approach
Calls	SRTP with DTLS-SRTP for 1:1. Group calls use an SFU with **insertable-stream E2EE (SFrame)** so the SFU forwards frames it cannot decrypt (ADR-006).
Stories	Encrypted to the selected audience's device keys
Channels	**Not E2EE** — encrypted in transit (TLS 1.3) and at rest. Stated in the UI (Part 5 §5.6).
Assistant conversation	**Not E2EE** — it is a conversation with a server. Stated in the UI (Part 4 §4.1).
At rest on device	SQLCipher, key in platform keystore
Transport	TLS 1.3, certificate pinning with a backup pin and a documented rotation runbook
Post-quantum	Hybrid X25519 + ML-KEM-768 key agreement, Phase 3. Recorded as a roadmap item with a date, not a marketing claim.

Verification. Safety numbers per contact, QR verification, and a mandatory acknowledgement when a contact's keys change (Part 2 §2.4).

What we deliberately do not build: key escrow, a server-side "recover my messages" path, plaintext backup to any cloud, or a message-content moderation pipeline. Each is technically simple and would void the core promise.

> **Honest statement of limits.** E2EE protects content in transit and at rest on our servers. It does not protect against a compromised device, a screenshot, a malicious participant in a conversation, or an OS-level attacker. We say this plainly in the product rather than allowing the marketing implication that encryption makes users invulnerable.

6.8 Offline Sync and Caching

The sync model.

FIGURE — OFFLINE-FIRST SYNCHRONISATION

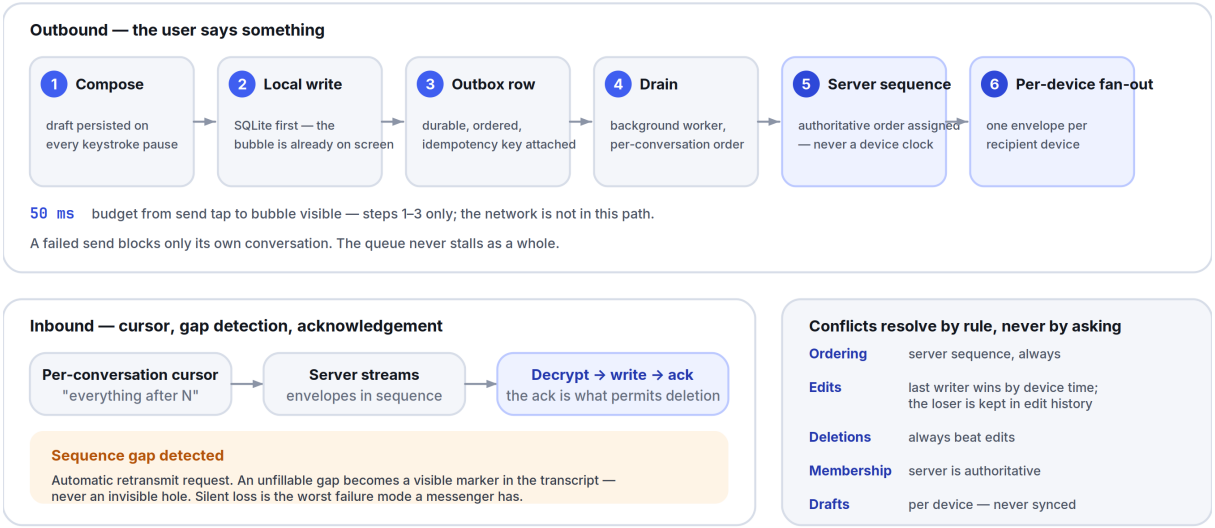


Figure 6.4 Local-first sync: the outbox guarantees ordering and delivery; gaps are surfaced rather than hidden.

1. The client writes to its **local SQLite store first** and renders from it. Always.
2. Mutations go into a durable **outbox** table with a monotonic local sequence, an idempotency key, and a retry policy.
3. A background worker drains the outbox in order, respecting per-conversation ordering. A failed send blocks only its own conversation, never the whole queue.
4. Inbound: the client holds a **per-conversation cursor**. On connect, it requests everything after the cursor. The server streams envelopes; the client decrypts, writes, advances the cursor, and acknowledges. Acknowledgement is what allows the server to delete.
5. **Gap detection**: sequence numbers are contiguous per conversation. A gap triggers an automatic retransmit request. An unfillable gap becomes a visible marker in the transcript (Part 5 §5.13) — never an invisible hole.

Conflict resolution is by rule, never by prompt (Part 3 §3.7): server sequence for ordering; last-writer-wins by device timestamp for edits, with losers retained in edit history; deletions always dominate edits; group membership is server-authoritative.

Caching.

Layer	Policy
Message store	Permanent on device until the user deletes it. Never evicted automatically.
Media	LRU with a user-configurable cap (default 4 GB); originals re-downloadable while server retention holds
Thumbnails	Permanent, small, generated on device

Layer	Policy
Avatars	Memory + disk LRU, content-hash-keyed, immutable
Search index	Rebuilt from the message store; disposable
CDN	Immutable media objects, `Cache-Control: immutable`, long TTL, keyed by content hash

> **A rule with teeth:** the app must fully render the conversation list and the last screen of any open conversation **with the network stack disabled**. This is asserted by an automated test on every release, not verified by hand.

6.9 Realtime Media (Calls)

- **Signalling** over the existing WebSocket — no separate signalling infrastructure.
- **1:1 calls attempt peer-to-peer first** (ICE with STUN), falling back to TURN relay when NAT traversal fails. P2P means lower latency and no media touching our servers.
- **Group calls use an SFU** (LiveKit, self-hosted) with insertable-stream E2EE, so the SFU routes frames it cannot decrypt (ADR-006). An MCU is rejected: server-side mixing means server-side plaintext.
- **Codecs:** Opus for audio (always, with in-band FEC and DTX), VP8 at MVP for video with AV1 evaluated in Phase 2 where hardware support exists.
- **The audio-priority rule** (Part 5 §5.4) is implemented as an explicit bandwidth allocator: audio gets its floor first, video receives the remainder, and simulcast layers are dropped from the top down.
- TURN servers are deployed in every region; media never crosses regions unnecessarily.

6.10 Scalability

Present target: 1 M registered users, 200 K concurrent connections, 50 K messages/second peak. That is a real target for the first two years and is achievable with the architecture above on a modest fleet. Everything beyond it is written as a *trigger*, not a plan we execute now.

Trigger	Action
>150 K concurrent sockets	Add gateway instances; shard connection routing by consistent hash on user ID

Trigger	Action
Postgres write CPU sustained >60 %	Read replicas for all non-authoritative reads; then partition `core` by user-ID range
Envelope writes >30 K/s **or** undelivered table >500 GB	Migrate envelopes to ScyllaDB behind the existing interface (ADR-004)
p95 client latency >400 ms in a region	Deploy a regional gateway + TURN + CDN edge; core stays central until data-residency requires otherwise
A regulatory data-residency obligation	Regional deployment with data pinned by account home region
Any single module's scaling profile diverges >5× from the rest	Extract that module from the monolith into its own deployable

What we are explicitly *not* doing at MVP: Kubernetes, service mesh, multi-region active-active, event sourcing, CQRS, a custom orchestration layer. Each is defensible at scale and each would cost us the first year (ADR-002).

Deployment at MVP: containers on a managed platform, one primary region, a warm standby in a second region, infrastructure as code (Terraform) from commit one. Migrating to Kubernetes later is a well-trodden path; migrating away from premature complexity is not.

6.11 Monitoring and Observability

Instrumentation: OpenTelemetry throughout — traces, metrics, logs — with trace context propagated from the client through the gateway to the datastore.

The four numbers on the wall, reviewed daily:

1. p50/p95/p99 send-to-delivered latency.
2. Message delivery success rate (the correctness metric from Part 5 §5.1).
3. Crash-free sessions and crash-free users.
4. Cold-start time at p95, segmented by device tier.

Stack: Prometheus + Grafana for metrics, Tempo/Jaeger for traces, Loki for logs, Sentry for client crashes and errors.

SLOs (Phase 1): message delivery 99.95 % monthly; API availability 99.9 %; call setup success 99 %. Error budgets are real: when a budget is exhausted, feature work stops until reliability work restores it. This is a written policy, not an aspiration.

> **Logging discipline — a privacy requirement, not a hygiene one:**

- **Message content is never logged.** Not at debug level, not behind a flag, not in a crash report. A lint rule and a CI check block any log call taking a message body.
- Logs carry IDs, never phone numbers, emails, or handles.
- Retention: 30 days for application logs, 7 days for connection logs, 90 days for security-relevant audit logs.
- Client crash reports are scrubbed on-device before upload, and the user can disable them.

Alerting distinguishes pages (user-visible breakage) from tickets (everything else). An alert that pages without an actionable runbook is deleted.

6.12 Testing

Layer	Coverage requirement	Tooling
Domain logic (Dart)	**90 % line coverage, enforced in CI**	`package:test`
Widgets	Every component in every state, both themes, LTR + RTL	`flutter_test`
Golden/visual	Every component; diffs require explicit approval	`golden_toolkit`
Integration (client)	Sign-up, send, receive, call, link device, offline→online	`integration_test` on real devices
Backend unit	85 % on `core`	Go `testing`
Backend integration	Full API against real Postgres/Redis in containers	`testcontainers`
Crypto conformance	**100 % of official libsignal test vectors, every commit**	Custom harness
Protocol compatibility	Client N against server N, N−1, N−2, N−3	Matrix job
Load	200 K concurrent connections, 50 K msg/s, weekly	k6 + custom WS harness
Chaos	Kill gateway, partition database, saturate the network — monthly in staging	Custom
Accessibility	Contrast, target size, missing semantic labels — every PR	Automated + manual VoiceOver/TalkBack gate
Performance	Cold start, frame rate, memory — every PR, on real hardware	`flutter driver` + a physical device lab

Non-negotiable test gates. A release is blocked by: any crypto conformance failure, any performance regression beyond budget (Part 8), any accessibility regression, any protocol-compatibility failure, or any drop in the offline-render assertion (§6.8).

The device lab. A physical rack of the Tier-A/B/C reference devices (Part 8 §8.7) running the performance and integration suites on every merge to main. Emulators do not tell the truth about jank, thermals, or memory pressure, and the cheap-Android experience is the one most likely to be quietly bad.

6.13 Data Retention and Minimisation

A published schedule, enforced by automated jobs, and audited quarterly:

Data	Retention
Undelivered message envelopes	Until delivered to all devices, or 30 days
Delivered message envelopes	Deleted immediately (server-side)
Media blobs	90 days after last access (free) / 365 days (Plus)
Connection metadata (IP, timestamps)	7 days
Push tokens	Until invalidated
Account record	Until deletion, then 30 days of tombstone, then purged
Assistant conversation	90 days by default, user-configurable down to 0
Security audit logs	90 days
Analytics events	13 months, aggregated and pseudonymous (Part 11 §11.5)

Deletion means deletion, including from backups within the backup rotation window (35 days), and this is verified by a quarterly audit that attempts to retrieve deleted records rather than assuming a policy was followed.

6.14 CI/CD

Pipeline (every pull request): format → static analysis → domain unit tests → widget + golden tests → backend unit + integration tests → crypto conformance → protobuf breaking-change check → accessibility checks → build all targets → performance smoke on a physical device.

On merge to main: full integration suite → device-lab performance suite → deploy backend to staging → nightly load test.

Release trains. A client release every two weeks; the backend deploys continuously behind feature flags. The two are decoupled by protocol versioning (§6.4) — the backend never requires a client release, and a client release never requires a coordinated backend deploy.

Client rollout: internal → 1 % → 10 % → 50 % → 100 %, each stage gated on crash-free rate and the four wall numbers (§6.11). Automatic halt on any regression. Every release is one tap from rollback, and rollback is *rehearsed*, not theorised.

Feature flags are server-controlled, default-off, and have a mandatory expiry date — a flag older than 90 days fails the build, because permanent flags are how a codebase becomes untestable.

Secrets never touch the repository; they live in a managed secret store with per-environment scoping and automatic rotation. Signing keys for both app stores are in an HSM with a documented, tested recovery procedure.

Supply chain: dependencies pinned by hash, an SBOM generated per release, automated CVE scanning that blocks on high severity, and reproducible builds for the client so that a published binary can be verified against its source. Reproducible builds matter enormously for an encrypted messenger: it is the only way for outsiders to check that the app they installed is the app we published.

6.15 Security Operations

- **Threat model published and maintained**, covering: malicious server, compromised device, malicious conversation participant, network adversary, hostile insider, and supply-chain attacker.
- **Third-party security audit before public launch**, and annually thereafter. The cryptographic implementation is audited separately by a specialist firm. Results are published.
- **Bug bounty** from public beta, with published severity tiers and response times.
- **Insider-risk controls:** production data access requires a ticket, is time-boxed, is logged immutably, and is reviewed weekly. No engineer has standing production database access.
- **Incident response:** documented severity levels, on-call rotation, a 72-hour user-notification commitment for any breach affecting user data, and a published post-mortem for anything user-visible.
- **Transparency report** twice yearly from Phase 2 (Part 4 §4.10).

PART 07

07

The Design System

Part 7 — Tokens, Components, Patterns

Governed by Part 2 (what things look like) and Part 3 (how they behave). This document is the buildable specification: the token contract, the component inventory, and the patterns that compose them.

> "A designer may not invent a value, and an engineer may not hard-code one."

IN THIS PART

> **The rule that makes a design system real:** a designer may not invent a value, and an engineer may not hard-code one. Every colour, space, radius, duration, and type style referenced in a screen must resolve to a token below. Anything else fails design review and CI lint.

7.1 Token Architecture

Three tiers. Screens consume tier 3 only; tier 3 references tier 2; tier 2 references tier 1. A screen that reaches past tier 3 is a bug.

FIGURE — DESIGN TOKEN HIERARCHY

Figure 7.1 Three tiers, one generated source. A screen that reaches past tier 3 is a bug.

```
Tier 1 — Primitive     orbit-500, void-100, space-4, radius-lg, dur-medium
                       Raw values. Theme-independent. Never used in a screen.

Tier 2 — Semantic      color.surface.primary, color.text.secondary, color.action.primary
                       Meaning, resolved per theme. This is where light/dark diverge.

Tier 3 — Component     bubble.outgoing.background, composer.height, button.primary.label
                       Component-scoped. What screens actually reference.
```

Distribution. Tokens are authored once in JSON (W3C Design Tokens format) and generated into: Dart (`SpaceTokens`), the Figma variable set, and a CSS custom-property sheet for the web client. **Generation is one-directional and automated in CI** — hand-editing a generated file is a build failure. This is what keeps design and code from drifting, which is the failure mode of every design system that dies.

Semantic token set (abbreviated; the full set is in the token repository):

Token	Light	Dark
color.bg.app	void-25	void-950
color.surface.primary	void-0	void-900
color.surface.elevated	void-0	void-850
color.text.primary	void-900	void-50
color.text.secondary	void-500	void-400
color.text.tertiary	void-400	void-500
color.text.onAction	void-0	void-0
color.action.primary	orbit-500	orbit-500
color.action.primaryText	orbit-600	orbit-300
color.border.subtle	void-200	void-700
color.ai.text	aurora-700	aurora-300
color.ai.surface	aurora-050	aurora-950
color.status.danger	danger	danger-dark
color.status.warning	warning-text	warning-dark
color.status.success	success-text	success-dark

7.2 Component Inventory

Every component is specified as: **props · states · variants · accessibility contract · when not to use it.** The last line is the one that keeps a system small.

Foundational

Component	Variants	Key states	Never use for
Button	primary, secondary, tertiary, destructive, icon	default, hover, pressed, disabled, loading, focus	Navigation — that's a link or a row
Avatar	24 / 32 / 40 / 48 / 64 / 96 px	image, initials, group-stack, online dot	Decorative imagery
Badge	count, dot	—	Anything that isn't addressed to the user (Part 3 §3.9)
Chip	filter, input, suggestion	selected, unselected, disabled	Primary actions

Component	Variants	Key states	Never use for
`Divider`	inset, full	—	Creating hierarchy that spacing should create
`Icon`	16 / 20 / 24 / 32 px	—	Conveying meaning without a label
`Spinner`	16 / 24 / 32 px	—	List loading — use `Skeleton`
`Skeleton`	text, avatar, media, row	shimmer, static (reduced motion)	Anything under 300 ms

Content

Component	Notes
`MessageBubble`	The most important component in the product. Variants: incoming, outgoing, system, deleted. Slots: reply-quote, content, attachments, reactions, meta (time + delivery glyph). Run-grouping and corner geometry per Part 2 §2.10.
`MediaTile`	Image, video, GIF. Always renders at known intrinsic dimensions so nothing shifts (Part 3 §3.4).
`VoiceNote`	Waveform, play/pause, speed, duration, transcript toggle.
`FileTile`	Type glyph, name, size, progress, download/open.
`LinkPreview`	Sender-generated (Part 5 §5.1). Title, description, image, domain. Collapsible.
`AiAnnotation`	The only component permitted to use `color.ai.*`. Slots: label, content, source-link, feedback.
`SystemMessage`	Centred, `footnote`, `color.text.secondary`. Group events, encryption notices, disappearing-message changes.

Navigation & containers

Component	Notes
`AppBar`	56 px. Title + back + max two actions. Translucent variant per Part 2 §2.9.
`BottomNav`	Exactly 4 items, labels always shown.
`ListRow`	56 / 72 px. Leading, title, subtitle, trailing, swipe actions. Entire row is one touch target.

Component	Notes
`Sheet`	Bottom sheet. Drag handle, snap points, dismissible. **Preferred over `Dialog` for anything non-blocking** (Part 3 §3.12).
`Dialog`	Blocking decisions only. Max two actions.
`Menu`	Long-press and overflow. Destructive items last, separated, in `danger`.
`Composer`	Text field, attach, mic, send. Fixed position, grows to 5 lines (Part 2 §2.14).
`Banner`	Persistent, dismissible, top of content. Offline, security warnings.
`Snackbar`	4 s, one action, non-blocking. Undo lives here (Part 3 §3.2).

Input

`TextField` · `SearchField` · `Switch` · `Checkbox` · `Radio` · `Slider` · `Picker` · `OtpField` · `SafetyNumberDisplay` (monospaced, chunked, copyable, QR-linked).

Components we deliberately do not have, recorded so nobody adds them: carousel, accordion, tooltip on touch surfaces, breadcrumb, stepper, tab-within-tab, floating action button on the conversation screen (the composer *is* the action), and any "onboarding tour" component.

7.3 Iconography

Specification in Part 1 §1.8. Delivery:

- One source SVG set on a 24 × 24 grid, 1.5 px stroke, exported to a Flutter icon font and a web sprite by the same pipeline that builds tokens.
- Sizes: 16 (inline), 20 (dense), 24 (default), 32 (prominent).
- **Every icon ships with a required `semanticLabel`.** The Flutter wrapper makes the label a required parameter, so an unlabelled icon is a *compile error* rather than an audit finding. This is the cheapest accessibility enforcement in the entire system.
- Icons inherit `color.text.*` and never carry colour of their own, except status icons.

7.4 Buttons (behavioural contract)

Property	Rule
Count	**One primary button per screen.** Two primaries means the screen has two purposes.
Loading	The button keeps its width, swaps its label for a spinner, and remains disabled. Never collapses or jumps.

Property	Rule
Disabled	Only when the action is genuinely impossible. Prefer an enabled button that explains what is missing on tap — a disabled button with no explanation is a dead end.
Destructive	Always paired with a confirmation or an undo, never both.
Touch target	≥44 × 44 px regardless of visual size, verified by automated test.
Label	A verb. “Send,” “Link device,” “Delete chat.” Never “OK,” “Submit,” or “Yes.”

7.5 Cards and Lists

Cards are for grouped, self-contained content that can be acted on as a unit. If the content is a list of similar things, it is a list, not a stack of cards. Card-per-row is the most common way a clean list becomes a cluttered screen.

Lists are the product’s primary structure. Rules: the whole row is the touch target; swipe actions match Part 3 §3.2 exactly, product-wide; dividers are inset to the text baseline; a list of one item is not a list, it is a row; and virtualisation is mandatory above 50 items — no exceptions, because the conversation list and the transcript are both unbounded.

7.6 Dialogs, Sheets, and Menus

Choosing between them — this decision is made wrongly more often than any other in mobile design:

Use	When
Snackbar	Something happened; the user may want to undo it
Sheet	The user needs to choose from options or fill something in, and can back out
Dialog	The user must decide before anything else can proceed, and the decision is hard to reverse
Full screen	The task has multiple steps or needs the whole viewport

Dialogs have at most two actions, with the safe one on the right in LTR (per platform convention) and the destructive one clearly marked. Sheets support drag-to-dismiss, snap points, and a visible handle. Menus place destructive items last, after a separator, in `danger`.

Nothing stacks. A sheet may not open a dialog which may not open another sheet. If a flow needs that, it is a full screen.

7.7 Motion Tokens

Token	Duration	Curve (cubic-bézier)	Use
`dur-instant`	80 ms	`standard (0.2, 0.0, 0.0, 1.0)`	Press feedback, ripples
`dur-fast`	120 ms	`standard-decelerate (0.0, 0.0, 0.0, 1.0)`	Small fades, tooltips, chips
`dur-medium`	200 ms	`emphasised-accelerate (0.3, 0.0, 0.8, 0.15)`	Sheet out, dismiss
`dur-default`	240 ms	`emphasised-decelerate (0.05, 0.7, 0.1, 1.0)`	Sheet in, menu in
`dur-screen`	280 ms	`emphasised (0.2, 0.0, 0.0, 1.0)`	Screen transitions
`dur-slow`	400 ms	`standard`	Rare: full-screen media transitions

Rules. Nothing exceeds 400 ms. Nothing springs or overshoots. Every animation is interruptible at any frame. Under `prefers-reduced-motion`, position and scale transitions become ≤100 ms cross-fades, and all looping motion stops (Part 1 §1.9).

7.8 Gestures

The complete, fixed set is in Part 3 §3.2. System-level requirements:

- **Gesture conflicts are resolved in favour of the platform.** The iOS edge-back and Android back gestures always win; our swipe zones start beyond the system's.
- **Every gesture has a discoverable, non-gesture equivalent.**
- **Haptics are informational**: a light tick on long-press engage, a medium tick on send, a distinct pattern on error. Nothing celebratory. All haptics respect the system setting.
- **Drag operations show a drop target before release** — never a mystery drop.

7.9 Transitions and Shared Elements

From → To	Transition
Conversation list → conversation	Push with the avatar as a shared element
Conversation → profile	Push; avatar expands to header
Message media → full-screen viewer	Shared-element expansion from the tile's exact bounds

From → To	Transition
Composer → attachment tray	Tray rises; composer does not move (Part 3 §3.2)
Any screen → sheet	Sheet rises from the triggering control; scrim fades to 40 %
Tab → tab	Cross-fade only, 120 ms. No horizontal slide — tabs are peers, not a sequence.

7.10 Design Patterns

Repeatable solutions, defined once so they are not re-solved differently on each screen.

Progressive disclosure. Show the common case; put the rest behind one clearly-labelled control. Never behind two.

Optimistic action. Update the UI immediately, reconcile in the background, surface only real failures (Part 3 §3.2).

Undo over confirm. For anything recoverable, act and offer undo. Reserve confirmation for the irreversible.

Inline over navigate. Solve it where the user is. Translation appears under the message; it does not open a screen.

Explain in place. When something is unavailable, say why at the point of unavailability — not in a help centre.

One question per screen. Onboarding and settings flows ask one thing at a time.

Honest state. Never show a state that isn't true. No fake progress, no optimistic "delivered," no placeholder avatars that look like real people.

The AI seam. Anything a model produced sits in an `AiAnnotation` with Aurora colour, a label, and a feedback control. There is exactly one way for AI output to appear in this product, so a user learns it once.

7.11 Contribution Rules

Adding to the system:

1. **Three uses before a component.** A pattern used twice is a copy; used three times it is a component. Premature components are as costly as duplicated code.
2. **Every component ships with:** the Flutter implementation, a Figma component with matched variants, golden tests for every state in both themes and both directions, an accessibility contract, and a "when not to use it" note.
3. **Every component is reviewed by design and engineering together.** A component approved by only one of them will be wrong in the other's dimension.

4. **Deprecation is scheduled, not implied.** A deprecated component gets a replacement, a migration note, and a removal date. Two components that do the same thing is how a design system stops being a system.
5. **A change to a tier-1 or tier-2 token requires CDO approval**, because it changes every screen at once.

PART 08

08

Performance Standards

Part 8 — The Numbers We Defend

Governed by Part 0 §0.6 clause 8: no feature ships that regresses these targets. Performance is a release gate, not a follow-up ticket.

> "A budget that is not measured in CI does not exist."

IN THIS PART

> **How to read this.** Every number below is a **budget**, measured on the reference hardware in §8.7, in production, at the stated percentile. A budget that is not measured in CI does not exist. Where a target differs by device tier, all three are given — the Tier-C number is the one that matters most, because it describes the experience of the users most likely to have no alternative.

8.1 Why Speed Is the First Feature

A messaging app is opened 50–150 times a day. A 400 ms cold start costs a heavy user roughly a minute a day, and — more importantly — it changes what the app *is*: something you decide to open rather than something you glance at. Every competitor in this category has, over a decade, traded launch time for features. That accumulated debt is the opening we are attacking.

This is why speed is Identity Pillar 1 (Part 0 §0.7) and why Part 0.9 rule 4 resolves performance-versus-beauty in performance's favour.

8.2 Cold Launch

Measurement	Tier A	Tier B	Tier C
Process start → first frame	**< 350 ms**	< 600 ms	< 1,100 ms
First frame → conversation list populated	**< 100 ms**	< 180 ms	< 350 ms
Total to interactive (p95)	**< 450 ms**	**< 800 ms**	**< 1,500 ms**
Warm launch to interactive	< 120 ms	< 200 ms	< 400 ms

How it is achieved, specifically:

- The conversation list renders from SQLite before any network call is made. Networking initialises *after* the first frame.
- The theme is read synchronously from platform preferences before the first frame, so there is never a flash of the wrong theme.
- Crypto (libsignal FFI) initialises lazily, on first use, not at launch.
- Deferred components use Flutter's deferred loading; nothing is constructed at startup that the first screen does not need.
- Shader warm-up runs on first launch only, from a pre-recorded SkSL bundle.

Enforcement. Cold start is measured on physical devices in CI on every pull request. A regression above budget fails the build. There is no "we'll fix it next sprint" path.

8.3 Frame Rate

Surface	Target
All scrolling and animation	**60 fps floor; 120 fps on capable displays**
Dropped frames while scrolling a 10,000-message transcript	**< 0.5 %**
Jank (frames > 16.6 ms) during any transition	**< 1 %**
Frame build time (p99)	**< 8 ms** — half the budget, leaving room for raster
Raster time (p99)	< 8 ms

The transcript is the benchmark. If the message list holds frame rate on Tier-C hardware with a large history, mixed media, and an active network, everything else in the product will too. Techniques are specified in Part 6 §6.2; the ones that matter most: cached message heights, `RepaintBoundary` per bubble, no save layers in scrolling content, and image decode at display resolution.

8.4 Latency

Interaction	Target (p95)
Touch → first visual response	**< 100 ms**
Keystroke → character rendered	< 16 ms (one frame)
Send tap → bubble visible	**< 50 ms** (local write, optimistic)
Send → delivered to a recipient device	**< 250 ms p50, < 700 ms p95**
Search keystroke → local results	**< 50 ms**
Open a conversation → last screen rendered	< 120 ms
Tap notification → conversation visible	< 400 ms
Call initiate → callee ringing	**< 1.2 s p75**
Media thumbnail → visible	< 100 ms (cached), < 400 ms (network)

The 100 ms rule (Part 3 §3.3) is the one to internalise: below roughly 100 ms an interface feels like a direct extension of the hand; above it, like a system being asked to do something.

8.5 Memory

State	Tier A	Tier B	Tier C
Idle (conversation list)	< 120 MB	< 100 MB	**< 80 MB**
Active conversation with media	< 220 MB	< 180 MB	**< 140 MB**
Video call	< 320 MB	< 280 MB	< 220 MB
Peak, any operation	< 400 MB	< 350 MB	**< 260 MB**

Figure 8.1 The numbers defended on every pull request, on real hardware, at every tier.

Why Tier C is stricter, not looser: low-memory Android devices kill background apps aggressively. An app with a large footprint is evicted between glances, which turns every warm launch into a cold one. Memory discipline on cheap hardware *is* launch-speed discipline.

Rules. Image caches are bounded in bytes, not entries. Decoded images are released on scroll-out. No unbounded collection anywhere in the client — every cache has an eviction policy asserted by a test. Memory is profiled on Tier-C hardware weekly, and leaks are release blockers.

8.6 Binary Size, Battery, and Network

Binary size. Android app bundle **< 30 MB** initial download; iOS **< 60 MB**; per-architecture split, deferred components for calls and AI models. Language packs and on-device models are downloaded on demand, never bundled — a user who needs two languages should not carry sixty.

Battery (measured over a standardised 1-hour scenario on the Tier-B reference device):

Scenario	Budget
Idle, connected, receiving occasional messages	**< 1 % per hour**
Active messaging	< 4 % per hour
Voice call	< 6 % per hour
Video call	**< 12 % per hour**

Achieved by: a single multiplexed connection (never one per feature), push-driven wake-ups with exponential backoff, batched writes, no polling anywhere, hardware codecs for all media, and no background work when the app is not foregrounded except the outbox drain.

Network.

Item	Budget
Text message overhead (envelope + protocol, excluding content)	**< 400 bytes**
Idle keepalive	< 1 KB per 5 minutes
Typing indicators	Coalesced, max 1 per 3 s per conversation
Presence	Push-driven, never polled
Daily background data, idle account	**< 1 MB**

Protobuf over JSON, gzip on anything above 1 KB, and delta sync rather than full-state refresh. A user on a metered 500 MB monthly plan must be able to use SpaceTalk normally, and this is tested against a simulated metered connection.

8.7 Reference Hardware and Network Profiles

Device tiers. Every budget is stated per tier; every tier is a physical device in the CI lab (Part 6 §6.12).

Tier	Definition	Reference devices	Share of target market
A	Flagship, ≤2 years old	Current iPhone Pro, current Pixel Pro	~15 %
B	Mainstream, 2–4 years old	iPhone from ~4 years ago, mid-range Android with 6 GB RAM	~50 %

Tier	Definition	Reference devices	Share of target market
C	Entry-level, low RAM	Android Go-class, 3–4 GB RAM, eMMC storage	~35 %

> **Tier C is the design target, not the fallback.** Any screen that only feels good on Tier A is unfinished.

Network profiles, applied in automated testing:

Profile	Bandwidth	RTT	Loss
Good	20 Mbps	30 ms	0 %
Typical mobile	4 Mbps	90 ms	0.5 %
Poor	500 kbps	300 ms	3 %
Very poor	100 kbps	800 ms	8 %
Offline	—	—	100 %

Messaging must remain fully usable on *Poor*. Voice calls must remain intelligible on *Very poor*. Every release runs the full integration suite across all five profiles.

8.8 Media Optimisation

Images. Uploaded as-is when the user chooses original quality (Part 5 §5.9). Otherwise: longest edge 2,048 px, JPEG q82 or AVIF where both ends support it. Thumbnails are generated on-device at 400 px and shipped inside the message envelope, so a thumbnail is visible before any media download begins. Progressive decode; blurhash placeholder at the exact final dimensions so nothing shifts.

Video. H.264 baseline for compatibility, HEVC where hardware supports it, capped at 1080p30 for sharing. Always hardware-encoded — software encoding on Tier C is a thermal and battery failure. A poster frame and duration ship in the envelope.

Audio. Opus 24 kbps mono for voice notes — indistinguishable from higher rates for speech and roughly a third of the bytes. Waveform data (64 samples) is precomputed on-device and sent in the envelope so the waveform renders before the audio downloads.

Universal rule: the recipient sees *something* correct — thumbnail, blurhash, waveform, poster frame — before any byte of the media itself arrives. This is what makes the app feel fast on a bad connection, and it costs a few hundred bytes per message.

8.9 Accessibility Score

Metric	Target
Automated accessibility checks	**100 % pass, zero suppressions**
Contrast (all text)	100 % of pairs meet Part 2 §2.15
Touch targets ≥44 px	100 %, asserted by test
Screen-reader labelled elements	100 % (compile-enforced for icons, Part 7 §7.3)
Full task completion via screen reader	Every MVP feature, verified manually per release
Dynamic type to 200 %	No clipping or overlap on any screen
Keyboard operation (web/desktop)	Every action reachable, focus always visible

8.10 Stability

Metric	Target	Gate
Crash-free sessions	**> 99.9 %**	Rollout halts below 99.8 %
Crash-free users	**> 99.5 %**	Rollout halts below 99.3 %
ANR rate (Android)	< 0.2 %	Play Store vitals threshold
Message delivery success	**100 %** (correctness, not a percentage to optimise)	Any loss is Sev-1
Data loss incidents	**0**	Any occurrence halts all releases
Backend availability	> 99.9 % monthly	Error budget policy (Part 6 §6.11)

8.11 The Performance Budget Process

1. **Every feature has a budget before it has a design.** "This must not add more than 8 MB of memory, 30 ms to cold start, or 200 KB to the binary." A feature that cannot state its budget is not specified yet.
2. **Budgets are measured in CI on physical hardware**, per pull request, per tier.
3. **A regression fails the build.** Not a warning, not a dashboard, not a ticket.
4. **Exceeding a budget requires an explicit trade**: something else gives up an equivalent amount, and the trade is recorded in the pull request. Budgets do not inflate quietly, which is the only way they survive contact with a roadmap.

5. **Production is measured continuously**, segmented by device tier, network profile, and region. Lab numbers describe the machine; production numbers describe the user.
6. **The four wall numbers** (Part 6 §6.11) are reviewed daily by the whole team, and are visible to everyone in the company.
7. **A quarterly performance week** where no features ship and the entire engineering team works on the numbers. Not a reward for good behaviour — a standing calendar item, because entropy is constant and every product in this category lost this fight by degrees.

PART 09

09

The Roadmap

Part 9 — Five Phases

Governed by Part 0. Every phase states its mission, what it builds, what it explicitly does not build, its team shape, and the gate that must be passed before the next phase begins. Phases are gated by evidence, not by calendar.

> "A phase does not begin because the previous one ran out of time."

IN THIS PART

> **The rule that makes this roadmap real:** a phase does not begin because the previous one ran out of time. It begins because the previous one met its gate. If a gate is missed, the response is to fix the product, not to move on to the next phase and hope.

Phase 1 MVP (Months 0–12)

Mission. Prove that a communication app can be *dramatically* faster and calmer than the incumbents, with intelligence that actually helps, and that people will move to it for those reasons alone.

FIGURE — FIVE PHASES, GATED BY EVIDENCE RATHER THAN BY CALENDAR

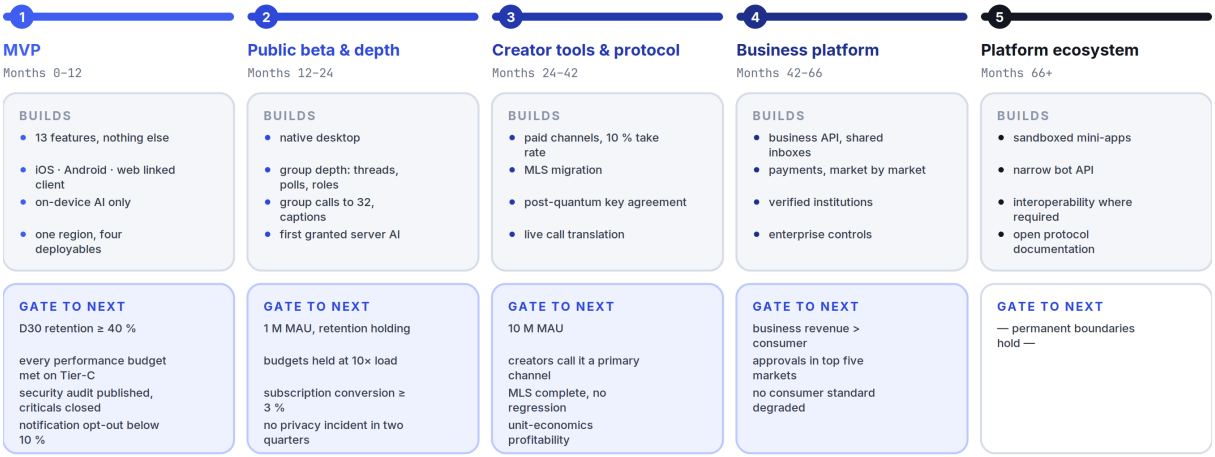


Figure 9.1 Each phase begins because the previous one met its gate — not because it ran out of time.

Build. Exactly the thirteen features in Part 5 — secure messaging, voice notes, voice calls, video calls, groups (≤1,000), channels, stories, AI assistant, file sharing, search, profiles, notifications, multi-device (≤4).

Platforms: iOS, Android, and the Flutter web build as the linked-device client (ADR-012).

Infrastructure: one region, warm standby, the four Go deployables (ADR-002), Postgres + Redis + object storage.

AI at MVP: **on-device only, plus the assistant conversation.** Reply suggestions, tap-to-translate, voice transcription, semantic search, scam detection — all local. Server-side AI features exist only as the assistant conversation, clearly labelled.

Explicitly NOT in Phase 1:

- Native desktop clients — the web client covers the need (ADR-012).
- Call summaries, live captions, live translation — quality is not there, and shipping them badly damages the AI promise more than omitting them.
- Payments, mini-apps, commerce, marketplace, bots, an API — every one of these is a platform, and a platform on top of an unproven product is a way to fail at two things.
- Groups above 1,000 — a protocol limit we are honest about (ADR-003).
- Threads, polls, roles, permissions, communities — Phase 2, once we know how groups actually get used here.
- Reels, feeds, discovery, recommendation — rejected permanently (Part 10).
- Address-book upload (ADR-010).
- Multi-region, Kubernetes, microservices (Part 6 §6.10).

Team shape. ~14 people: 4 client engineers, 4 backend, 1 security/crypto, 1 ML, 2 design, 1 product, 1 ops.

Gate to Phase 2 — all four must be true:

1. **D30 retention ≥ 40 %** for users who sent a message in their first week.
2. **Every number in Part 8 met on Tier-C hardware**, in production, at p95.
3. **A completed third-party security audit** of the cryptographic implementation, with all high and critical findings resolved, published.
4. **Qualitative evidence that the calm promise holds** — notification opt-out below 10 %, and users describing the product as fast and quiet in unprompted research.

Not a gate: signup count, DAU, or time-in-app.

Phase 1 Execution Sequence

The build order matters, because the dependencies are real and several of them are one-way doors. Quarters are indicative; the ordering is not.

Q1 — Foundations that cannot be retrofitted. Protobuf schemas and the protocol versioning scheme (Part 6 §6.4). The libsignal FFI binding with its conformance harness in CI from the first commit — if this integration is going to fail, we need to know in month one, not month eight. The token pipeline (Part 7 §7.1) and the component library skeleton. Local database schema and migration framework, with forward-migration tests against synthetic historical databases. The gateway and `core` skeletons with authentication. **Physical device lab operational.** Pre-MVP research from Part 13 §13.2 runs in parallel throughout.

Exit: two devices exchange an encrypted message end to end, over the real protocol, with conformance tests green.

Q2 — The core loop, at quality. Messaging (Part 5 §5.1) complete: transcript, composer, delivery states, reply, react, edit, delete, disappearing. Offline sync and the outbox (Part 6 §6.8). Push with content-free payloads (ADR-007). Groups with Sender Keys. Onboarding

(J1) and profiles. Performance budgets enforced in CI from the first transcript commit — retrofitting performance into a shipped list is the mistake this ordering exists to prevent.

Exit: J1, J2, and J3 complete on Tier-C hardware within budget. Internal dogfooding begins and does not stop.

Q3 — Everything real-time, plus the differentiator. Voice and video calls (1:1 first, then small groups). Voice notes. File sharing. Multi-device linking (J6). Then the AI layer: on-device translation (J4), transcription, reply suggestions, semantic search, and scam detection (J7). AI comes after the core loop deliberately — it is the differentiator, but it is worthless bolted onto a messenger that is not yet excellent.

Exit: all thirteen features functionally complete. Third-party security audit begins.

Q4 — Hardening, not features. No new features. Performance work against every budget in Part 8. Accessibility passes with real screen-reader users. Full localisation into the twelve launch languages with native-speaker review. Load testing to 10× projected launch traffic. Chaos testing. Security-audit findings resolved and the audit published. Staged rollout: internal → closed beta → open beta → general availability.

Exit: the four Phase 1 gate conditions below.

Standing rules for Phase 1. Dogfood from Q2 — the team uses SpaceTalk as its primary messenger, which is the only reliable way to notice the small daily failures. No feature merges without its budget met (Part 8 §8.11). No quarter adds a fourteenth feature; the MVP boundary is Part 0 §0.8 and it is not negotiated during execution.

Phase 2 Public Beta and Depth (Months 12–24)

Mission. Take the proven core to a wider audience, add the depth that daily users are actually asking for, and prove the infrastructure holds at 10× the load.

Build.

- **Native desktop** (macOS, Windows, Linux) from the Flutter codebase; unlimited history sync to linked devices.
- **Groups get depth:** threads, polls, admin roles and permissions, group descriptions, member-level muting, and the moderation tooling that group owners have by then told us they need.
- **Messaging depth:** scheduled send, pinned messages, formatted text, per-conversation themes, custom notification sounds, quiet hours.
- **Calls:** group voice to 32, group video to 32, screen sharing, on-device background blur, **live captions**, and call summaries with per-participant consent (Part 4 §4.5).
- **Channels:** scheduled posts, drafts, multi-admin, richer analytics, paid subscriptions.
- **AI:** the first server-assisted features under the explicit-grant model (ADR-005) — unread summaries, higher-quality translation for language pairs where on-device quality is insufficient. **Passkeys** for recovery.

- **Business, first version:** verified business profiles, a customer-conversation inbox, and away messages. No CRM, no automation, no payments — just "a business can be talked to properly."
- **Infrastructure:** a second region for latency, read replicas, the first module extraction if §6.10 triggers fire. Bug bounty opens. First transparency report.

Explicitly NOT in Phase 2: payments, mini-apps, marketplace, an open bot API, groups above 1,000, live translation.

Team shape. ~40 people. Trust & safety becomes a real function, not a rotation.

Gate to Phase 3:

1. **1 M monthly active users**, with D30 retention holding ≥ 40 % at scale.
2. **Performance budgets held at 10× load** — the numbers must not have drifted.
3. **Subscription conversion ≥ 3 %** with positive contribution margin per paying user (Part 11 §11.4).
4. **Zero unresolved critical security findings**, and no privacy incident in the preceding two quarters.

Phase 3 Creator Tools and Protocol Maturity (Months 24–42)

Mission. Let people who broadcast to an audience make a living inside SpaceTalk, and finish the protocol work that the first two phases deliberately deferred.

Build.

- **Creator monetisation:** paid channel subscriptions, one-off supporter payments, and a **10 % platform take rate** — deliberately below the market's 30 %, because we do not need to fund an advertising business (ADR-009 has a revenue upside as well as a cost).
- **Live audio broadcast** to channel subscribers. Not a social audio product — a channel format.
- **Creator analytics** that report reach and engagement without profiling subscribers, because we do not collect the data that would allow richer reporting, and we say so.
- **Protocol maturity: MLS migration** (ADR-003), lifting the group ceiling and improving the multi-device story; **hybrid post-quantum key agreement** (X25519 + ML-KEM-768); federated on-device search across a user's own linked devices.
- **AI:** live call translation (the flagship Phase 3 capability), cross-device assistant memory.
- **Communities:** groups of groups, events, shared membership — the WeChat/Discord shape, arrived at from the messaging side rather than the server side.
- **Infrastructure:** multi-region with data residency, ScyllaDB migration if triggered, regional TURN and SFU fleets.

Explicitly NOT in Phase 3: general-purpose payments, an app platform, commerce, dating, gaming.

Gate to Phase 4: 10 M MAU · creator earnings meaningful enough that creators describe SpaceTalk as a primary channel · MLS migration complete with no regression in delivery or latency · profitability on a unit-economics basis.

Phase 4 Business Platform (Months 42–66)

Mission. Become the channel through which businesses and institutions talk to people — the highest-margin, most defensible revenue in messaging, and the one that does not require becoming an advertising company.

Build.

- **Business API** (the public HTTP/JSON surface deferred since Part 6 §6.4) for customer messaging, notifications, and support, priced per conversation.
- **Business tooling:** shared team inboxes, assignment and routing, templates, quick replies, business hours, and conversation-level analytics.
- **AI for businesses:** draft assistance, auto-summarised support threads, and language coverage — all under the same consent rules as consumers, with no exception for commercial context.
- **Payments, carefully and regionally:** in-conversation payment requests and receipts, launched market by market with real licensing and real compliance, never as a global switch-flip.
- **Verified institutional accounts** for schools, health services, and government — the use cases where guaranteed delivery to subscribers (Part 5 §5.6) is worth the most.
- **Enterprise-grade controls:** audit logs, retention policies, SSO for business accounts, regional data residency.

Explicitly NOT in Phase 4: advertising (permanently), a social feed (permanently), a general app store, cryptocurrency of any kind.

Gate to Phase 5: business revenue exceeding consumer subscription revenue · regulatory approvals held in the top five markets · no degradation of any consumer-facing performance or privacy standard as a result of business features. That last gate is the important one — the failure mode of every messaging platform that added a business layer is that the consumer product got worse.

Phase 5 Platform Ecosystem (Months 66+)

Mission. Let third parties build inside SpaceTalk — under rules strict enough that the product does not become the thing it was built to replace.

Build.

- **Mini-apps**, sandboxed, with a permission model in which an app can see only what the user explicitly hands it, and **never** the conversation it is invoked from unless the user passes specific content to it.
- **A bot API** for legitimate automation, with strict, published rate limits and no ability to initiate contact with a user who has not opted in. The absence of unsolicited bot contact is a feature, and it is the reason to keep the API narrow.
- **Interoperability**, where regulation requires it (EU Digital Markets Act) — implemented honestly, with clear labelling of which conversations cross to another provider and therefore leave our encryption guarantees. Users must never be misled about where a promise stops applying.
- **Open protocol documentation** and reproducible clients, so the security claims are independently verifiable.
- **Institutional deployments** — education, healthcare, government — with the compliance posture each requires.

Permanent boundaries at every phase. These are the things that do not arrive at Phase 6:

- No advertising.
- No algorithmic feed of strangers.
- No content moderation of private conversations.
- No mini-app that can read a conversation.
- No feature whose success metric is time-in-app.

Cross-Phase: What Never Changes

Standard	Phase 1	Phase 5
Cold start (Tier C, p95)	< 1,500 ms	< 1,500 ms
E2EE default for personal messages	Yes	Yes
AI without explicit grant on private content	Never	Never
Notifications from non-humans	Zero	Zero
Primary navigation destinations	4	4
Advertising	None	None

The infrastructure required to hold these standards changes enormously between Phase 1 and Phase 5. The standards themselves do not change at all. That is what makes them standards.

PART 10

10

Scope Governance

Part 10 — The Full Ambition, Triaged

Governed by Part 0 §0.8 and §0.9. This is the permanent register of every idea proposed for SpaceTalk. Nothing is silently dropped. Each entry is either scheduled to a phase or rejected with a stated reason.

> "Some of the requested items are not features. They are companies."

IN THIS PART

10.1 Why This Document Exists

The founding brief for this project asked for every feature of WhatsApp, Telegram, Facebook, Instagram, Discord, Signal, and WeChat, plus payments, healthcare, education, commerce, government services, a creator studio, and an AI companion — all at once.

FIGURE — HOW THE FULL ECOSYSTEM BECAME THIRTEEN FEATURES

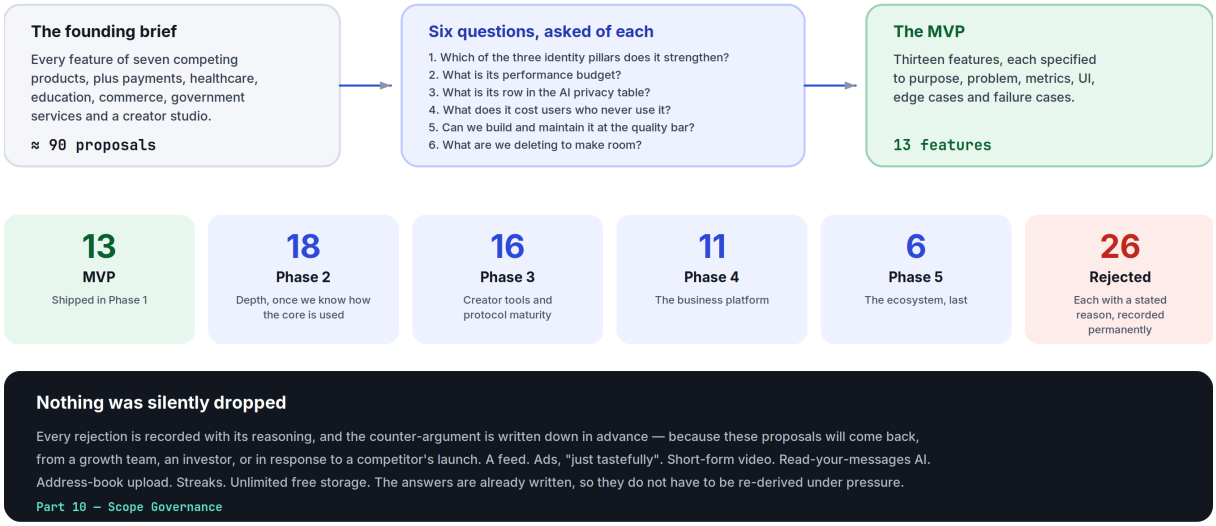


Figure 10.1 The triage: ~90 proposals in, 13 features out, 26 permanent rejections — each with its reason on the record.

That brief describes an ecosystem worth roughly a decade of work by thousands of engineers, and it contains genuinely good ideas. It also contains, in its final paragraph, its own correction: *design the full ecosystem, then identify a focused MVP, validate it, and expand in stages.* This document is that correction made operational.

> **The honest engineering assessment**, stated once so it does not need repeating:

- **WeChat's superapp was not built as a superapp.** It shipped as a messenger in 2011 and did not add mini-programs until 2017 — six years and hundreds of millions of users later. The platform was built on the distribution the messenger earned, not the other way round.
- **Every product that launched as an ecosystem failed as one.** Users adopt one thing at a time. A product that is thirty things is not adopted at all; it is browsed once.
- **The binding constraint is not ambition, it is quality per feature.** A team that builds thirteen features builds them well. The same team building sixty builds sixty poor ones, and in a communication product a poor feature is not neutral — it makes the app slower, larger, and harder to learn for every user, including the ones who never touch it.
- **Some of the requested items are not features, they are companies.** "Payments" is a licensing, compliance, fraud, and treasury organisation. "Healthcare" is a regulated business under HIPAA, GDPR Article 9, and dozens of national regimes. Calling them features on a roadmap is the clearest sign a plan is not real.

So: the full ecosystem is designed here. The MVP is thirteen features. The path between them is Part 9.

10.2 How to Read the Register

Verdict	Meaning
MVP	In Part 5
P2 / P3 / P4 / P5	Scheduled to that phase in Part 9
Rejected	Will not be built. Reason stated. Reversing a rejection requires a Part 0.10 amendment.

10.3 WhatsApp-Class Features

Feature	Verdict	Reasoning
Personal messaging, voice/video calls, groups	**MVP**	The core.
Broadcast channels	**MVP**	Shipped as Channels (Part 5 §5.6), chronological and unranked.
Status / Stories	**MVP**	Shipped without viewer counts or a ring above the inbox (Part 5 §5.7).
Disappearing messages	**MVP**	Per-conversation setting.
Multi-device sync	**MVP**	Per-device keys, no primary device (ADR-012).
Message editing, reactions	**MVP**	With visible edit history.
HD media	**MVP**	Original-quality sending is a first-class option, not buried.
Polls	**P2**	Genuinely useful in groups; needs group depth first.
Live location	**P2**	High value, real privacy design work — auto-expiry, granular audience, no historical trail retained.
Screen sharing	**P2**	Needs the group-call infrastructure that Phase 2 builds.
Communities (groups of groups)	**P3**	Requires MLS and moderation tooling to be honest at scale.

10.4 Telegram-Class Features

Feature	Verdict	Reasoning
Advanced search	**MVP**	On-device, local-first (Part 5 §5.10).

Feature	Verdict	Reasoning
Multiple accounts on one device	**P2**	Common in our target markets; needs multi-identity key management done carefully.
Scheduled messages, drafts	**P2**	Cheap, useful, no architectural cost.
Folder organisation	**P2**	Only if research shows users actually have enough conversations to need it. Not assumed.
Massive groups / supergroups	**P3**	Blocked on MLS (ADR-003). We publish the 1,000 cap rather than pretending otherwise.
Voice chat rooms	**P3**	As a channel format, not a social-audio product.
Live streaming	**P3**	Channel-owner broadcast only.
Bots	**P5**	With a permission model where a bot cannot initiate contact.
Mini Apps	**P5**	Sandboxed, no conversation access. The last thing we build, not the first.
Unlimited cloud storage	**Rejected as stated**	It is not free, and "unlimited" is a promise funded either by advertising (ADR-009 forbids it) or by subscribers cross-subsidising a small number of heavy users. We ship honest, stated limits with a paid tier instead.
Secret chats as a separate mode	**Rejected**	All personal chats are already E2EE by default. A separate "extra secure" mode implies the default is not secure, which teaches users exactly the wrong thing. Per-device isolation ships as a property, not a mode.

10.5 Facebook-Class Features

Feature	Verdict	Reasoning
Friends / social graph	**Rejected**	We have contacts and subscriptions. A bidirectional friend graph exists to power a feed and a recommendation engine, and we are building neither (Part 0.6 clause 2).
News Feed	**Rejected — permanently**	The single most consequential rejection in this document. A ranked feed of strangers is structurally incompatible with §0.6 clause 2 and with the entire calm thesis. It is also the mechanism by which every product in this category became the thing users complain about.
Pages	**P4**	As verified business profiles, not as a content-publishing surface.
Groups	**MVP**	Already core.
Events	**P3**	Inside communities, where they belong.
Marketplace	**Rejected**	A classifieds business with its own fraud, trust, logistics, and payments problems. It is a company, not a feature. Commerce arrives, if at all, through business messaging in Phase 4.

Feature	Verdict	Reasoning
Watch / Gaming	**Rejected**	Video content and games are attention businesses. Directly contrary to §0.4.
Dating	**Rejected**	An entirely different product with an entirely different safety model. Bundling it would compromise both.
Fundraising	**P5**	Only if payments infrastructure exists and the compliance burden is already carried.
Memories	**Rejected**	Resurfacing old content is engagement machinery wearing sentiment as a costume, and it has real capacity to hurt people.
Professional mode	**P4**	As part of business profiles.

10.6 Instagram-Class Features

Feature	Verdict	Reasoning
Stories	**MVP**	Deliberately capped (Part 5 §5.7).
Broadcast channels	**MVP**	Shipped as Channels.
Live streaming	**P3**	Channel format.
Creator tools & analytics	**P3**	Reach and engagement, without subscriber profiling.
Collaborative posts	**P3**	Co-authored channel posts.
Professional dashboards	**P4**	Business tooling.
Reels / short-form video	**Rejected — permanently**	The purest expression of the attention economy. Requires a recommendation engine, which requires a behavioural profile, which requires the data collection we have forsworn. It would also, on its own, make the app slower, larger, and louder for every user.
Posts / grid / carousels	**Rejected**	A permanent public content surface is a social network. Stories cover ephemeral sharing; channels cover broadcast.
Shopping	**Rejected**	See Marketplace.
Filters / AR effects	**P3, minimal**	Background blur in calls (on-device) and basic crop/rotate. No face filters — they are a content-platform feature and a heavy client cost.

10.7 Discord-Class Features

Feature	Verdict	Reasoning
Roles and permissions	**P2**	For groups, at the level groups actually need.
Voice channels	**P3**	As persistent group voice rooms inside communities.
Moderation tools	**P2 (channels) / P3 (communities)**	For public content only. Private conversations are never moderated because they cannot be read — a distinction we state plainly (Part 5 §5.6).
Stage events	**P3**	Channel live audio.
Screen sharing	**P2**	Already scheduled.
Servers as the primary structure	**Rejected**	Discord's server model is excellent for communities and hostile to one-to-one conversation. Our atomic unit is the conversation (Part 3 §3.1). Communities arrive in Phase 3 built on that unit, not replacing it.
Gaming integrations	**Rejected**	A different product for a different audience.

10.8 Signal-Class Features

Feature	Verdict	Reasoning
E2EE by default	**MVP**	Same protocol family, same guarantees (ADR-003).
Safety-number verification	**MVP**	With mandatory acknowledgement on change.
Sealed sender	**P3**	Genuine metadata reduction. Requires careful abuse-prevention design, since it removes a signal we use for spam.
Private contact discovery	**P3 research**	ADR-010 explains why we ship *no* address-book upload rather than a hashed one that leaks.
Post-quantum key agreement	**P3**	Hybrid X25519 + ML-KEM-768, on the roadmap with a date rather than in the marketing today.
"Stronger privacy than Signal"	**Rejected as a claim**	Signal's model is excellent and we adopt its protocol. We differ in some places (phone-optional identity is stronger; on-device AI is a capability they lack) and are weaker in others (they have years of audits and a non-profit structure that removes commercial pressure). Claiming superiority we have not earned would be exactly the kind of unearned confidence §0.4 forbids. We will let audits speak.

10.9 WeChat-Class Features

Feature	Verdict	Reasoning
Business messaging	**P4**	The commercially strongest item in this entire register, and the reason to build a business platform at all.
Payments	**P4, region by region**	Requires licensing, treasury, fraud operations, and per-market compliance. Launched as a real business line, never as a feature toggle.
Booking / ordering	**P5**	Via mini-apps, not as first-party features.
Government services	**P4, partnership-led**	Verified institutional accounts (Part 9 Phase 4). Guaranteed delivery to subscribers is genuinely valuable here.
Healthcare	**Rejected as a first-party product; P4 as a channel**	We will not build medical records, telemedicine, or prescriptions. We will let a verified health service message its patients — which is the 80 % of the value at 2 % of the regulatory burden.
Education	**Rejected as a first-party product; P4 as a channel**	Same reasoning. A school broadcasting to parents through a verified channel is high value. An LMS is a different company.
Transport	**Rejected**	No plausible connection to communication.
Mini Apps	**P5**	Sandboxed (Part 9 Phase 5).

10.10 The "Invent New Features" List

Proposal	Verdict	Reasoning
AI companion that summarises, translates, transcribes, detects scams, searches	**MVP**	This is the assistant (Part 4, Part 5 §5.8) — built on-device first.
AI that writes replies	**MVP, as suggestions only**	Drafts, never sends (Part 4 §4.2 rule 4).
AI that schedules meetings, organises files, answers from past conversations	**P4**	Requires an assistant-actions permission model that does not exist yet.
Universal real-time translation in chats	**MVP** (chats) / **P3** (calls)	Text now; calls when latency and quality justify it (Part 4 §4.4).
AI search by memory, person, image, location, object	**MVP, partially**	On-device semantic and literal search. Image and object search are **P3** and on-device only. **Emotion search is rejected** (Part 4 §4.11).

Proposal	Verdict	Reasoning
Universal communication (text → voice → video → livestream → webinar → community → course)	**Partially scheduled, mostly rejected as framed**	Text↔voice↔video switching is **MVP** and genuinely good. Video→livestream is **P3**. Livestream→webinar→community→course is a chain of increasingly different products; "one click turns a chat into a course" is a slide, not a specification.
AI Camera	**Rejected**	A camera "smarter than any social media camera" is a content-platform feature. We need a camera that captures quickly and reliably.
AI Creator Studio (generate posts, reels, podcasts, ads, logos, presentations)	**Rejected**	A creative-tools company. It has no dependency on being a messenger, which is the clearest possible sign it belongs in a different product.
AI Business Platform (CRM, invoices, orders, marketing automation)	**P4, narrowly**	Shared inbox, templates, and analytics. Not CRM, not invoicing, not marketing automation — those are SaaS categories with entrenched incumbents.
Commerce platform (stores, subscriptions, auctions, escrow, delivery)	**Rejected**	Multiple companies.
Deepfake detection	**P4 research**	We will not advertise a capability we cannot measure (Part 4 §4.7).
Quantum-resistant cryptography	**P3**	Scheduled with a specific algorithm (ADR-003, Part 6 §6.7).
Wearables, Smart TVs, cars	**P3, minimal**	Watch: notify and reply. TV: calls only. Car: platform standards (CarPlay/Android Auto), never a custom interface.

10.11 Rejections That Will Be Re-Proposed

These will come back — from a growth team, an investor, or a competitor's launch. They are rejected permanently, and the counter-argument is written here so it does not have to be re-derived under pressure:

Proposal	The pitch	The answer
A feed	"Users spend hours in feeds; we're leaving engagement on the table."	Engagement is not our metric (Part 11 §11.5). A feed requires a ranking model, which requires a behavioural profile, which requires the data collection ADR-009 and Part 0.6 forbid. It also permanently changes what the app is.
Ads, "just tastefully"	"One sponsored channel wouldn't hurt."	It would create an incentive to build more inventory, and that incentive never reverses. ADR-009.

Proposal	The pitch	The answer
Short-form video	"It's where attention is."	It is where attention is *taken*. Rejected in §10.6.
Read-your-messages AI	"Competitors do server-side AI and it's much better."	It is better, and it voids the promise. ADR-005 gives users the choice explicitly instead.
Address-book upload	"Growth is 3× slower without it."	Correct, and accepted (ADR-010).
Streaks and gamification	"Retention would go up."	Retention through manufactured obligation is the definition of a dark pattern (Part 0.6 clause 6).
Unlimited free storage	"Telegram does it."	Telegram's economics are not ours, and "unlimited" is never true. §10.4.
A crypto token	"Payments, but decentralised."	Adds regulatory risk, volatility, and fraud surface to a product whose entire value is trust.

10.12 The Process for Adding Anything

1. **Which of the three identity pillars does it strengthen** (Part 0 §0.7)? If none: rejected, no further discussion.
2. **What is its performance budget** (Part 8 §8.11)? A feature without one is not specified.
3. **What is its row in the AI privacy table** (Part 4 §4.9), if it touches user content?
4. **What does it cost every user who never uses it** — in binary size, cold start, memory, and cognitive load on the navigation? This question kills more proposals than any other, and it is the one most often skipped.
5. **Can a team of our current size build *and maintain* it at the quality bar?** If not, it is deferred to a phase where the answer is yes, or rejected.
6. **What are we deleting to make room?** Not always required — but asked every time, because a product that only adds is a product that is getting worse.

PART 11

11

Business, Growth & Compliance

Part 11 — How It Sustains Itself, and Under What Rules

Governed by Part 0, especially §0.6 clause 1 (no advertising) and ADR-009. Business decisions that conflict with Part 0.6 are void, regardless of revenue impact.

> "If a metric could be improved by making the product more annoying, it is not one of our metrics."

IN THIS PART

11.1 The Business Thesis

We are building a product whose defining qualities — calm, speed, privacy — are exactly the qualities an advertising business destroys. So the business model is chosen first and constrains everything after it, rather than being discovered later under pressure.

Three revenue lines, in the order they arrive:

1. **Consumer subscription (Phase 2).** People pay for more storage, larger files, and higher-quality AI.
2. **Business platform (Phase 4).** Businesses pay to have conversations with customers. Highest margin, most defensible, and structurally aligned — a business pays for a conversation the user *wanted*.
3. **Creator take rate (Phase 3).** A 10 % cut of creator earnings, deliberately below the market's 30 %, because we do not need to fund an ad business.

What the model implies. Free users cost money. Storage, bandwidth, and push are real per-user costs, so unit economics are a permanent product constraint, not a finance department problem. Every free-tier limit in this product exists because someone did the arithmetic, and every one of them is stated honestly in the interface rather than discovered at the moment of failure.

11.2 Subscription Plans

	SpaceTalk (free)	SpaceTalk Plus	SpaceTalk Business
Price	—	~$4.99/mo, regionally adjusted	Per-seat + per-conversation
Messaging, calls, groups, channels, stories	Everything	Everything	Everything
File size	2 GB	10 GB	10 GB
Media retention (server)	90 days after last access	1 year	Policy-controlled
Linked devices	4	10	Per policy
On-device AI	**All of it**	All of it	All of it
Server-assisted AI	Limited monthly allowance	Generous allowance	Pooled allowance
Translation language packs	5 concurrent	Unlimited	Unlimited
Shared inbox, roles, analytics	—	—	Yes
SSO, audit logs, data residency	—	—	Yes

Principles that govern pricing.

- **Nothing that protects a user is behind a paywall.** Encryption, scam detection, privacy controls, and blocking are free forever. Charging for safety is the clearest way to lose the right to call the product trustworthy.
- **Regional pricing is real pricing**, indexed to purchasing power, not a token discount. A product that intends to serve emerging markets and prices in US dollars does not actually intend to.
- **No feature is removed from the free tier once shipped.** Growth-by-degradation is a dark pattern.
- **Cancellation is one tap**, in-app, with no retention interstitial (Part 0.6 clause 6).
- **The upgrade prompt appears at the moment of the limit**, states the specific limit, and never interrupts a conversation.

11.3 Creator Economy (Phase 3)

- **Paid channel subscriptions** and one-off supporter payments.
- **10 % platform take rate**, versus 30 % elsewhere. This is a deliberate, structural advantage that follows from ADR-009: an advertising business has to fund itself from somewhere, and we do not have one.
- **Payouts within 7 days**, in local currency, with fees stated up front.
- **No algorithmic distribution**, which means no creator ever has to guess whether the platform will show their post. 100 % subscriber delivery (Part 5 §5.6) is the product promise to creators, and it is the opposite of every incumbent's relationship with them.
- **Analytics report reach and engagement, never subscriber demographics** — we do not collect that data, so we cannot sell reporting built on it, and we tell creators why.
- **No exclusivity contracts, no creator fund, no promotional boosts.** Every one of those is a mechanism for the platform to pick winners.

11.4 Unit Economics

Approximate per-user monthly cost at Phase 2 scale, which is what every free-tier limit in §11.2 is derived from:

Cost	Free user	Plus user
Storage (media, ciphertext)	$0.02–0.08	$0.15–0.40
Bandwidth (egress + CDN)	$0.03–0.10	$0.10–0.30
Compute (gateway, core, push)	$0.01–0.03	$0.02–0.05
Calls (TURN/SFU)	$0.01–0.05	$0.02–0.08

Cost	Free user	Plus user
Server-assisted AI	~$0.00 (on-device)	$0.10–0.50
Total	**~$0.07–0.26**	**~$0.39–1.33**

The economics work because of two architectural decisions, and it is worth naming them: delivered message envelopes are deleted rather than archived (ADR-004), so the server does not carry the world's message history; and AI runs on-device by default (ADR-005), so the largest cost line in a modern AI product is close to zero for most usage. Privacy decisions and cost decisions turn out to be the same decisions here — which is why they are stable.

Target: contribution-positive per paying user from launch of Plus; company-level profitability by Phase 3 (Part 9 gate).

11.5 Analytics

What we measure:

- Delivery success, latency percentiles, crash rate, cold start — by device tier, network profile, and region.
- Retention cohorts (D1, D7, D30, D90).
- Feature adoption as a *count of users who used it*, never as time spent.
- Funnel completion for onboarding, device linking, and subscription.
- Error and failure rates, per surface.

What we do not measure, ever: time in app, session count as a goal, scroll depth, message volume as a success metric, or anything about message *content*.

How.

- **Event data is pseudonymous and aggregated**, retained 13 months (Part 6 §6.13).
- **No third-party analytics SDKs in the client.** Not one. Every analytics SDK is a data-sharing relationship with a company the user never agreed to.
- **Analytics are opt-out in the EU and opt-in wherever law requires it**, with a plain-language description of every event category.
- **No event may carry message content, contact identifiers, or a precise location.** Enforced by a schema allowlist in CI — an event that adds a disallowed field fails the build.

The governing rule: if a metric could be improved by making the product more annoying, it is not one of our metrics. This single test eliminates almost every engagement metric in the industry.

11.6 Growth Strategy

We have deliberately given up the industry's main growth engine (ADR-010: no address-book upload). What replaces it:

1. **Be conspicuously faster.** Speed is the most demonstrable, least explainable-away differentiator. A side-by-side launch comparison is a marketing asset that requires no argument.
2. **Share links and QR codes as the primary connection mechanism** (Part 3 §3.10), designed to be pleasant enough that people actually send them.
3. **Channel-led acquisition.** A creator, school, or institution brings its audience. This is why Channels is an MVP feature rather than a Phase 2 one, and why guaranteed delivery matters commercially as well as ethically.
4. **Language-first market entry.** Enter markets where translation is the daily problem, not an occasional one — multilingual regions, migrant communities, cross-border families and businesses. In those markets translation is not a feature, it is the reason to switch.
5. **Trust as a growth channel.** Published audits, a transparency report, reproducible builds, and open protocol documentation. Slow, compounding, and very hard for an ad-funded competitor to copy.
6. **No growth hacking.** No dark-pattern invites, no "your friend joined" notifications, no contact-list nagging, no fake activity. Every one of them would violate Part 0.6.

Honest expectation: slower early growth than a contacts-uploading competitor, with better retention per acquired user. The gate metrics in Part 9 are retention metrics, not signup metrics, precisely because that is the shape we expect and the shape we want.

11.7 Moderation Architecture

> **The foundational distinction, stated plainly:** private conversations are end-to-end encrypted, so we cannot read them and therefore cannot moderate their content. Public content — channels, public group metadata, profiles, and usernames — we can and do moderate. We state this distinction clearly rather than blurring it, because blurring it is how platforms end up promising both perfect privacy and perfect safety and delivering neither.

What we do for private conversations:

- **User-side tools:** block (silent and complete), report (which shares only the specific reported messages, with the user told exactly what will be shared), message requests for unknown senders, and granular privacy controls.
- **Metadata-based abuse prevention** (Part 4 §4.7): fan-out rate, account age, block and report rates, registration patterns. This catches the overwhelming majority of spam without any content access.
- **On-device scam detection** with content never leaving the device.

What we do for public content:

- Reactive review of reported channels and profiles, by a trained human team with published policies and a published appeals process.
- Proactive detection limited to impersonation (name similarity at registration) and known-illegal content hashes where legally required.
- Published enforcement statistics in the transparency report.

Child safety. The most difficult area, and the one where we refuse to be vague. We will not build client-side scanning of private messages: it is a general-purpose surveillance capability that, once built, cannot be limited to one purpose, and the technical consensus is that it does not survive adversarial contact. What we do instead: default-private profiles for accounts registered as minors, strict limits on who may initiate contact with them, aggressive metadata-based detection of grooming *patterns* (mass contact of new accounts, rapid block accumulation), immediate reporting to the relevant authority when public content is identified, and active participation in industry information-sharing. We will state this position publicly and defend it, rather than implying a capability we have chosen not to build.

Government requests. Answered only through valid legal process, narrowly scoped, with the affected user notified unless legally prohibited. We publish what we were compelled to provide — and, importantly, what we were *architecturally unable* to provide. Where a jurisdiction demands a backdoor, we exit the market rather than comply. This is a Part 0.6 clause 3 consequence, and it is written down now so it is not decided for the first time under pressure.

11.8 Internationalisation

Launch languages (Phase 1, 12): English, Spanish, Portuguese, French, German, Arabic, Hindi, Indonesian, Russian, Turkish, Japanese, Simplified Chinese. Chosen by target-market reach, not by convenience.

Engineering requirements, enforced in CI:

- **No concatenated sentences.** Every user-visible string is a complete, translatable unit with named placeholders. Building a sentence from fragments produces gibberish in most languages and is a build failure here.
- **Full ICU message format** for plurals, gender, and select cases. Languages with six plural forms are not an edge case, they are Arabic and Russian.
- **RTL is a first-class layout**, not a mirror transform (Part 2 §2.13). Every screen is reviewed in Arabic before it ships.
- **Locale-aware everything:** dates, times, numbers, currency, name order, calendars (Hijri and Gregorian where relevant), and week start.

- **Pseudo-localisation in CI** — every string expanded 40 % and wrapped in markers, so truncation and hard-coded strings are caught automatically rather than by a translator later.
- **Translation quality is reviewed by native speakers, per release.** Machine translation of the interface of a product whose flagship feature is translation would be indefensible.

Content translation is a separate concern, specified in Part 4 §4.4.

11.9 Legal and Compliance

Regime	Obligation	Our position
GDPR (EU)	Lawful basis, DSARs, erasure, portability, DPO, DPIAs	Data minimisation is architectural (Part 6 §6.13). Export and delete are self-service (Part 0.6 clause 10). DPIA per feature touching personal data.
ePrivacy	Consent for non-essential storage	No advertising trackers exist to consent to.
DSA (EU)	Notice-and-action, transparency reporting, appeals	Applies to public content. Private messaging is out of scope, and we document why.
DMA (EU)	Interoperability if designated a gatekeeper	Phase 5. Implemented with explicit labelling of where our encryption guarantees stop (Part 9 Phase 5).
CCPA/CPRA (California)	Disclosure, deletion, opt-out of sale	We do not sell data. Disclosure and deletion are self-service.
COPPA / age assurance	Under-13 protections; varying regional age gates	Minimum age 13 (16 where required). Age assurance without collecting identity documents — a hard problem we will solve conservatively rather than by demanding IDs.
UK Online Safety Act	Duties of care; potential scanning pressure	Compliance with everything short of breaking encryption. Our published position on client-side scanning (§11.7) applies.
Telecom / VoIP	Per-market registration; emergency-calling rules	Market-by-market legal review before launch. **We state clearly that SpaceTalk does not support emergency calls** — a real obligation that VoIP products routinely handle badly.
Payments (Phase 4)	Licensing, KYC/AML, PCI-DSS	A separate regulated business line, launched per market. Never a feature toggle.
Export control	Cryptography export regimes	Standard notifications filed; we use published, standard algorithms.
Data residency	Regional storage requirements	Regional deployment from Phase 3 (Part 6 §6.10).

Standing commitments.

- Terms of service and privacy policy written in plain language, at a reading level a fifteen-year-old can follow, with a summary at the top. If our own users cannot understand what they agreed to, consent is a formality rather than a fact.
- Every material change is notified in-app 30 days in advance.
- No arbitration clause that removes a user's right to a remedy.
- The privacy policy describes what we actually do, verified against the codebase annually by someone outside the team that wrote it.

11.10 Risk Register

The things most likely to kill this product, and what we do about each.

Risk	Likelihood	Impact	Mitigation
Network effects — nobody switches messengers alone	High	Fatal	Channel-led and language-led acquisition (§11.6); make the switch worth it for a *pair* of people, not one
Growth too slow without contact discovery	High	Severe	Accepted cost (ADR-010); gates are retention-based, and we fund accordingly
A crypto implementation flaw	Medium	Fatal	Use libsignal, never roll our own; conformance vectors in CI; independent audit pre-launch and annually
Performance promise not met on Tier C	Medium	Severe	Budgets as release gates (Part 8 §8.11); physical device lab; quarterly performance week
On-device AI quality below expectation	Medium	Moderate	Honest per-language accuracy notes (Part 4 §4.10); explicit-grant server path for users who want it
Abuse at scale without content moderation	Medium	Severe	Metadata detection, message requests, on-device classifiers, fast reporting (§11.7)
Regulatory pressure to break encryption	Medium	Fatal if conceded	Published position; exit a market rather than comply (§11.7)
A big-tech competitor copies the calm positioning	Medium	Moderate	They cannot: their revenue depends on the machinery we refuse to build. This is the structural moat.
Team dilutes the standards under growth pressure	**High**	**Fatal, slowly**	This Bible, the amendment process (Part 0 §0.10), and gates that are evidence-based rather than calendar-based. Every product in this category lost this fight by degrees, and the loss was never a single decision.

The last row is the one to take most seriously. No competitor will beat SpaceTalk in a way that is visible on a dashboard. The realistic failure is thirty small, individually reasonable

compromises over four years, at the end of which the product is another loud messenger with an AI tab. That is what the constitution in Part 0 exists to prevent, and it only works if people actually invoke it.

PART 12

12

Architecture Decision Records

Part 12 — Decisions With Consequences

Each record states the decision, the context, the alternatives seriously considered, the consequences we accept, and the conditions under which we would revisit. An ADR is never edited after acceptance — it is superseded by a new one.

> "An ADR is never edited after acceptance — it is superseded by a new one."

IN THIS PART

Status values: Accepted · Superseded · Deprecated.

ADR-001 Flutter for every client platform

Status: Accepted

Context. We must ship iOS, Android, and a linked-device client at MVP with a small team, and reach desktop in Phase 2. Native-per-platform gives the best ceiling on performance and platform integration. Cross-platform gives velocity and consistency.

Decision. Flutter for all client surfaces, with thin native platform channels for notifications, camera, keystore, and background execution.

Alternatives considered.

- *Native Swift + Kotlin.* Best possible performance and platform fidelity. Rejected: it is two teams, two release cycles, and two implementations of every subtle piece of encryption and sync logic — which is also two places for those bugs to differ. For a team of our size it is the difference between shipping and not.
- *React Native.* Larger hiring pool. Rejected: the JS bridge remains a latency and jank risk in exactly our hottest path (a long, media-rich, constantly-updating scroll view), and our differentiator is measured frame rate.
- *Kotlin Multiplatform with native UI.* Shares logic, not UI. A genuinely strong option. Rejected on the grounds that our UI *consistency* is a brand requirement (Part 0 §0.5), and KMP would leave us building the transcript twice.

Consequences we accept.

- Some platform features arrive later than native apps get them.
- Larger binary size than a native app (mitigated per Part 8 §8.6).
- We must be unusually disciplined about Flutter's known jank sources (Part 6 §6.2) — shader warm-up, save layers, image decode — and we treat them as first-class engineering work, not incidental tuning.
- Flutter web is not as good as a native desktop app; hence ADR-012.

Revisit if. Frame-rate targets prove unreachable on Tier-C hardware after a dedicated optimisation effort, or a platform materially restricts non-native clients.

ADR-002 A modular monolith in Go, not microservices

Status: Accepted

Context. The reference architectures for messaging at scale are microservice-based. We are not at scale, and we will not be for at least two years.

Decision. Four Go deployables (`gateway`, `core`, `media`, `push`), with `core` internally modular and boundaries enforced by an import linter. Postgres + Redis + object storage. No Kubernetes, no service mesh, no event sourcing at MVP.

Alternatives considered.

- *Microservices from day one.* Rejected: distributed transactions, tracing complexity, and deployment overhead consume a small team's entire capacity, and every boundary drawn before the domain is understood is drawn in the wrong place.
- *Elixir/Phoenix.* Genuinely excellent for millions of concurrent connections, and the BEAM's supervision model fits a gateway well. Rejected on hiring: the pool is small, and a language nobody can hire for is a liability that compounds. Go gets us most of the concurrency story with a far larger pool.
- *Node.js/TypeScript.* Rejected for the fan-out path — single-threaded event-loop stalls under CPU-bound crypto and protobuf work are hard to reason about at p99.

Consequences we accept.

- A larger blast radius per deploy in `core`, mitigated by feature flags and staged rollout.
- We will have to extract services later, under load. We reduce that cost by enforcing module boundaries now, so extraction is mechanical.
- Some scaling ceilings arrive earlier. Each has a written trigger (Part 6 §6.10).

Revisit if. Any module's scaling profile diverges more than 5× from the rest, or team size exceeds roughly 25 backend engineers, at which point deployment independence starts paying for itself.

ADR-003 Signal Protocol via libsignal; Sender Keys for groups; MLS deferred

Status: Accepted

Context. We promise end-to-end encryption for personal messaging as a non-negotiable (Part 0 §0.6.3). We must choose a protocol and an implementation.

Decision. X3DH + Double Ratchet via the official `libsignal` Rust implementation, bound into Flutter through `dart:ffi`. Group messaging uses Sender Keys. **MLS (RFC 9420) is a Phase 3 migration**, planned and scheduled, not claimed earlier.

Alternatives considered.

- *Implement the protocol ourselves in Dart.* Rejected without hesitation. Cryptographic implementation bugs are silent, catastrophic, and disproportionately harm the people most in need of the protection.
- *MLS at MVP.* The better long-term protocol: logarithmic group operations, a clean multi-device story, and an actual standard. Rejected for MVP because implementations were not yet battle-tested at the maturity we need, and the integration risk sits on the

critical path of everything. We take Sender Keys' known ceiling and schedule the migration.

- *Matrix/Olm+Megolm.* Rejected: adopting the federation model brings metadata and moderation properties we have not chosen.

Consequences we accept.

- Sender Keys make group key distribution O(members), which is exactly why groups are capped at 1,000 at MVP (Part 5 §5.5). We state the cap as a product decision because it *is* one, derived from this protocol choice.
- Removing a member requires a full sender-key rotation — a visible cost in large groups.
- The FFI boundary is the highest-risk integration in the client and is treated accordingly (pinned versions, official test vectors on every commit, dedicated audit — Part 6 §6.12).

Revisit. Scheduled: Phase 3 MLS migration, planned as a dual-stack transition with per-conversation upgrade, never a flag day.

ADR-004 PostgreSQL for everything at MVP; ScyllaDB only on trigger

Status: Accepted

Context. Messaging workloads are usually cited as a wide-column-store use case. We have no traffic yet.

Decision. Postgres 16 for all persistent state, with message envelopes in monthly partitions. All envelope access goes through one repository interface so the store can be swapped. ScyllaDB is deployed only when envelope writes sustain >30 K/s or the undelivered table exceeds 500 GB.

Alternatives considered.

- *Cassandra/Scylla from day one.* Rejected: no transactions, weaker consistency, harder operations, and vastly more expensive per unit of engineering attention — to solve a problem we do not have.
- *A separate store per data type at MVP.* Rejected: five datastores is five on-call surfaces.

Consequences we accept.

- A migration is in our future if we succeed. We reduce its cost with the repository interface and by never letting application code depend on Postgres-specific behaviour in the envelope path.
- Partition maintenance is operational work we take on now.

Key enabling insight. Delivered envelopes are deleted, not archived (Part 6 §6.5). The server is a relay, not an archive. That single decision keeps the hot dataset proportional to *undelivered* traffic rather than to total history, and it is what makes Postgres viable far longer than intuition suggests.

ADR-005 AI never processes E2EE content without an explicit, visible, revocable grant

Status: Accepted

Context. The product promises both strong privacy and strong intelligence. These genuinely conflict, and the conflict cannot be resolved by wording.

Decision. A three-tier rule, in priority order: on-device first; server-side only with a per-conversation, visible, revocable grant that is disclosed to all participants; otherwise the feature does not ship. The assistant conversation is a separate, explicitly non-E2EE surface, labelled as such in the interface.

Alternatives considered.

- *Server-side AI on all content with a privacy policy.* Rejected: it makes the encryption promise a marketing claim.
- *No AI features at all.* Rejected: it forfeits one of the three things the product exists to be (Part 0 §0.7).
- *Confidential computing / TEEs for server-side processing.* Promising, and on the Phase 4 research list. Rejected for MVP: attestation for a user is not verifiable in practice today, and "trust our enclave" is a weaker guarantee than "it never left your phone."

Consequences we accept.

- Some AI features are worse than a competitor's equivalent, because small on-device models are worse than large server models. We take the quality hit and say why.
- Semantic search covers only on-device history (Part 4 §4.6).
- On-device model work becomes a core engineering competency and a real cost centre.
- We must ship an audit that continuously verifies zero ungranted AI invocations on E2EE content, and treat a non-zero result as Sev-1.

ADR-006 LiveKit SFU with insertable-stream E2EE for group calls

Status: Accepted

Context. 1:1 calls can be peer-to-peer. Group calls cannot — mesh topology fails past three or four participants on mobile uplinks.

Decision. Self-hosted LiveKit as the SFU, with E2EE via insertable streams (SFrame), so the SFU forwards frames it cannot decrypt. 1:1 calls attempt P2P first and fall back to TURN.

Alternatives considered.

- *MCU (server-side mixing).* Better for very weak clients. Rejected outright: mixing requires decryption, which means server-side plaintext media.
- *A managed third-party SFU.* Faster to ship. Rejected: media routing is core infrastructure, and a third party in the media path is a party in a conversation.
- *Mesh for small groups.* Retained as an optimisation for 3-participant calls where uplink measurement supports it; not the primary architecture.

Consequences we accept.

- Operating an SFU fleet is real work (TURN, regional deployment, capacity planning).
- Insertable-stream E2EE costs some CPU and rules out server-side features that would need plaintext (server-side recording, server-side transcription) — which is consistent with ADR-005 and therefore not a loss we mind.
- Simulcast layer selection must be implemented carefully to honour the audio-priority rule (Part 5 §5.4).

ADR-007 Push notification payloads carry no content

Status: Accepted

Context. The simplest push implementation sends the message text in the payload. It is also the one that hands message content to Apple and Google.

Decision. Payloads contain only `{envelope_id, conversation_id_hash, priority}`. The device fetches and decrypts locally — on iOS via a Notification Service Extension, on Android in the FCM handler.

Alternatives considered.

- *Content in the payload.* Rejected: it defeats E2EE at the last metre for the most-read text in the product.
- *Generic "New message" with no fetch.* Rejected: it degrades the experience for no additional privacy over the chosen design.

Consequences we accept.

- A network round trip in the notification path, which we budget for (p95 < 2 s end to end).
- The iOS extension has a hard memory limit (24 MB) and a short execution window, so the decryption path must be lean and cannot load the full client stack.

- If the fetch fails, we fall back to “New message” rather than showing nothing.

ADR-008 Local-first architecture with an outbox

Status: Accepted

Context. Our users are frequently on unreliable networks. A request/response client is unusable there, and “offline support” bolted on afterwards never really works.

Decision. The on-device SQLite database is the source of truth for the UI. All mutations are written locally first and drained from a durable outbox. The server is a synchronisation and relay mechanism.

Alternatives considered.

- *Network-first with a cache.* Rejected: every screen becomes a loading state, and offline behaviour is a permanent afterthought.
- *A CRDT-based sync engine.* Genuinely elegant. Rejected as over-engineering for our data shapes: messages are append-only with server-assigned ordering, which is a much simpler problem than general concurrent editing. We would be paying CRDT complexity for a conflict class we mostly do not have.

Consequences we accept.

- Migration discipline on the local schema becomes critical — a bad client migration can destroy a user’s history, so every migration is forward-tested against real historical databases in CI.
- Local storage grows with history, requiring user-visible storage management.
- Some server-side changes need client-side reconciliation logic.

ADR-009 No advertising, ever; subscription and business revenue only

Status: Accepted

Context. Advertising is the default business model for communication products at scale, and it is available to us.

Decision. No advertising in any form. Revenue from consumer subscriptions (SpaceTalk Plus), business platform fees, and, from Phase 3, creator monetisation take rates. Codified as Part 0 §0.6 clause 1.

Alternatives considered.

- *Ads in a non-conversation surface.* Rejected: it would create an internal incentive to build such a surface and then to grow it — which is precisely how every calm product became a loud one.
- *Selling anonymised data.* Rejected: not re-identification-safe in practice, and inconsistent with everything else here.

Consequences we accept.

- Slower revenue ramp and a harder fundraising story against ad-supported comparables.
- Free users cost money; unit economics discipline is a permanent product constraint, not a finance problem (Part 11 §11.4).
- We cannot subsidise unlimited free storage, which is why file retention has honest, stated limits.

> **Why it is an ADR and not just a policy.** Because it constrains architecture: no ad-serving infrastructure, no user-profiling pipeline, no engagement-ranking model, and no data-collection surface built "in case." Those absences are load-bearing.

ADR-010 No address-book upload at MVP; username-first discovery

Status: Accepted

Context. Contact discovery drives growth, and every mainstream approach leaks. Hashed phone numbers are trivially reversible across the whole global number space; private set intersection at scale needs infrastructure (secure enclaves, custom protocols) that a startup cannot make credible.

Decision. No address-book upload at MVP. Discovery is by username, QR code, or share link. Discoverability by phone/email is opt-in and off by default (Part 5 §5.11).

Alternatives considered.

- *Hashed phone-number upload.* The industry standard. Rejected: the hash provides essentially no protection for a 10–15 digit space, and shipping it would mean holding a social graph we promised not to build.
- *PSI with a trusted enclave.* The right long-term answer. Deferred: we cannot make the attestation story honest at our current maturity.

Consequences we accept.

- **Materially slower viral growth.** This is the single largest deliberate growth cost in the product, and we are taking it with open eyes. Growth strategy compensates through share links and channel-led acquisition (Part 11 §11.6).
- Some users find the app "empty" at first, which puts real weight on the onboarding empty state (Part 3 §3.6).

Revisit if. A discovery mechanism exists that we can explain honestly in two sentences to a non-technical user and defend against a well-resourced adversary. Phase 3 research item.

ADR-011 SpaceTalk is a greenfield product, not an extension of the existing StromeX codebase

Status: Accepted

Context. This repository already contains StromeX — an AI knowledge-work product built on FastAPI (Python) and Next.js, with its own Editorial Bible under `docs/`. SpaceTalk is a real-time communication platform. Reuse was considered.

Decision. SpaceTalk is built as a separate product with its own stack (Flutter + Go). The StromeX application code and documentation are left untouched; SpaceTalk documentation lives under `docs/spacetalk/`.

Alternatives considered.

- *Extend the existing FastAPI backend.* Rejected: Python's concurrency model is a poor fit for holding hundreds of thousands of persistent WebSocket connections and for the CPU-bound fan-out path, and retrofitting a request/response codebase into a realtime one is usually slower than starting clean.
- *Reuse the Next.js web client as the primary surface.* Rejected: the MVP is mobile-first (Part 0 §0.5), and ADR-001 commits the client to Flutter.

Consequences we accept.

- Two stacks in one organisation, with the operational and hiring overhead that implies.
- Shared concerns (identity, billing) will need deliberate integration if the two products ever converge.

Open question for the founder. If SpaceTalk is intended as a *rename* of StromeX rather than a second product, this ADR is wrong and must be superseded — the two products have different missions, users, and architectures, and merging them would require revisiting Part 0 §0.1 first.

ADR-012 Multi-device with per-device identity keys; web as the MVP linked client

Status: Accepted

Context. The MVP requires multi-device support. Historically, encrypted messengers either tethered companion devices to a primary phone (fragile, and it dies with the phone's battery) or shared keys across devices (which weakens the security model).

Decision. Every device is an independent cryptographic identity with its own sessions and its own prekey bundle. Senders encrypt once per recipient *device*. No primary device exists. At MVP, the linked client is the Flutter web build; native desktop follows in Phase 2 from the same codebase.

Alternatives considered.

- *Tethered companion devices.* Simpler. Rejected: the phone must stay online, which is exactly the failure mode users hate most.
- *Shared identity key across devices.* Rejected: one compromised device compromises all of them, and revocation becomes meaningless.
- *Native desktop at MVP.* Rejected on scope: the web client covers the linked-device need at a fraction of the cost, and shipping four platforms well at MVP is not achievable excellence.

Consequences we accept.

- Fan-out is O(recipients × devices), which we cap at 4 devices per account.
- Sender-key rotation on device changes adds load in large groups.
- History does not automatically appear on a new device; it is an explicit, user-controlled sync with a stated transfer size (Part 5 §5.13).
- Web client key storage (IndexedDB, non-extractable WebCrypto keys) is weaker than a native keystore. We state this in the interface at link time rather than implying parity.

PART 13

13

Research, Journeys & IA

Part 13 — Who We Are Building For, and What They Do

Governed by Part 0. Behaviour rules are in Part 3; this document holds the research programme, the user model, the journeys, and the information architecture those rules serve.

> "A persona presented as fact is one of the most reliably damaging artefacts in product development."

IN THIS PART

An honest framing, up front. No user research has been conducted for SpaceTalk. The user model below is therefore a set of **hypotheses to be tested**, not findings to be cited. They are written as falsifiable statements with a stated test, because a persona presented as fact — invented, given a name and a stock photo, and then quoted in design reviews for three years — is one of the most reliably damaging artefacts in product development. Every hypothesis here is either confirmed by the research in §13.2 before Phase 1 ships, or revised.

13.1 The Core Hypotheses

#	Hypothesis	How we falsify it
H1	People notice and value messaging speed, and can be moved by it. Cold start and send latency are felt, not just measured.	Side-by-side timed task tests against incumbents; unprompted mentions of speed in post-use interviews. If nobody mentions speed unprompted, Identity Pillar 1 is not a positioning, only an engineering standard.
H2	Notification fatigue is severe enough that "quiet by default" is a reason to switch, not merely a nice property.	Diary studies of notification volume and reaction; measure whether the calm promise appears in stated switching reasons.
H3	Translation inside conversation is a daily need for a large, identifiable population — not an occasional convenience.	Field research in multilingual regions and among migrant and cross-border-business communities; frequency counts, not attitudes.
H4	Users will accept a phone-number-free, username-first identity and find people via links and QR codes.	Onboarding completion and first-conversation rates without address-book access. **This is the highest-risk hypothesis in the product** (ADR-010).
H5	Users want AI help with specific tasks (translate, catch up, transcribe) and actively dislike an assistant that speaks unprompted.	Concept tests contrasting invited versus proactive assistance; measure irritation as carefully as usefulness.
H6	Privacy is a tiebreaker, not a primary driver, for mainstream users — but it is a primary driver for an identifiable early-adopter segment large enough to launch into.	Segment-level willingness-to-switch studies. If false, the go-to-market in Part 11 §11.6 needs reordering, though nothing in Part 0.6 changes.
H7	Group conversations fail for structural reasons (undifferentiated urgency), and @mention-cuts-through-mute materially fixes it.	Longitudinal group-health tracking: mute rate, abandonment, and message-per-member decay over 90 days.

The rule: any of these that survives Phase 1 unvalidated must be marked as unvalidated wherever it is used to justify a decision. We do not launder assumption into fact by repetition.

13.2 The Research Programme

Before MVP ships:

Method	Scope	Answers
Contextual inquiry	40 participants, 4 markets, on their own devices, in their own environments	How messaging actually happens — one-handed, interrupted, on bad networks. Grounds Part 3 §3.12.
Comparative timed tasks	30 participants	H1. Does speed register perceptually?
Notification diary study	25 participants, 2 weeks	H2. Real volume, real reactions.
Multilingual field study	30 participants across 3 multilingual regions	H3. Translation frequency and failure modes.
Onboarding usability	20 participants, iterative	H4. The 90-second target in Part 3 §3.10.
AI concept testing	25 participants	H5. Invited versus proactive.
Accessibility research	12 participants who use screen readers, switch control, or large type daily	Not a compliance check — a design input. Run early enough to change the design, which means before the components are built.

Continuously, after launch:

- Quarterly longitudinal panel (~50 users, tracked over time) — the only method that reliably catches slow degradation.
- Retention-cohort interviews with users who *left*, which is where the real information is.
- Group-health tracking (H7).
- Per-language AI quality panels, native speakers, feeding the published accuracy notes (Part 4 §4.10).

Research standards. Recruit outside our own network. Never test with a design or engineering team member facilitating their own work. Include Tier-C devices and poor networks in every session — testing on a flagship on office Wi-Fi produces conclusions that do not survive contact with the market we are entering. Publish findings internally in full, including the ones that contradict the roadmap.

13.3 Who We Are Building For

Segments, defined by behaviour and constraint rather than by demography — and stated as hypotheses per §13.1.

The cross-language communicator. Talks daily with people who do not share their first language: family across borders, a small business with foreign suppliers, a migrant worker. Currently copy-pastes into a translation app, several times a day, losing tone and context each time. *This is our sharpest wedge, and translation is why they would switch.*

The overwhelmed group member. In fifteen groups, has muted eleven, and now misses things that matter. Wants to participate partially without either drowning or disappearing. Served by @mention-cuts-through-mute, granular mute, and the absence of engagement notifications.

The privacy-attentive early adopter. Already uses an encrypted messenger, cannot persuade their contacts to join it, and gives up capability for privacy today. Served by not making them choose — and reached first, because they are the segment that evaluates on architecture rather than on familiarity.

The constrained-device user. An entry-level phone, limited storage, a metered data plan, and an unreliable network. Currently endures an app that is slow, huge, and data-hungry. Served by every number in Part 8 — and this is the segment most likely to be quietly failed if Tier C is treated as a fallback rather than the design target.

The broadcaster. A creator, a teacher, a clinic, or a municipal office that needs to reach a subscribed audience reliably. Currently at the mercy of an algorithm that decides whether their announcement is seen. Served by 100 % chronological delivery (Part 5 §5.6).

Who we are explicitly not building for: people who want a social feed, an audience of strangers, short-form video, or a place to be discovered. That is not a market we are underserving by accident; it is one we are declining on purpose (Part 10).

13.4 Primary User Journeys

Each journey states the entry point, the steps, the target time, the failure modes, and what it is worth measuring.

FIGURE — J1: FIRST RUN TO FIRST MESSAGE SENT

Target: under 90 seconds, four screens. Nothing is requested until it is needed for something the user already wants.

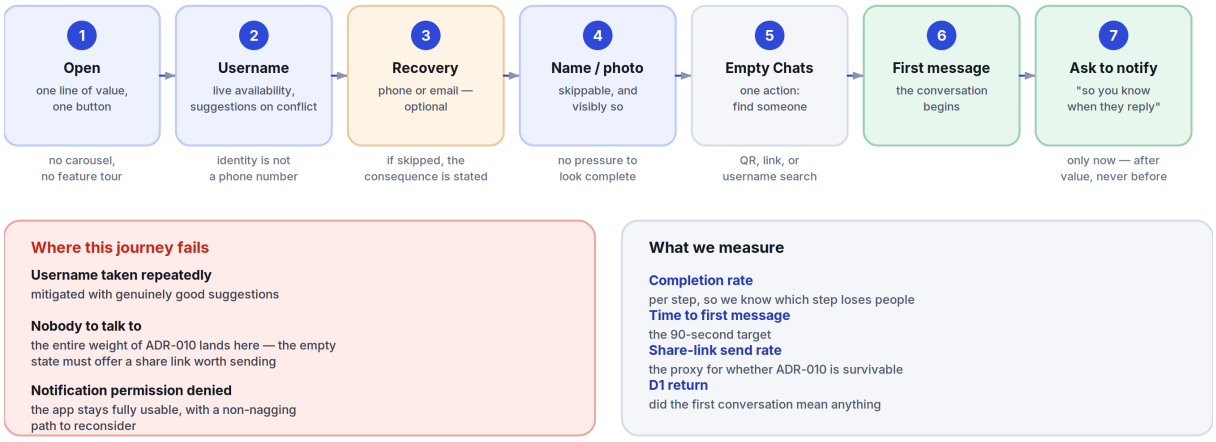
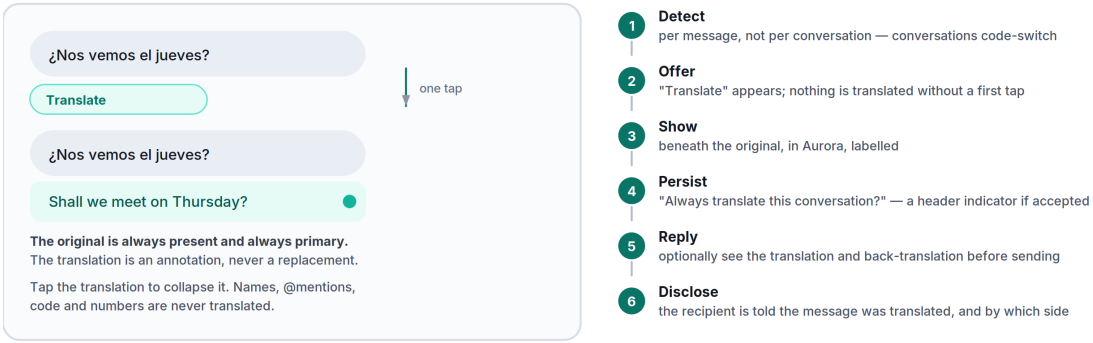


Figure 13.1 The onboarding journey, its three failure modes, and the four numbers that tell us whether it works.

FIGURE — J4: A CONVERSATION ACROSS A LANGUAGE BARRIER

The journey that most defines the product's differentiation.



Hidden translation would create a false impression of shared fluency — so it is disclosed. Low-confidence output is labelled "Rough translation" rather than presented as reliable.

Figure 13.2 Translation as an annotation on the conversation, never a replacement for what was actually said.

J1 — First run to first message sent

Target: under 90 seconds, four screens (Part 3 §3.10).

1. Open → value statement, one line, one button. *(no carousel, no account tour)*
2. Choose a username → live availability check, suggestions on conflict.
3. Add a recovery method → phone or email, with a plain, unavoidable statement of what happens if they skip it and lose the device (Part 5 §5.11).
4. Display name and optional photo → skippable, and visibly so.
5. Land on an empty Chats screen with one action: *Find someone* (QR, link, or username search).
6. First conversation → first message sent.
7. **Only now** ask for notification permission, framed as "so you know when they reply" (Part 3 §3.9).

Failure modes. Username taken repeatedly (mitigate with good suggestions); no one to talk to (the entire weight of ADR-010 lands here — the empty state must offer a share link that is genuinely pleasant to send); permission denied (the app must remain fully usable, with a non-nagging path to reconsider).

Measure: completion rate per step, time to first message, share-link send rate, and D1 return.

J2 — Daily glance

The highest-frequency journey in the product, run 50–150 times a day. It has three steps and must be flawless.

1. Notification or app icon → app open (**<450 ms Tier A / <1,500 ms Tier C**, Part 8 §8.2).
2. Conversation list, already populated from disk. No spinner, no skeleton, no shift.
3. Tap → transcript at the last-read position, rendered locally, in <120 ms.

Failure modes. Cold-start regression (a release gate); list jumping as network data arrives (forbidden by Part 3 §3.4); notification opening the wrong conversation or losing the back stack (Part 3 §3.1 rule 6).

Measure: cold start p95 by tier, notification-to-conversation latency, and any rendered-then-shifted event, which is treated as a defect rather than a metric.

J3 — Send under bad conditions

Entry: composing on a poor or intermittent connection — the condition a large share of our users live in permanently.

1. Type; draft persists on every keystroke pause.
2. Send → bubble appears immediately with a pending glyph (**<50 ms**, local write).
3. Network absent → one quiet banner, no per-message error, no modal.
4. Reconnect → queue drains in order; ticks update in place; no announcement.

Failure modes. Reordering (prevented by outbox sequencing, Part 6 §6.8); silent loss (the worst failure in a messenger — prevented by the durable outbox and gap detection); an error-per-message storm (prevented by the severity ladder, Part 3 §3.5).

Measure: outbox drain success, message loss (target: zero, treated as correctness), and duplicate delivery (prevented by idempotency keys).

J4 — Cross-language conversation

The journey that most defines the product's differentiation.

1. Receive a message in another language.
2. **Automatic detection** → a "Translate" affordance appears under the message. Nothing is translated without a tap the first time.
3. Tap → translation appears beneath the original, in Aurora, labelled. The original stays.
4. Offer: "Always translate this conversation?" → a persistent header indicator if accepted.
5. Reply → optionally see the translation and back-translation before sending.
6. The recipient sees that the message was translated (Part 4 §4.4).

Failure modes. No language pack (offer download over Wi-Fi, state the size); low-confidence output (label it "Rough translation" rather than presenting it as reliable); names and @mentions mangled (never translated).

Measure: translation invocation rate, always-on adoption, conversations sustained across a language boundary, and reported translation errors per language pair.

J5 — Catching up on a busy group

1. Open a group with 47 unread messages.

2. A collapsed card offers "Summarise 47 messages." **Never generated automatically** (Part 4 §4.5).
3. Tap → summary appears above the transcript, labelled, collapsible, with each claim linked to its source message.
4. Tap a claim → jump to that message, highlighted.

Failure modes. A summary that misses the one thing that mattered (mitigate with per-claim source links so the user can verify in seconds, and one-tap reporting); on-device model unavailable on Tier C (offer the server path with an explicit grant, and say why).

Measure: summary invocation, source-link tap-through (a proxy for whether people trust it enough to check), and report rate.

J6 — Linking a second device

1. On the new device: "Link to an existing account" → a QR code appears, valid for 60 s.
2. On the phone: Settings → Devices → Link a device → scan.
3. **Both screens show the new device's safety number**; the user confirms they match.
4. Choose history: none / 30 days / everything — **with the transfer size shown** before it starts.
5. **All existing devices receive a notification**, and a permanent entry appears in the device log (Part 5 §5.13).

Failure modes. QR expiry mid-flow (regenerate automatically, no restart); history transfer interrupted (resumable); a device the user does not recognise (the log and the notification are the defence, which is exactly why silent linking is forbidden).

Measure: link completion rate, time to complete, and the rate at which users actually read the device notification — because a security notification nobody reads is not a security control.

J7 — Encountering a scam

1. A message arrives from an unknown sender → held as a **message request**: one preview, no notification sound, no delivery receipt.
2. The on-device classifier flags a known fraud pattern.
3. An inline `danger` strip states the **specific** pattern: "This message asks for a verification code. SpaceTalk never asks for one."
4. The message remains readable. We warn; we do not censor personal communication.
5. One-tap block and report; reporting states exactly what will be shared before it is shared.

Failure modes. A false positive on a legitimate message — the trust catastrophe this feature is most likely to cause. Precision is tracked per pattern class with automatic rollback below 95 % (Part 4 §4.7).

Measure: precision and recall per pattern, block-after-warning rate, and — the number that matters — user-reported fraud losses.

13.5 Information Architecture

```
SpaceTalk
│
├── Chats ──────────────────────────── [primary destination]
│   ├── Filter chips: All · Unread · Groups · Channels
│   ├── Conversation rows (people, groups, channels, assistant — one list, by recency)
│   ├── Archived  (below the fold, never a badge)
│   └── Conversation
│       ├── Transcript
│       ├── Composer  (text · attach · voice · send)
│       ├── Conversation profile
│       │   ├── Members / contact info
│       │   ├── Media · Files · Links
│       │   ├── Disappearing messages
│       │   ├── Encryption + safety number
│       │   └── Mute · Block · Report · Leave
│       └── Search within conversation
│
├── Calls ──────────────────────────── [primary destination]
│   ├── History (all · missed)
│   └── Start a call
│
├── Stories ────────────────────────── [primary destination]
│   ├── Others' stories
│   ├── My story  (+ viewers, visible only to me)
│   └── Create  (audience chosen before posting, every time)
│
└── Settings ───────────────────────── [primary destination]
    ├── Profile  (username · name · photo · bio)
    ├── Privacy  (per-field visibility · read receipts · who can add me · who can call me ·
blocked)
    ├── Devices  (linked devices · link new · device log)
    ├── Notifications  (per type · quiet hours · previews)
    ├── Assistant  (features on/off · grants · memory · Privacy Centre)
    ├── Storage & data  (media cache · auto-download · metered-network behaviour)
    ├── Appearance  (theme · text size · language)
    └── Help & about  (privacy policy · terms · transparency report · version)
```

Global search is available from Chats and spans conversations, messages, media, files, and people (Part 3 §3.8). It is not a destination, because searching is something you do *from* somewhere.

IA rules.

1. **Maximum depth is three levels** from any tab. Settings → Assistant → Memory is the deepest legal path.

2. **Every conversation type is a row in one list.** A channel, a group, a person, and the assistant differ in what they contain, not in where they live. This is the single decision that keeps the app learnable.
3. **Nothing lives in two places.** If it belongs in Settings, there is no shortcut to it from a conversation that creates a second mental model.
4. **Archived is a place, not a state to be badged.** It never contributes to any count.
5. **Adding a destination requires CPO and CDO sign-off** (Part 3 §3.1). The default answer is no.

13.6 Design File Specification

How design artefacts are organised so that the system in Part 7 stays real rather than becoming a folder of stale screens.

Structure:

```
SpaceTalk Design (Figma)
├── 00 Foundations      Tokens as Figma variables — generated, never hand-edited
├── 01 Components       The library in 07 §7.2, every variant, every state
├── 02 Patterns         Composed patterns from 07 §7.10
├── 03 Flows            The journeys in §13.4, as connected prototypes
├── 04 Surfaces         Screen specs, per feature, per breakpoint, both themes, LTR + RTL
├── 05 Motion           Prototypes with real timings from 07 §7.7
├── 06 Explorations     Not a source of truth. Dated. Archived quarterly.
└── 07 Marketing        Brand assets per 01
```

Rules.

- **Tokens flow one way:** JSON source → Figma variables *and* Dart *and* CSS, generated in CI (Part 7 §7.1). A designer who changes a hex value in Figma has changed nothing; they must change the token source.
- **Every screen exists in four versions** — light LTR, light RTL, dark LTR, dark RTL — before it is handed off. Not "we'll check RTL later," because later means never and Arabic is a launch language.
- **Every screen is specified at default and 200 % type scale.**
- **Every interactive element is annotated** with its component name, its states, and its accessibility label.
- **Redlines are not produced by hand.** If a spec cannot be read from tokens and component names, the design is not using the system.
- **A screen not built within one quarter moves to Explorations.** A library full of unbuilt screens is a library nobody trusts.

Appendix A — Decision register

The twelve decisions with lasting consequence, in one view. Full records, with the alternatives considered and the conditions for revisiting each, are in Part 12.

#	Decision	Why	Cost accepted
ADR-001	**Flutter for every client platform**	Two native teams would mean two implementations of every subtle piece of encryption and sync logic — and two places for those bugs to differ.	Flutter's known jank sources become first-class engineering work
ADR-002	**A modular monolith in Go, not microservices**	Module boundaries are enforced by an import linter from day one, so extraction later is mechanical. Deployment complexity is deferred until a scaling profile actually diverges.	Larger blast radius per deploy, mitigated by flags and staged rollout
ADR-003	**Signal Protocol via libsignal; Sender Keys; MLS deferred**	Cryptographic implementation bugs are silent, catastrophic, and fall hardest on the people most in need of protection. We do not write our own.	Sender Keys are O(members): groups capped at 1,000 until MLS in Phase 3
ADR-004	**PostgreSQL for everything; ScyllaDB only on trigger**	Delivered envelopes are deleted, not archived. The hot dataset tracks undelivered traffic, not total history — which keeps Postgres viable far longer than intuition suggests.	A migration is in our future if we succeed; the interface exists to absorb it
ADR-005	**AI never processes encrypted content without a visible grant**	On-device first; server-side only with a per-conversation, revocable, disclosed grant; otherwise the feature does not ship. The assistant conversation is a separate, labelled surface.	Some AI features are worse than a competitor's. We take the hit and say why
ADR-006	**LiveKit SFU with insertable-stream E2EE for group calls**	An MCU would mix server-side, which means plaintext server-side. Rejected outright.	Operating an SFU fleet is real work; server-side recording is impossible
ADR-007	**Push notification payloads carry no content**	The simplest implementation hands the most-read text in the product to Apple and Google.	A network round trip in the notification path, budgeted at p95 under 2 s
ADR-008	**Local-first architecture with an outbox**	A request/response client is unusable on an unreliable network, and offline support bolted on afterwards never really works.	Local schema migrations become critical; every one is tested against real histories

#	Decision	Why	Cost accepted
ADR-009	**No advertising, ever; subscription and business revenue only**	An ad surface creates an incentive to build more inventory, and that incentive never reverses. It constrains architecture: no ad serving, no profiling pipeline, no engagement ranking.	Slower revenue ramp; free users cost money; unit economics are a product constraint
ADR-010	**No address-book upload; username-first discovery**	Hashing a 10–15 digit number space provides essentially no protection. We decline to hold a social graph we promised not to build.	Materially slower viral growth — the largest deliberate growth cost in the product
ADR-011	**SpaceTalk is greenfield, not an extension of StromeX**	Python's concurrency model is a poor fit for hundreds of thousands of persistent sockets, and retrofitting a request/response codebase into a realtime one is usually slower than starting clean.	Two stacks in one organisation. Open question if SpaceTalk is a rename
ADR-012	**Per-device identity keys; web as the MVP linked client**	A tethered companion device dies with the phone's battery; a shared identity key means one compromised device compromises all of them.	Fan-out is O(recipients × devices); history sync is explicit and user-controlled

All twelve are Accepted at version 1.0. None has been superseded. An ADR is never edited after acceptance — a changed decision becomes a new record that supersedes the old one, so the reasoning behind a reversal stays legible.

Appendix B — The numbers, consolidated

Every budget and target that a release is measured against, in one place. Device tiers are defined in Part 8 §8.7: Tier A flagship, Tier B mainstream, Tier C entry-level — and Tier C is the design target, not the fallback.

PERFORMANCE BUDGETS — MEASURED IN CI, ON PHYSICAL HARDWARE, PER PULL REQUEST

Budget	Tier A	Tier B	Tier C	Source
Cold launch to interactive (p95)	< 450 ms	< 800 ms	**< 1,500 ms**	Part 8 §8.2
Frame rate floor	60 fps	60 fps	**60 fps**	Part 8 §8.3
Dropped frames, 10,000-message scroll	< 0.5 %	< 0.5 %	**< 0.5 %**	Part 8 §8.3
Touch → first visual response	< 100 ms	< 100 ms	**< 100 ms**	Part 8 §8.4
Send tap → bubble visible	< 50 ms	< 50 ms	**< 50 ms**	Part 8 §8.4
Send → delivered (p50 / p95)	250 / 700 ms	250 / 700 ms	**250 / 700 ms**	Part 8 §8.4
Search keystroke → local results	< 50 ms	< 50 ms	**< 50 ms**	Part 8 §8.4
Call initiate → callee ringing (p75)	< 1.2 s	< 1.2 s	**< 1.2 s**	Part 8 §8.4
Memory, idle	< 120 MB	< 100 MB	**< 80 MB**	Part 8 §8.5
Memory, peak	< 400 MB	< 350 MB	**< 260 MB**	Part 8 §8.5
Battery, idle connected	< 1 %/h	< 1 %/h	**< 1 %/h**	Part 8 §8.6
Battery, video call	< 12 %/h	< 12 %/h	**< 12 %/h**	Part 8 §8.6
Daily background data, idle account	< 1 MB	< 1 MB	**< 1 MB**	Part 8 §8.6
Crash-free sessions	> 99.9 %	> 99.9 %	**> 99.9 %**	Part 8 §8.10
Message delivery success	100 %	100 %	**100 %**	Part 8 §8.10

PRODUCT AND GOVERNANCE TARGETS

Target	Value	Why it is set there	Source
On-device AI execution rate	**> 85 % of all invocations**	the privacy promise made measurable	Part 5 §5.8
Ungranted AI on encrypted content	**zero, audited continuously**	any non-zero result is a Sev-1 incident	Part 5 §5.8
Scam-warning precision	**> 95 % per pattern class**	automatic rollback below the threshold	Part 4 §4.7
Notification opt-out rate	**< 10 %**	the clearest measure of the calm promise	Part 5 §5.12
Notifications from non-humans	**zero, verified per release**	audited across every notification type	Part 5 §5.12
D30 retention (Phase 1 gate)	**≥ 40 %**	not signups — the gate is retention	Part 9
Subscription conversion (Phase 2 gate)	**≥ 3 %**	with positive contribution margin	Part 9
Accessibility automated checks	**100 % pass, zero suppressions**	a launch requirement, not a Phase 2 item	Part 8 §8.9
Groups at MVP	**≤ 1,000 members**	derived from Sender Keys, not arbitrary	Part 5 §5.5
Linked devices at MVP	**≤ 4**	each an independent cryptographic identity	Part 5 §5.13
File size, free / Plus	**2 GB / 10 GB**	stated up front, not discovered at 99 %	Part 11 §11.2
Creator take rate	**10 %**	against a market standard of 30 %	Part 11 §11.3

A budget that is not measured in CI does not exist. Exceeding one requires an explicit trade — something else gives up an equivalent amount, recorded in the pull request — so budgets do not inflate quietly.

Appendix C — Glossary

Terms as they are used in this document. Where a term has a governing definition, the section reference is authoritative and this entry is a summary of it.

Term	As used here	Governing section
Aurora	The intelligence colour. Reserved absolutely for assistant output, so a reader can tell at a glance whether a person or a model produced something.	Part 2 §2.3
Callout panel	A tinted panel marking a passage that changes what you should do, not merely what you should know.	—
Channel	A one-to-many broadcast a subscriber chooses to receive, delivered chronologically to 100 % of subscribers. Not end-to-end encrypted, and labelled as such.	Part 5 §5.6
Conversation	The atomic unit of the product. A person, a group, a channel and the assistant are all rows in one list, differing in what they contain rather than where they live.	Part 3 §3.1
Device tier	A, B or C — flagship, mainstream, entry-level. Every performance budget is stated per tier, and Tier C is the design target.	Part 8 §8.7
Double Ratchet	The message-encryption ratchet providing forward secrecy and post-compromise security, used via libsignal for 1:1 messages.	Part 6 §6.7
Envelope	The routed unit the server stores: conversation, sender device, recipient device, sequence, ciphertext. The server can see who talked to whom and when, and nothing else.	Part 6 §6.4
Grant	An explicit, per-conversation, revocable permission for server-side AI processing, shown in the header and disclosed to every participant.	Part 4 §4.1
Local-first	The on-device database is the source of truth for the interface; the network updates it in the background rather than serving it.	Part 6 §6.8
MLS	Messaging Layer Security, RFC 9420. The group protocol that lifts the 1,000-member ceiling, scheduled for Phase 3.	Part 6 §6.7
Natural zone	The bottom 40 % of a phone screen, reachable by thumb without a regrip. Every high-frequency action lives there; no destructive action does.	Part 3 §3.12
Orbit	The brand action colour, hue ≈ 231°. One accent, so “what is the primary action here?” is answerable without thought.	Part 2 §2.2

Term	As used here	Governing section
Outbox	The durable, ordered, idempotent queue of local mutations that drains when connectivity returns. A failed send blocks only its own conversation.	Part 6 §6.8
Run	Consecutive messages from one sender within 60 seconds, grouped with tightened spacing so a burst reads as one utterance.	Part 2 §2.10
Sender Keys	The group-messaging scheme in which each sender holds a per-group chain key, rotated on every membership removal. O(members) on distribution — hence the 1,000-member cap.	Part 6 §6.7
SFrame	Frame-level encryption for real-time media, letting the SFU forward frames it cannot decrypt.	Part 6 §6.9
SFU	Selective Forwarding Unit. Routes media streams for group calls without mixing them — mixing would require server-side plaintext.	Part 6 §6.9
Standfirst	The italic line under a part title stating what that part governs.	—
Token tier	Primitive, semantic or component. Screens reference tier 3 only; reaching past it is a bug.	Part 7 §7.1
Void	The neutral scale, cool-tinted so it sits with Orbit rather than fighting it. The interface is 90 % neutral.	Part 2 §2.5
X3DH	Extended Triple Diffie-Hellman — the asynchronous key agreement that lets a first message be encrypted to a recipient who is offline.	Part 6 §6.7

Index

Alphabetical, by subject. Each entry gives the governing section and its page. Section numbers are stable across editions; page numbers are for this one.

D

E

F

G

H

I

J

K

L

M

N

O

P

R

S

T

U

V

W

Section number first, then the page it begins on. Every reference in this index is a live link in both editions.

END OF DOCUMENT

SpaceTalk Editorial Bible, Version 1.0.

Fourteen parts, four appendices, twenty figures, twelve decision records.

> “Room to talk.”